PlanB. Trusted Agentic Operator

DHBW Duale Hochschule Baden-Württemberg Heidenheim

WEB-PROGRAMMIERUNG · EINHEIT 0

Web-Programmierung 2026 – Willkommen

Dozent



Jens Reiner

Team Lead - PlanB. GmbH

Digital Product Factory – Engineering Frontier

Bachelor – Software Engineering Universität Ulm

https://www.linkedin.com/in/jens-reiner/

In 17 Einheiten entsteht das Backend der Beer App

Einheiten	Block	Themen
1–4	Grundlagen des Webs	Verteilte Systeme, Client-Server-Modell, HTTP, JSON
5–9	Backend mit NestJS	erstes Projekt, Services und Dependency Injection, Validierung mit Zod, Fehlerbehandlung, Module
10–12	API und Daten	Anforderungen in Code übersetzen, REST-Design und OpenAPI, Datenbank-Anbindung
13–14	Sicherheit	Authentifizierung, Autorisierung
15–16	Über JSON hinaus	WebSockets, serverseitige Erzeugung von HTML
17	Rückblick	Architektur und Ausblick

Jede Einheit beginnt mit einem Auftrag der regionalen Brauereien

- Die Vorlesung zeigt das Konzept im Live-Coding an den Brauereien.
- In der Übung setzt du es an Bieren und Bewertungen um.
- Das Skript enthält die Aufgaben, gestufte Hinweise und typische Fehler.
- Mit der Collection in Postman prüfst du dein Ergebnis nach der Übung.

MERKSATZ

Ab Einheit 5 gibt es je Einheit einen Lösungsstand. Wer eine Einheit auslässt, arbeitet mit ihm weiter.

Das Paket enthält alle Unterlagen und Lösungen auf einmal

Unterlage	Inhalt	Öffnen mit
web-programmierung.postman_collection.json	Selbstkontrolle, ein Ordner je Einheit	Postman
beer-api/	Basisprojekt und Lösungsstände einheit-05 bis einheit-17	VS Code
ctf-server/	Server für das Capture the Flag	VS Code
Glossar, Anforderungen, material/	Begriffe, Anforderungen der Beer App, Dashboard	VS Code und Chrome

Alle Unterlagen liegen in einem Repository auf GitHub

- https://github.com/wasdJens/dhbw-reiner-wi-hdh
- **Code → Download ZIP** lädt alle Unterlagen als ZIP-Datei herunter.
- Alternativ klonst du das Repository mit `git clone` und dem Link

Für die Übungen brauchst du sechs Programme

Programm	Wofür	Ab Einheit
Chrome (google.com/chrome)	DevTools mit Network-Tab	1
Postman (postman.com/downloads)	Anfragen senden, Collection ausführen	3
Node.js 24 (nodejs.org)	CTF-Server und Beer API starten	3
VS Code (code.visualstudio.com)	Code der Beer API bearbeiten	5
Git (git-scm.com)	eigene Commits im Projekt	5

Ein Projekt öffnest du in VS Code als Ordner

1. Wähle in VS Code **File → Open Folder…** und den Ordner beer-api/basisprojekt.
2. Öffne ein Terminal mit **Terminal → New Terminal**.
3. Installiere die Pakete mit npm install.
4. Starte den Server mit npm run start:dev; er antwortet unter http://localhost:3000.
5. Den CTF-Server startest du ebenso, mit npm start und unter Port 3001.
6. Ab Einheit 12 nennt die README des Projekts weitere Schritte vor dem Start.

Die Collection importierst du einmal in Postman

1. Klicke in Postman auf **Import** und wähle web-programmierung.postman_collection.json.
2. Starte den Server der Einheit: Beer API oder CTF-Server.
3. Wähle in der Collection den Ordner der Einheit und klicke auf **Run**.
4. Ein roter Test nennt die Anfrage, deren Antwort von der Aufgabe abweicht.

MERKSATZ

Die Anfragen der Collection sind die Lösung. Starte einen Ordner erst nach der Übung.

Einheit 0 · Willkommen

FRAGE

Welche Fragen und Anmerkungen gibt es?

WEB-PROGRAMMIERUNG · EINHEIT 1

Verteilte Systeme

Einheit 1 · Verteilte Systeme

FRAGE

Was kostet eine Stunde Ausfall des Amazon-Shops?

Am 20. Oktober 2025 fiel bei AWS ein Datenbankdienst aus; Snapchat, Netflix und andere Apps waren stundenlang gestört. Die regionalen Brauereien entscheiden, welche Teile der Beer App wie verfügbar sein müssen.

Aufgabe: ein bekanntes System in seine Teile zerlegen

- Du wählst einen Streaming-Dienst, einen Online-Shop oder einen Messenger.
- Du zeichnest Clients, Server, den Ort der Daten und fremde Dienste.
- Jede Verbindung ist eine Nachricht mit Richtung.
- Für drei Teile hältst du fest, was bei ihrem Ausfall weiterläuft.
- Die Übung ist Partnerarbeit, auf Papier oder in draw.io.

MERKSATZ

Jeder Kasten in der Skizze ist ein Teil, der allein ausfallen kann.

Der Amazon-Shop setzt je Stunde rund 60 Mio. Dollar um

- Nettoumsatz 2024 ohne AWS: 530,4 Mrd. US-Dollar
- 2024 hat 366 Tage, also 8 784 Stunden
- 530,4 Mrd. ÷ 8 784 h ≈ 60 Mio. US-Dollar je Stunde
- Ein Ausfall kostet weniger, wenn Kunden später kaufen, und mehr durch Folgekosten.

MERKSATZ

Die Rechnung ist ein Durchschnitt. Zur Hauptverkaufszeit ist eine Stunde mehr wert, nachts weniger.

Jede weitere Neun der Verfügbarkeit teilt die erlaubte Ausfallzeit durch zehn

Verfügbarkeit	Kurzform	Ausfall im Jahr	Ausfall im Monat (30 Tage)
99 %	zwei Neunen	3 Tage 15 Stunden 36 Minuten	7 Stunden 12 Minuten
99,9 %	drei Neunen	8 Stunden 45 Minuten 36 Sekunden	43 Minuten 12 Sekunden
99,99 %	vier Neunen	52 Minuten 34 Sekunden	4 Minuten 19 Sekunden
99,999 %	fünf Neunen	5 Minuten 15 Sekunden	26 Sekunden

Ein verteiltes System besteht aus eigenständigen Teilen und erscheint als ein System

- Jeder Teil ist ein Rechner oder Prozess mit eigenem Speicher und eigener Uhr.
- Jeder Teil kann allein ausfallen.
- Die Teile arbeiten nur über Nachrichten zusammen, ohne gemeinsamen Speicher.
- Die Nutzer sehen eine einzige Anwendung.

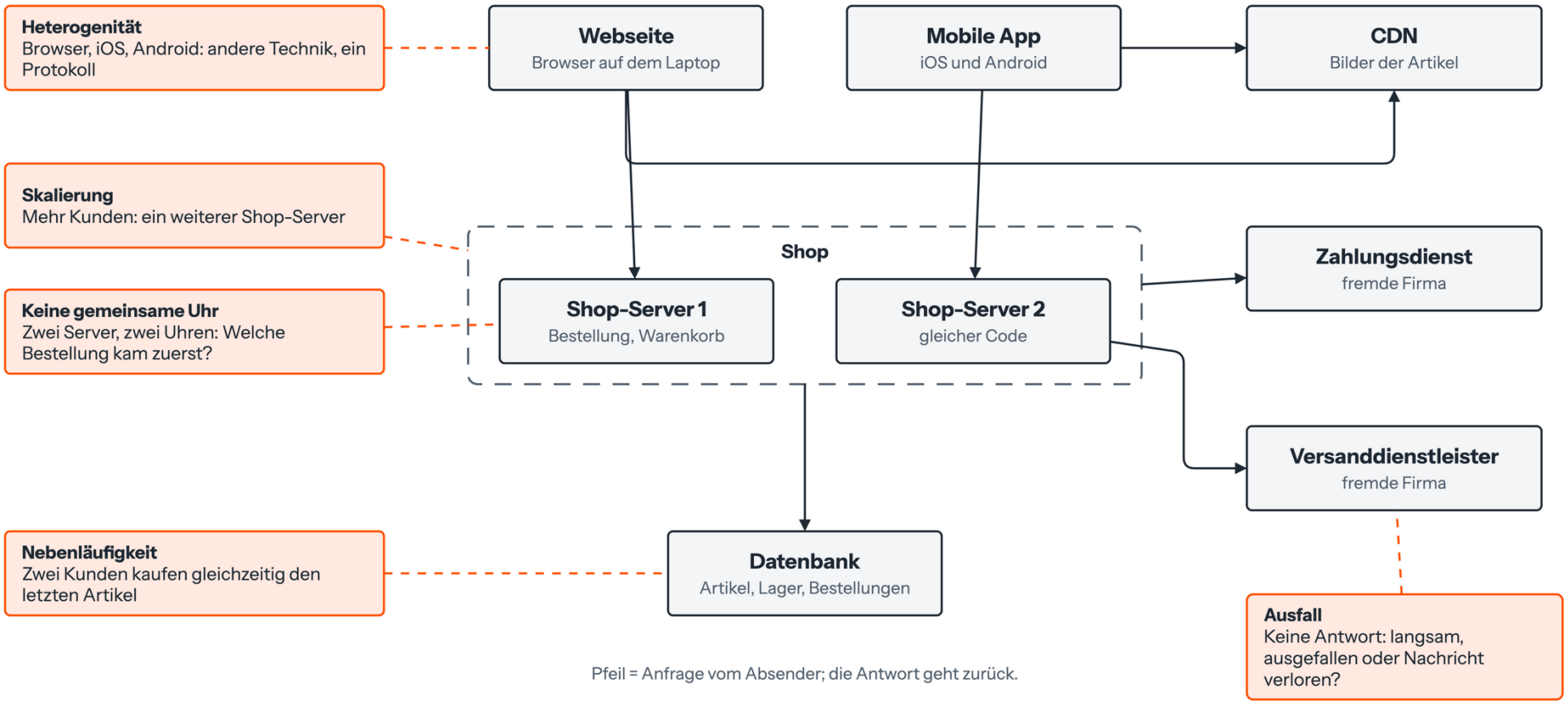
MERKSATZ

„A distributed system is a collection of autonomous computing elements that appears to its users as a single coherent system.“ (van Steen, Tanenbaum)

Web-Anwendungen sind verteilt, weil Nutzer und Daten getrennt sind

- Die Nutzer sitzen an vielen Orten, mit eigenen Geräten.
- Die Daten liegen an einer Stelle und sind für alle gleich.
- Die Last verteilt sich auf mehrere Server.
- Fremde Dienste übernehmen Zahlung, Versand oder Bilder.

Im Online-Shop entsteht jede Herausforderung an einem bestimmten Teil



Webseite und Mobile App senden Anfragen an zwei Shop-Server und laden die Bilder von einem CDN. Die Server senden Anfragen an eine Datenbank, einen Zahlungsdienst und einen Versanddienstleister. Orange Karten zeigen, wo Heterogenität, Skalierung, fehlende gemeinsame Uhr, Nebenläufigkeit und Ausfall entstehen.

Fünf Herausforderungen entstehen, weil die Teile getrennt sind

Herausforderung	Bedeutung	Beispiel im Shop
Heterogenität	Teile laufen auf verschiedener Technik	Browser, iOS und Android brauchen ein gemeinsames Protokoll
Ausfall	Ein Teil fällt aus, der Rest läuft weiter	Der Versanddienst antwortet nicht, Bestellungen gehen weiter ein
Keine gemeinsame Uhr	Jeder Rechner hat eine eigene, abweichende Uhr	Zwei Server: Welche Bestellung kam zuerst?
Nebenläufigkeit	Viele Clients greifen gleichzeitig zu	Zwei Kunden kaufen den letzten Artikel
Skalierung	Mehr Nutzer brauchen mehr Teile	Ein weiterer Shop-Server für das Weihnachtsgeschäft

Transparenz verbirgt die Verteilung vor den Nutzern

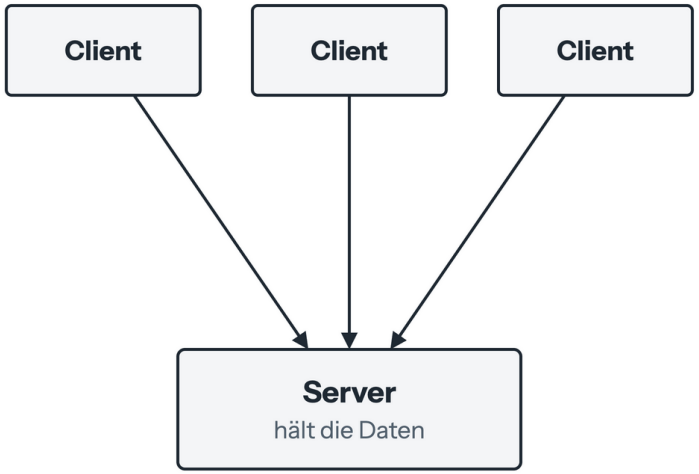
Transparenz	Die Nutzer merken nicht, …
Zugriff	wie die Daten dargestellt sind und wie man auf einen Teil zugreift
Ort	an welchem Standort ein Teil läuft
Migration	dass ein Teil umzieht
Replikation	dass es mehrere Kopien eines Teils gibt
Nebenläufigkeit	dass andere Nutzer denselben Teil gleichzeitig nutzen
Ausfall	dass ein Teil ausfällt und wiederhergestellt wird

Vollständige Transparenz ist nicht erreichbar

- Ein langsames Netz bleibt für die Nutzer spürbar.
- Ein Ausfall lässt sich nicht immer verbergen.
- Ein System, das jeden Ausfall verbergen will, wartet im Zweifel endlos.
- Deshalb meldet eine API dem Client, was schiefging.

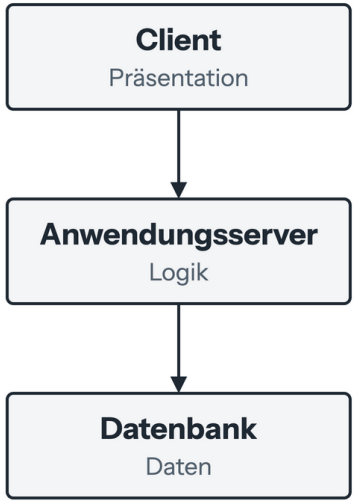
Drei Architekturmodelle ordnen die Teile unterschiedlich an

Client-Server



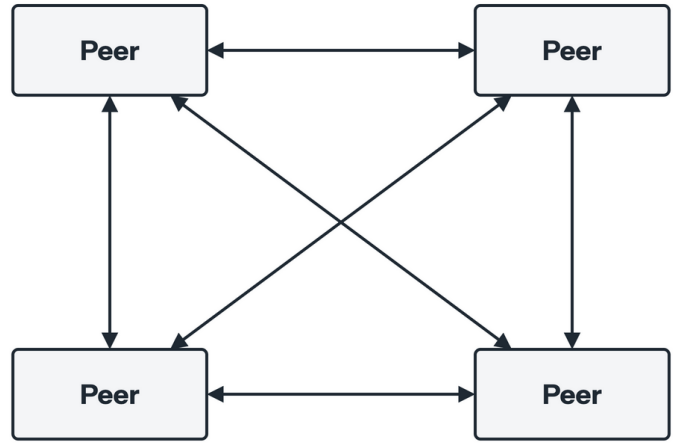
Viele Clients fragen einen Server, der Server antwortet. Pfeil = Anfrage.

Mehrschichten



Jede Schicht läuft auf eigenen Rechnern und skaliert einzeln.

Peer-to-Peer



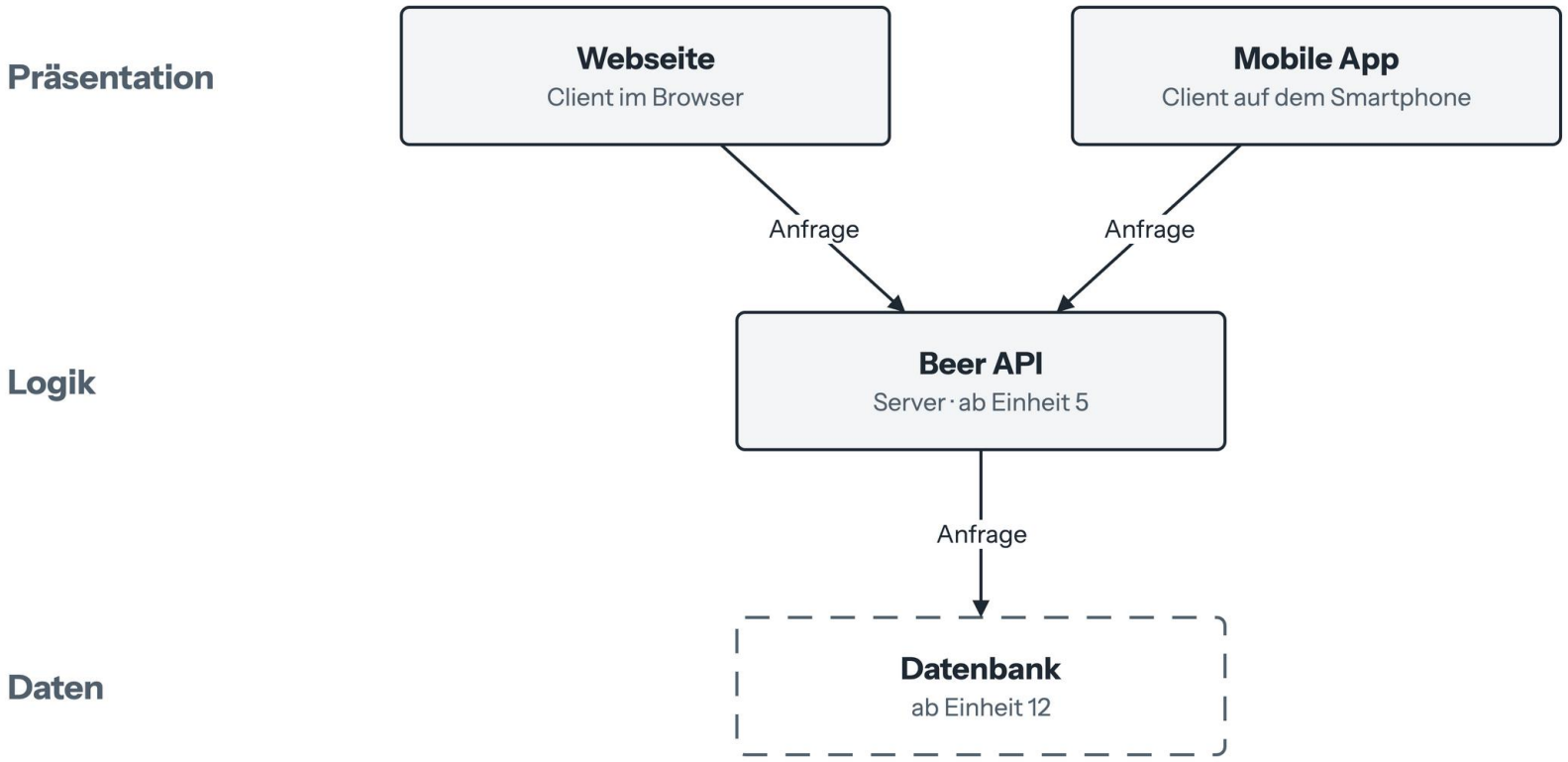
Jeder Peer ist Client und Server. Es gibt keinen zentralen Teil.

Links Client-Server: drei Clients senden Nachrichten an einen Server. Mitte Mehrschichten: Client, Anwendungsserver und Datenbank übereinander. Rechts Peer-to-Peer: vier Peers, jeder mit jedem verbunden.

Jedes Modell hat eine Stärke und eine Schwäche

Modell	Aufbau	Stärke	Schwäche
Client-Server	Clients fragen, ein Server hält die Daten	Daten an einer Stelle, zentral pflegbar	Der Server ist ein zentraler Ausfallpunkt
Mehrschichten	Präsentation, Logik und Daten auf eigenen Rechnern	Jede Schicht skaliert einzeln	Mehr Teile, mehr Nachrichten
Peer-to-Peer	Jeder Teil ist Client und Server	Kein zentraler Ausfallpunkt	Niemand kann Daten zentral ändern oder löschen

Die Beer App folgt der Mehrschichtenarchitektur mit drei Schichten



Pfeil = Anfrage vom Absender; die Antwort geht zurück.

Webseite und Mobile App senden Anfragen an die Beer API, die Beer API an die Datenbank. Die Beer API entsteht ab Einheit 5, die Datenbank ab Einheit 12.

Fällt die Beer API aus, zeigen Webseite und App keine Daten

Teil fällt aus	Folge
Webseite	Die Mobile App läuft weiter.
Mobile App	Die Webseite läuft weiter.
Beer API	Webseite und App zeigen keine Daten.
Datenbank	Die Beer API antwortet mit einem Fehler.

Web-Technologien geben verteilten Anwendungen ein gemeinsames Protokoll und fertige Bausteine

- HTTP verbindet Teile auf verschiedener Technik und löst so die Heterogenität.
- Erprobte Server und Bibliotheken gibt es für jede Sprache.
- Viele Anbieter stellen Hosting und fremde Dienste bereit.
- Ausfall, Uhr und Nebenläufigkeit bleiben Aufgabe der Anwendung.
- Wenig Sinn ergeben sie bei Steuerung in Echtzeit und minimaler Latenz.

Live-Demo: Der Network-Tab zeigt, woraus eine Seite besteht

1. https://de.wikipedia.org/wiki/Verteiltes_System öffnen
2. DevTools öffnen, Tab Network, Disable cache einschalten
3. Seite neu laden, Spalten Name, Status und Type lesen
4. Für die Live-Fragen: über More tools den Drawer Request conditions öffnen
5. Muster mit Add condition eintragen, Enable blocking and throttling einschalten
6. Seite neu laden und die rot markierten Requests lesen

Einheit 1 · Verteilte Systeme

FRAGE

Live-Frage: Von wie vielen Hosts lädt der Wikipedia-Artikel?

Spalte Domain im Network-Tab, Disable cache an, neu laden

Auflösung: fünf Hosts, weil getrennte Server getrennte Aufgaben haben

BEISPIEL

- Text, Gestaltung und Skripte kommen von de.wikipedia.org.
- Symbole und Logos liegen auf upload. und thumb.wikimedia.org.
- Skripte für Werkzeuge kommen von meta. und auth.wikimedia.org.
- Für die Nutzer ist das transparent: Sie sehen einen Artikel.

Domain	Requests
de.wikipedia.org	29
upload.wikimedia.org	2
thumb.wikimedia.org	1
meta.wikimedia.org	1
auth.wikimedia.org	1
(ohne Domain, data:)	3

Einheit 1 · Verteilte Systeme

FRAGE

Live-Frage: Ich blockiere alle Hosts von wikimedia.org. Was zeigt die Seite?

Request conditions: *://*.wikimedia.org/*

Auflösung: Der Artikel ist vollständig, weil der Text von einem anderen Host kommt

BEISPIEL

- Fünf Requests stehen rot im Network-Tab.
- Es fehlen ein Symbol im Artikel, zwei Logo-Dateien und ein Skript.
- Der Server für den Text antwortet weiter.
- Die Seite zeigt den Text ohne die blockierten Dateien.

```
blockiert (blocked:devtools)
thumb.wikimedia.org   Disambig-dark.svg (2×)
upload.wikimedia.org  Wikipedia-logo-v2-25-alt.svg
upload.wikimedia.org  Wikipedia-tagline-de-25.svg
meta.wikimedia.org    Wikiminiatlas.js

geladen
de.wikipedia.org      29 Requests, ganzer Text
```

Einheit 1 · Verteilte Systeme

FRAGE

Live-Frage: Ich blockiere nur load.php. Was zeigt die Seite?

Request conditions: *://de.wikipedia.org/w/load.php*

Auflösung: Der Inhalt bleibt, die Gestaltung fehlt, weil load.php CSS und Skripte liefert

BEISPIEL

- Die Seite erscheint als Text ohne CSS.
- Logo, Menü und Artikel sind weiter da.
- Auch ein Teil desselben Servers fällt einzeln aus.
- Jeder Request ist eine eigene Nachricht.

Requests gesamt	9	(statt 37 bis 42)
blockiert	4	load.php
geladen	5	de.wikipedia.org
CSS-Regeln aus Inline-CSS	1	(statt über 700)

Übung

Die Aufgaben stehen vollständig im Skript

Aufgaben dieser Einheit

ÜBUNG

1. Aufgabe 1.1: ein System in seine Teile zerlegen
2. Aufgabe 1.2: den Ausfall von drei Teilen durchspielen
3. Zusatz 1.3: die Verfügbarkeit einer Kette berechnen

SELBSTKONTROLLE

- Aufgaben ab Skript 1.9, Hinweise ab 1.13
- Musterskizze nach dem Review: Skript 1.17

Review: zwei Skizzen im Vergleich

- Zwei Paare zeigen ihre Skizzen.
- Was ist gleich, was ist anders?
- Welcher Ausfall legt am meisten still?
- Welcher Teil braucht deshalb die meisten Neunen?

MERKSATZ

Einheit 2 arbeitet an der Skizze weiter: Welche Teile baut man selbst, welche kauft man ein?

Das solltest du jetzt können

- ✓ Ein verteiltes System nach van Steen und Tanenbaum erklären
- ✓ Fünf Herausforderungen verteilter Systeme an einem Beispiel nennen
- ✓ Client-Server, Mehrschichten und Peer-to-Peer unterscheiden
- ✓ Ein System skizzieren und die Folge eines Ausfalls benennen
- ✓ Eine Verfügbarkeit in Ausfallzeit pro Jahr umrechnen

WEB-PROGRAMMIERUNG · EINHEIT 2

Client-Server-Modell und der Browser

Einheit 2 · Client-Server-Modell und der Browser

FRAGE

Hosting, E-Mail, Anmeldung: Was kaufen die regionalen Brauereien für die Beer App ein?

Die regionalen Brauereien bezahlen jeden Teil der Beer App, ob selbst gebaut oder eingekauft. Die Skizze aus Einheit 1 zeigt, welche Teile es gibt.

Aufgabe: drei Seiten im Network-Tab vermessen

- Du misst einen Wikipedia-Artikel, ein GitHub-Repository und einen Online-Shop.
- Vor dem Laden schätzt du die Zahl der Requests.
- Bei GitHub vergleichst du das Timing zweier Requests.
- Die Übung ist Partnerarbeit mit Chrome.

MERKSATZ

Jede Zeile im Network-Tab steht für eine Datei, die die Seite braucht. Die Spalte Size zeigt, ob sie vom Server oder aus dem Cache kam.

Die Brauereien kaufen ein, was die Beer App nicht von anderen unterscheidet

Teil der Beer App	Entscheidung	Grund
Hosting der Server	kaufen	Betrieb rund um die Uhr ist das Geschäft des Anbieters.
E-Mail-Versand	kaufen	Zustellung ohne Spam-Ordner verlangt Erfahrung und einen guten Ruf als Absender.
Anmeldung	kaufen oder bauen	Ein Identity Provider nimmt dem Betreiber die Passwörter ab.
Beer API mit Bieren, Bewertungen und Regeln	bauen	Diese Daten und Regeln bietet kein Anbieter an.

Client und Server sind Rollen, die ein Programm in einer Verbindung einnimmt

Begriff	Bedeutung	In der Beer App
Client	beginnt die Kommunikation und sendet Requests	Webseite, Mobile App, Postman
Server	wartet auf Requests und beantwortet jeden mit einer Response	Beer API
Request und Response	ein Paar; ohne Request sendet der Server nichts	Die Webseite fragt die Biere ab, die Beer API liefert die Liste.
Server als Client	Ein Server fragt selbst einen anderen Server.	Die Beer API fragt die Datenbank.

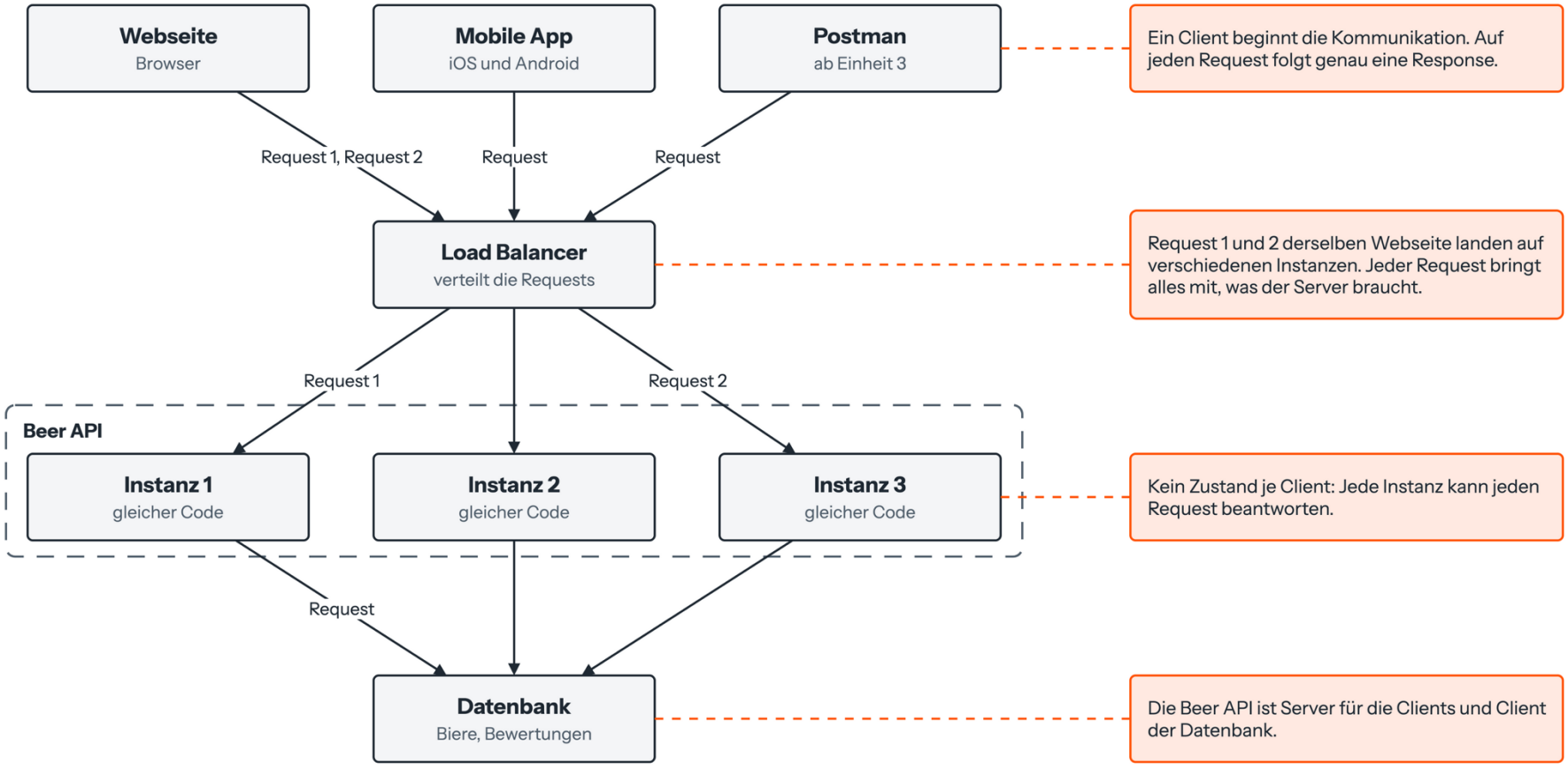
Ein zustandsloser Server merkt sich nichts über den einzelnen Client

- Jeder Request enthält alles, was der Server für die Response braucht.
- Beim zweiten Request weiß der Server nicht, wer eben schon gefragt hat.
- Die Daten der Anwendung, etwa die Biere, hält der Server trotzdem.
- Eine Anmeldung bringt der Client bei jedem Request als Nachweis mit.

MERKSATZ

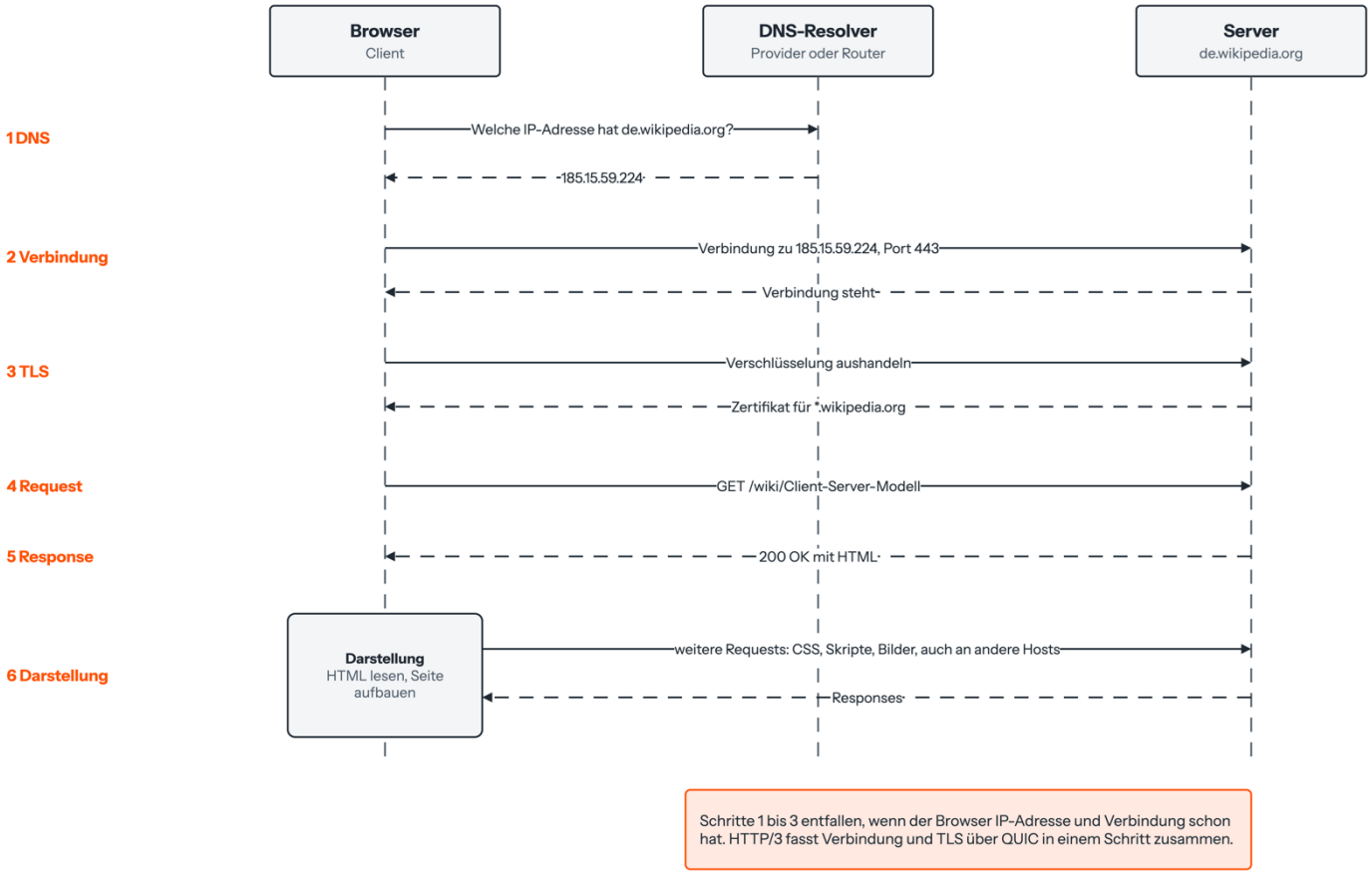
Der Server speichert zwischen zwei Requests keine Daten über den Client.

Ohne Zustand je Client kann jede Instanz jeden Request beantworten



Webseite, Mobile App und Postman senden Requests an einen Load Balancer. Er verteilt sie auf drei Instanzen der Beer API mit gleichem Code; Request 1 und 2 der Webseite landen auf Instanz 1 und Instanz 3. Alle Instanzen fragen dieselbe Datenbank.

Der Aufruf einer URL besteht aus sechs Schritten



Sequenz zwischen Browser, DNS-Resolver und dem Server de.wikipedia.org: 1 DNS übersetzt den Namen in 185.15.59.224, 2 Verbindung zu Port 443, 3 TLS mit Zertifikat für *.wikipedia.org, 4 Request GET /wiki/Client-Server-Modell, 5 Response 200 OK mit HTML, 6 Darstellung mit weiteren Requests für CSS, Skripte und Bilder, auch an andere Hosts.

Der Timing-Tab zeigt die Schritte 1 bis 5 eines Requests einzeln

Schritt	Eintrag im Timing-Tab	Was dort passiert
1 DNS	DNS Lookup	Name in IP-Adresse übersetzen
2 Verbindung	Initial connection	Verbindung zu IP-Adresse und Port aufbauen
3 TLS	SSL, ein Teil von Initial connection	verschlüsseln und Zertifikat prüfen
4 Request	Request sent	Request senden
5 Response	Waiting for server response, Content Download	auf die Response warten, Inhalt laden
6 Darstellung	kein Eintrag	Jede weitere Datei hat eine eigene Zeile im Network-Tab.

Der Browser ist ein HTTP-Client, der die Response zusätzlich darstellt

- Das HTML verweist auf CSS, Skripte, Bilder und Schriften.
- Jede Datei fordert der Browser mit einem eigenen Request an, oft von anderen Hosts.
- Skripte fordern mit Fetch weitere Daten an, ohne die Seite neu zu laden.
- Postman, curl und Mobile Apps senden dieselben Requests ohne Darstellung.
- Der Server beantwortet jeden Request nach denselben Regeln, egal welcher Client ihn sendet.

Der Network-Tab zeigt jeden Request als Zeile mit Status, Type und Initiator

Spalte	Bedeutung	Beispiel
Name	Pfad oder Datei des Requests	Client-Server-Modell
Status	Statuscode; erste Ziffer: 2 Erfolg, 3 Umleitung, 4 Fehler des Clients, 5 Fehler des Servers	200, 302, 404
Type	Art des Inhalts	document, stylesheet, script, png, fetch
Initiator	Auslöser des Requests	Other für die Adresszeile, sonst Datei und Zeile
Size	übertragene Größe oder Herkunft aus dem Cache	Größe in kB, (memory cache), (disk cache)
Time	Dauer bis zur vollständigen Response	Millisekunden

Live-Demo: die Schritte eines Requests im Network-Tab

1. Neuen Tab öffnen, DevTools öffnen, Tab Network.
2. Preserve log und Disable cache einschalten.
3. https://de.wikipedia.org/wiki/Client-Server-Modell laden.
4. Ersten Request anklicken und den Tab Timing lesen.
5. Die Spalte Initiator lesen.
6. Das Zertifikat über das Symbol links in der Adresszeile öffnen.

Der erste Request zeigt DNS, Verbindung, TLS und die Wartezeit auf den Server

BEISPIEL

- Initiator Other: Die Adresszeile hat den Request ausgelöst.
- Die Verbindung kostet mehr Zeit als die DNS-Anfrage.
- Das Zertifikat gilt für jeden Host eine Ebene unter wikipedia.org.
- Die Werte schwanken je Lauf und Netz.

```
Client-Server-Modell  200  document  Other  h2

DNS Lookup                8 ms
Initial connection       45 ms
  SSL                    24 ms
Request sent              0.1 ms
Waiting for server response   319 ms

Zertifikat  *.wikipedia.org
von         Let's Encrypt (YE2)
gültig bis  2. Januar 2027
```

Einheit 2 · Client-Server-Modell und der Browser

FRAGE

Live-Frage: Ich zeige mit der Maus auf einen Link. Geht ein Request ans Netz?

Filter Fetch/XHR, Maus auf den Link „Server“ im ersten Absatz, ohne Klick

Auflösung: ja, ein Fetch mit JSON, weil die Vorschau nur Daten lädt

BEISPIEL

- Ein Skript fragt eine API von Wikipedia.
- Die Seite wird dabei nicht neu geladen.
- Die Response ist JSON mit Titel, Kurztext und Bild.
- Das Skript baut daraus die Vorschau.

```
GET /api/rest_v1/page/summary/Server
Status  200   Type  fetch
Content-Type: application/json; charset=utf-8

{"title": "Server",
 "description": "Computer für den Zugriff auf …",
 "extract": "In der Informatik ist ein Server …",
 "thumbnail": {"source": "https://thumb.wikimedia…"}}
```

Einheit 2 · Client-Server-Modell und der Browser

FRAGE

Live-Frage: Was steht als erster Eintrag bei http://www.dhbw.de?

Adresse mit http eintippen; Preserve log an

Auflösung: 307 Internal Redirect, weil Chrome http selbst auf https umstellt

BEISPIEL

- Den ersten Eintrag erzeugt Chrome selbst.
- Erst der zweite Request geht über TLS an den Server.
- Der Server leitet / auf /startseite um.
- Ohne Browser antwortet der Server auf http mit 301.

```
Name                        Status
http://www.dhbw.de/         307 Internal Redirect
https://www.dhbw.de/        307
https://www.dhbw.de/startseite  200

curl -I http://www.dhbw.de
HTTP/1.1 301 Moved Permanently
```

Einheit 2 · Client-Server-Modell und der Browser

FRAGE

Live-Frage: Wie viele Requests gehen beim Neuladen ans Netz?

Client-Server-Modell, Disable cache aus, einmal laden, dann neu laden

Auflösung: 3 von 25, weil der Browser Kopien aus seinem Cache nimmt

BEISPIEL

- 22 Zeilen zeigen in Size einen Cache statt einer Größe.
- Ans Netz gehen nur das Dokument und zwei Skripte.
- Die übrigen Anfragen erreichen den Server nie.
- Andere Server bestätigen eine Kopie mit 304 (Einheit 3).

```
Neuladen, 25 Requests
(memory cache)      11
(disk cache)        11
Netz, Status 200    3   Dokument, zwei Skripte
```

Übung

Die Aufgaben stehen vollständig im Skript

Aufgaben dieser Einheit

ÜBUNG

1. Aufgabe 2.1: drei Seiten im Network-Tab vermessen
2. Aufgabe 2.2: das Timing zweier Requests vergleichen
3. Zusatz 2.3: Daten-Requests finden und Make or Buy zuordnen

SELBSTKONTROLLE

- Aufgaben ab Skript 2.8, Hinweise ab 2.14
- Messwerte zum Vergleich nach dem Review: Skript 2.18

Review: drei Seiten im Vergleich

- Zwei Paare zeigen ihre Tabellen.
- Wo lag die Schätzung am weitesten daneben?
- Welche Seite lädt von den meisten Hosts?
- Welche fremden Hosts sind eingekaufte Teile?

MERKSATZ

Einheit 3 liest einen Request Zeile für Zeile: Methode, Header, Statuscode und Body.

Das solltest du jetzt können

- ✓ Client und Server einer Verbindung zuordnen, auch wenn ein Server Client ist
- ✓ Erklären, warum zustandslose Server hinter einem Load Balancer skalieren
- ✓ Die sechs Schritte beim Aufruf einer URL nennen
- ✓ Im Network-Tab ersten Request, Timing, Initiator und Daten-Requests finden
- ✓ Make or Buy für einen Teil eines Systems begründen

PlanB. Trusted Agentic Operator

DHBW Duale Hochschule Baden-Württemberg Heidenheim

WEB-PROGRAMMIERUNG · EINHEIT 3

HTTP

Einheit 3 · HTTP

FRAGE

3 % 4xx, 0,2 % 5xx: Wer muss handeln?

Die Beer API ist eine Woche in Betrieb. Die regionalen Brauereien sehen an den Statuscodes, ob die API funktioniert und wer einen Fehler beheben muss.

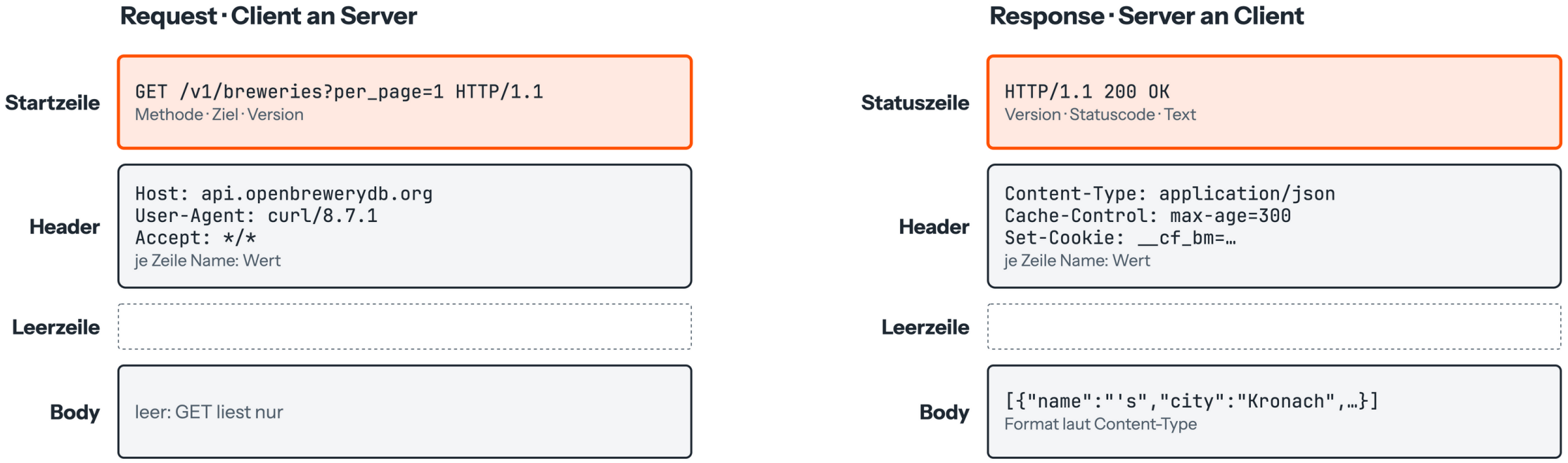
Aufgabe: vier Level im Capture the Flag lösen

- Ein Server mit einem Brauhaus stellt Rätsel aus mehreren Anfragen.
- Jedes gelöste Level liefert ein Flag FLAG{…} und den Ort des nächsten Levels.
- Level 0 bis 3 sind die Aufgaben, Level 4 ist ein Zusatz.
- Die Übung ist Partnerarbeit mit Postman oder curl.

MERKSATZ

Jede Antwort des Servers nennt, was fehlt: im Statuscode, in einem Header oder im Body.

Request und Response haben denselben Aufbau



Eine HTTP-Nachricht besteht aus Startzeile, Headern, einer Leerzeile und dem Body. Beim Request nennt die Startzeile die Methode, bei der Response den Statuscode.

Eine echte Response trägt 22 Header vor dem Body

HTTP

```
HTTP/1.1 200 OK
Date: Sun, 25 Oct 2026 13:51:56 GMT
Content-Type: application/json
Transfer-Encoding: chunked
Connection: keep-alive
Server: cloudflare
Cache-Control: etag, max-age=300, public
x-ratelimit-limit: 120
x-ratelimit-remaining: 119
set-cookie: __cf_bm=4gzbIgfzwfGKoPgRcTz44IJSOlOx7sUQvWeYAC8LwaE-1792936316…
… 12 weitere Header

[{"id":"ae7b3174-8be8-4d53-a3a5-9b8240970eea","name":"'s","brewery_type":…
```

Eine URL hat sechs Teile

```
https://api.openbrewerydb.org:443/v1/breweries?per_page=3#top

Schema    https
Host      api.openbrewerydb.org
Port      443
Pfad      /v1/breweries
Query     per_page=3
Fragment  top
```

Sicher heißt: ändert nichts. Idempotent heißt: zweimal wirkt wie einmal

Methode	Absicht	sicher	idempotent
GET	lesen	ja	ja
POST	anlegen oder verarbeiten	nein	nein
PUT	ganz ersetzen	nein	ja
PATCH	teilweise ändern	nein	nicht garantiert
DELETE	löschen	nein	ja
OPTIONS	erlaubte Methoden erfragen	ja	ja

Die erste Ziffer des Statuscodes nennt die Klasse und wer handelt

Klasse	Bedeutung	Wer handelt
1xx	Information, die Antwort folgt noch	niemand
2xx	Erfolg	niemand
3xx	Umleitung oder Kopie verwenden	der Client folgt dem Hinweis
4xx	Die Anfrage ist so nicht erfüllbar	der Client ändert die Anfrage; steigt der Anteil nach einem Release, prüft der Betreiber
5xx	Der Server konnte eine gültige Anfrage nicht erfüllen	der Betreiber

Die wichtigsten Codes für Erfolg und Umleitung

Code	Bedeutung
200 OK	Erfolg, der Body enthält das Ergebnis
201 Created	angelegt; der Header Location nennt die neue Ressource
204 No Content	Erfolg ohne Body, zum Beispiel nach dem Löschen
301 Moved Permanently	dauerhaft unter der URL aus Location
302 Found, 307 Temporary Redirect	vorübergehend unter der URL aus Location; 307 behält die Methode
304 Not Modified	Die Kopie des Clients ist noch aktuell

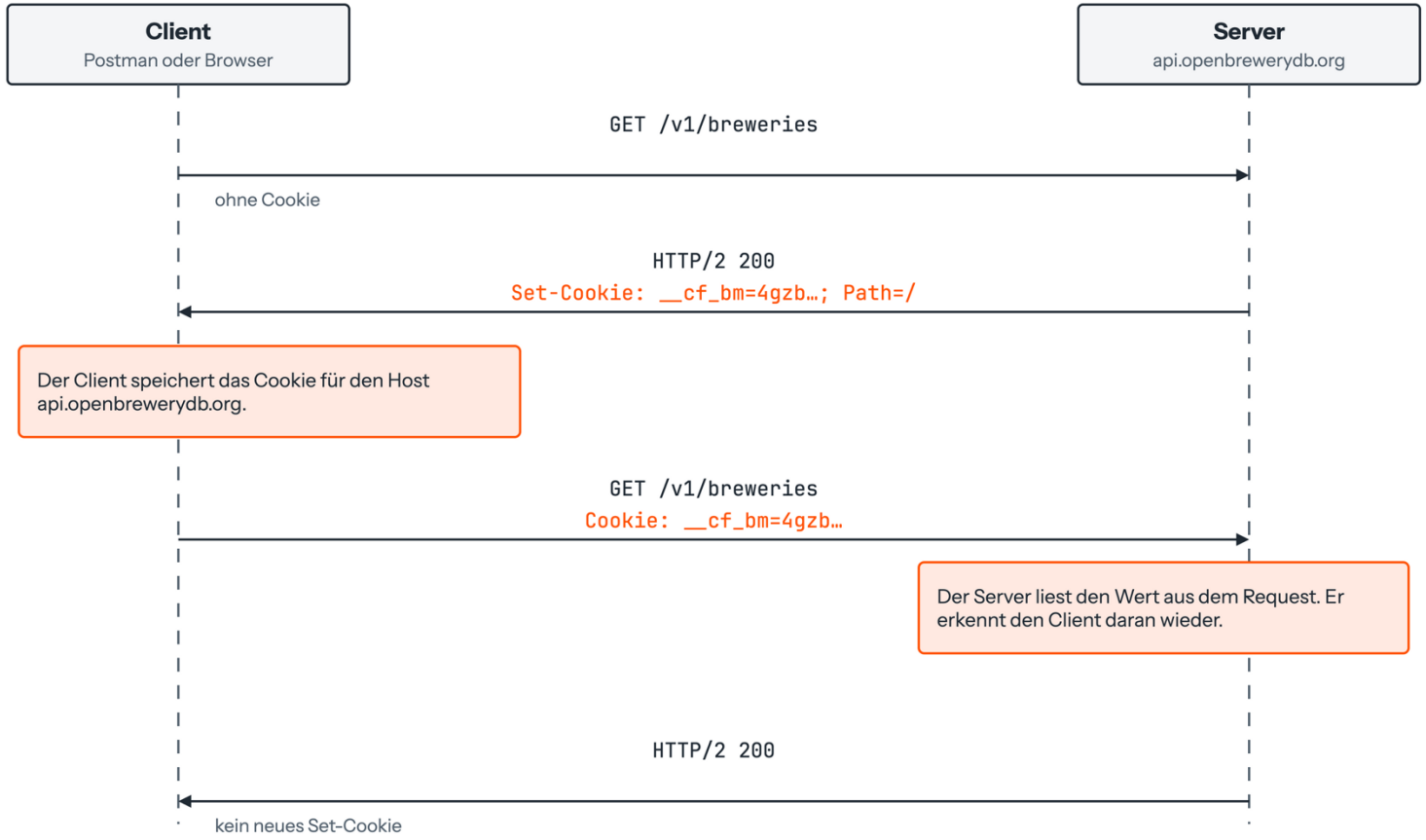
Die wichtigsten Codes für Fehler

Code	Bedeutung
400 Bad Request	Anfrage falsch aufgebaut oder Daten ungültig
401 Unauthorized	nicht angemeldet; WWW-Authenticate nennt das Verfahren
403 Forbidden	bekannt, aber nicht berechtigt
404 Not Found	unter dieser URL gibt es nichts
405 Method Not Allowed	Methode hier nicht erlaubt; Allow nennt die erlaubten
429 Too Many Requests, 503 Service Unavailable	zu viele Anfragen bzw. Server überlastet; Retry-After nennt die Wartezeit

Header tragen Zusatzangaben als Zeile aus Name und Wert

Gruppe	Header	Bedeutung
Content	Content-Type, Content-Length	Format und Länge des Bodys
Accept	Accept, Accept-Language	Wünsche des Clients an Format und Sprache
Cache	Cache-Control, ETag	wie lange eine Kopie gilt, welche Fassung sie ist
Auth	Authorization, WWW-Authenticate	Anmeldedaten und Aufforderung zur Anmeldung
Cookie	Set-Cookie, Cookie	Zustand, den der Client speichert und mitschickt
ohne Gruppe	Location, Allow, Retry-After	Ziel, erlaubte Methoden, Wartezeit

Cookies bringen Zustand in ein zustandsloses Protokoll



Der Server setzt ein Cookie mit Set-Cookie. Der Client speichert es und schickt es bei jeder weiteren Anfrage an denselben Host im Header Cookie mit, solange Path und Expires des Cookies passen.

Live-Demo: ein Request in Postman, Teil für Teil gelesen

1. GET https://api.openbrewerydb.org/v1/breweries?per_page=3 senden
2. Statuscode, Zeit und Größe über dem Body lesen
3. Header der Response den Gruppen zuordnen
4. In der Console den Request mit allen Headern lesen
5. Erneut senden: Postman schickt das Cookie mit

Einheit 3 · HTTP

FRAGE

Live-Frage: Was antwortet die API auf DELETE für eine Brauerei?

DELETE https://api.openbrewerydb.org/v1/breweries/<id> mit der ID der ersten Brauerei aus dem Body

Auflösung: 405, weil die API diese Ressource nur lesen lässt

BEISPIEL

- Der Pfad existiert, die Methode ist dort nicht erlaubt.
- Der Header allow nennt die erlaubten Methoden.
- Der Body ist eine HTML-Seite, weil Postman Accept: */* sendet.
- Wie Accept das Format steuert, zeigt Einheit 4.

```
HTTP/2 405
date: Sun, 25 Oct 2026 13:52:16 GMT
content-type: text/html; charset=utf-8
server: cloudflare
allow: GET, HEAD

<!DOCTYPE html>
…
<title>An Error Occurred: Method Not Allowed</title>
```

Dieselbe Anfrage mit curl: Postman ist nur eine Oberfläche

BASH

```
# Statuszeile, Header und Body der Response
curl -i "https://api.openbrewerydb.org/v1/breweries?per_page=3"

# zusätzlich der Request: > geht zum Server, < kommt zurück
curl -v "https://api.openbrewerydb.org/v1/breweries?per_page=3"

# andere Methode und eigener Header
curl -i -X DELETE -H "Accept: application/json" \
  "https://api.openbrewerydb.org/v1/breweries/<id>"
```

Einheit 3 · HTTP

FRAGE

Live-Frage: Was antwortet der CTF-Server auf HEAD /ctf?

curl -I http://localhost:3001/ctf

Auflösung: 200 mit den Headern von GET, aber ohne Body

BEISPIEL

- HEAD verlangt die Header einer GET-Antwort ohne den Body.
- Content-Length nennt die Länge des Bodys, den GET liefern würde.
- So prüft ein Client Größe und Aktualität, ohne den Inhalt zu laden.
- HEAD ist sicher und idempotent wie GET.

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Content-Length: 316
Date: Sun, 25 Oct 2026 13:52:22 GMT
Connection: keep-alive
Keep-Alive: timeout=5
```

Einheit 3 · HTTP

FRAGE

Live-Frage: Was antwortet der CTF-Server auf GET /ctf#flag?

curl -v "http://localhost:3001/ctf#flag"

Auflösung: Das Fragment verlässt den Client nie

BEISPIEL

- Die Startzeile enthält /ctf, das Fragment fehlt.
- Der Client schneidet das Fragment vor dem Senden ab.
- Ein Browser springt damit zu einer Stelle der Seite.
- Der Server kann ein Fragment deshalb nicht auswerten.

```
> GET /ctf HTTP/1.1
> Host: localhost:3001
> User-Agent: curl/8.7.1
> Accept: */*
>
< HTTP/1.1 200 OK
< Content-Type: application/json; charset=utf-8
< Content-Length: 316
```

Übung

Die Aufgaben stehen vollständig im Skript

Aufgaben dieser Einheit

ÜBUNG

1. Aufgabe 3.1: Anmelden und Siegel (Level 0 und 1)
2. Aufgabe 3.2: Umzug und Ausweis (Level 2 und 3)
3. Zusatz 3.3: Feierabend (Level 4)

SELBSTKONTROLLE

- Server und Regeln: Skript 3.10 und 3.11
- Aufgaben ab Skript 3.12, Hinweise ab 3.17
- Lösung: Ordner der Einheit in der Collection, erst nach der Übung

Review: zwei Wege durch Level 2 und 3

- Ein Team zeigt Level 2: In welcher Antwort stand das Flag?
- Ein Team zeigt Level 3: Warum hat der Server dem Cookie geglaubt?
- Welcher Teil der Antwort hat jeweils weitergeholfen: Statuscode, Header oder Body?
- Der Ordner der Einheit 3 in der Collection zeigt je Level die Anfragen der Lösung.

MERKSATZ

Ein Cookie liegt beim Client, und der Client kann es ändern. Wie ein Server seinen Wert prüft, zeigt Einheit 13.

Das solltest du jetzt können

- ✓ Request, Response und URL in ihre Teile zerlegen
- ✓ Zu einer Absicht die Methode wählen und als sicher oder idempotent einordnen
- ✓ An einem Statuscode erkennen, wer handeln muss
- ✓ Aus Statuscode und Headern einer Antwort die nächste Anfrage ableiten
- ✓ Dieselbe Anfrage in Postman und mit curl senden

WEB-PROGRAMMIERUNG · EINHEIT 4

JSON und Datenaustausch

Einheit 4 · JSON und Datenaustausch

FRAGE

Der Betreiber benennt das Feld abv in alcohol um: Was sieht der Gast?

Drei Brauereien zeigen auf ihrer Webseite den Alkoholgehalt aus der Beer API. Ein festes JSON-Format erspart den regionalen Brauereien die Abstimmung mit jedem einzelnen Client.

Aufgabe: vier Level im Capture the Flag und das JSON der Beer App

- Level 5 bis 8 setzen das Capture the Flag aus Einheit 3 fort.
- Der Server verlangt ein bestimmtes Format, einen JSON-Body und Änderungen ohne Verlust.
- Danach schreibst du Brauerei, Biere und Bewertungen der Beer App als JSON.
- Level 9 ist ein Zusatz.

MERKSATZ

Jede Antwort nennt, welches Format der Server liefert oder erwartet: im Statuscode, in einem Header oder im Body.

Ein Objekt im Programm wird für die Leitung zu Text



Der Server macht aus einem Objekt mit JSON.stringify einen JSON-Text. Der Text steht im Body mit Content-Type application/json. Der Client macht daraus mit JSON.parse wieder ein Objekt.

JSON kennt sechs Datentypen und keinen für Datum und Zeit

Typ	Beispiel	Bedeutung
Objekt	{"name": "Helles"}	Paare aus Schlüssel und Wert; der Schlüssel ist ein String
Array	[1, 2, 3]	geordnete Liste von Werten
String	"Grafing"	Text in doppelten Anführungszeichen
Zahl	5.2	ganz oder mit Dezimalpunkt; es gibt nur eine Art Zahl
Boolean	true, false	Wahrheitswert, klein geschrieben
null	null	kein Wert

JSON ist strenger als ein Objekt im Code von JavaScript

Schreibweise	JavaScript	JSON
Schlüssel ohne Anführungszeichen: {name: 1}	erlaubt	Fehler
einfache Anführungszeichen: {'name': 1}	erlaubt	Fehler
Komma nach dem letzten Element: [1, 2,]	erlaubt	Fehler
Kommentar mit //	erlaubt	Fehler
undefined, Funktion, Date	erlaubt	gibt es nicht

Ein umbenanntes Feld bricht die Clients, die es lesen; der Server meldet weiter 200

- Der Server liefert gültiges JSON; der Anteil von 4xx und 5xx steigt nicht.
- Der Client liest abv und bekommt undefined, ohne Fehlermeldung.
- Der Gast sieht eine leere Stelle; die Brauerei bemerkt den Fehler vor dem Betreiber.
- Ein neues Feld stört keinen Client; Umbenennen, Entfernen und ein neuer Typ brechen sie.

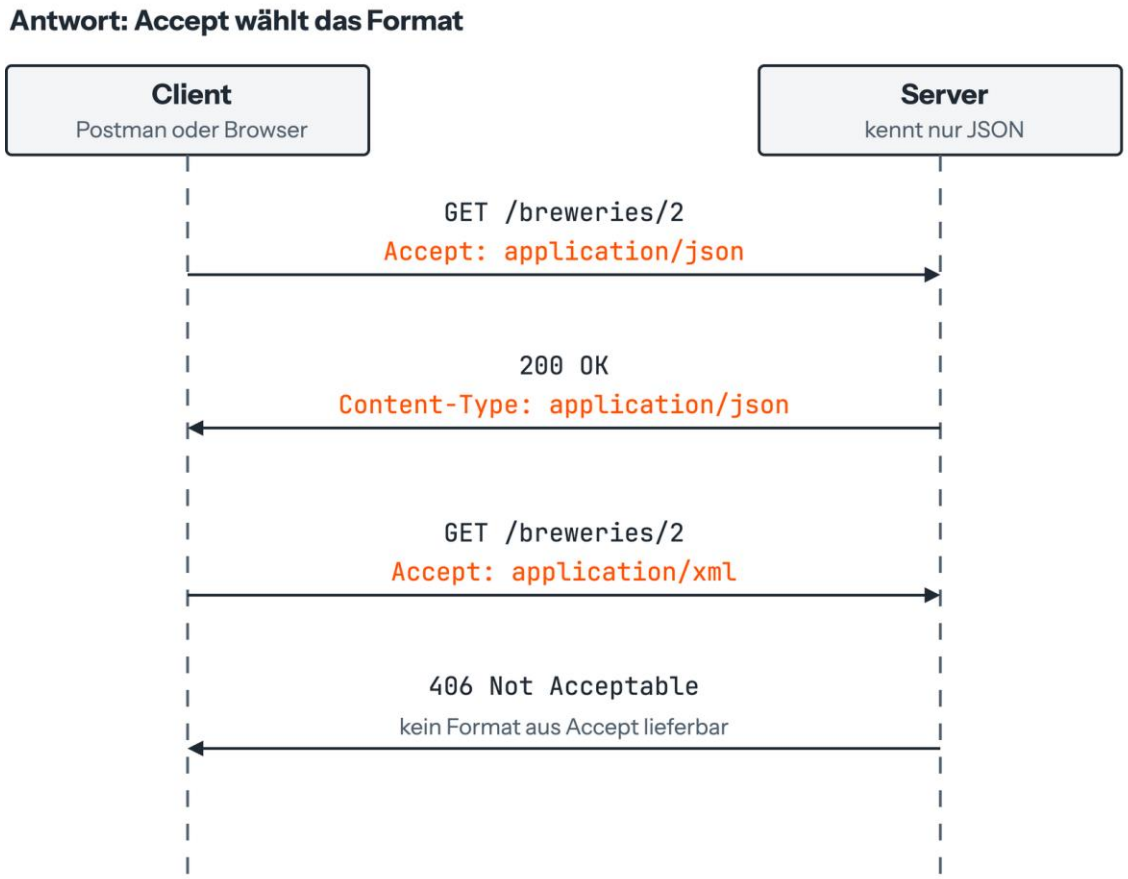
MERKSATZ

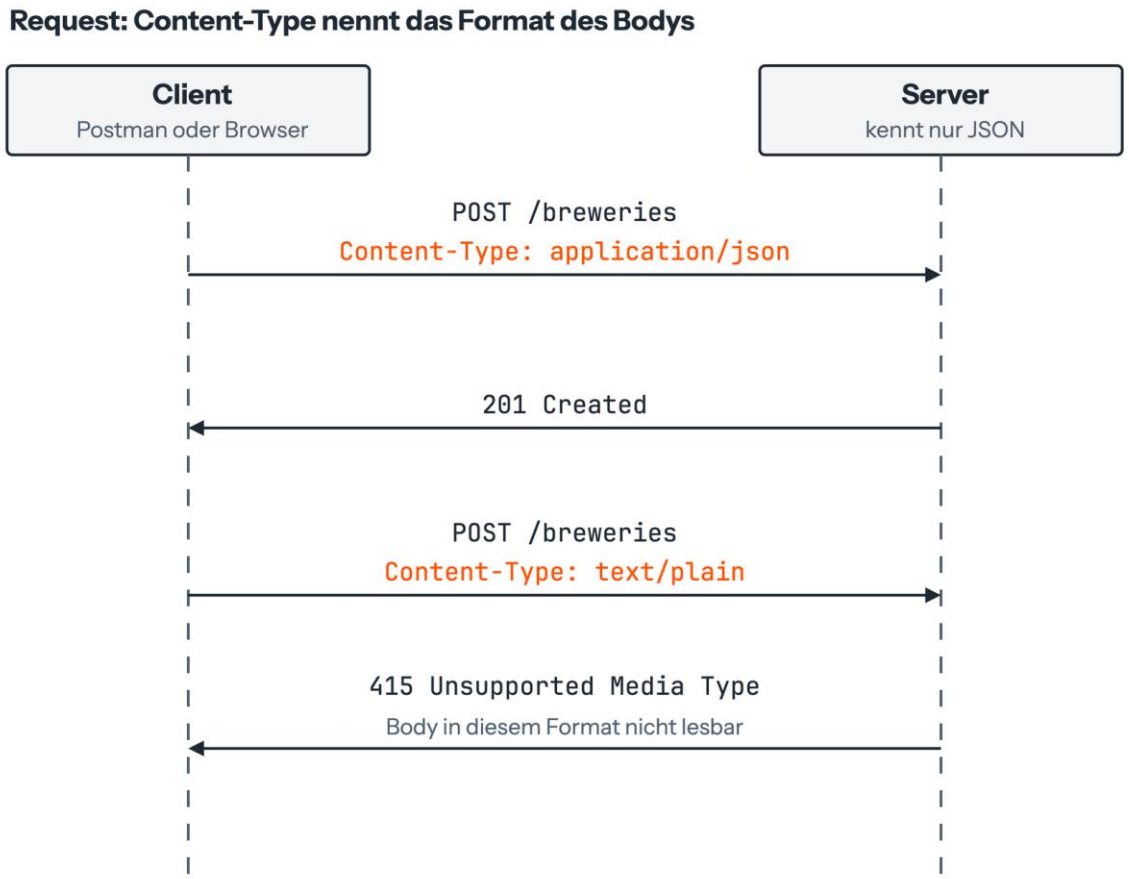
Die Open Brewery DB liefert umbenannte Felder eine Zeit lang doppelt: state neben state_province, street neben address_1.

Die Beer API legt ihr JSON mit fünf Konventionen fest

Konvention	Beispiel	Grund
Schlüssel englisch in camelCase	breweryId, averageRating	so benennen JavaScript und TypeScript ihre Variablen
Liste als Array, leer als []	[{"id": 1, …}, {"id": 2, …}]	der Client läuft ohne Sonderfall über die Elemente
Zeitpunkt als String nach ISO 8601 in UTC	"2026-10-27T09:15:00.000Z"	sortierbar, eindeutig, mit Zeitzone
der Server vergibt die id, ein Verweis heißt Ressource plus Id	"breweryId": 2	jede Brauerei steht einmal unter ihrer URL
null heißt bekannt leer, ein fehlendes Feld nicht angegeben	"averageRating": null	ohne Bewertung gibt es keinen Durchschnitt

Accept wählt das Format der Antwort, Content-Type nennt das Format des Bodys



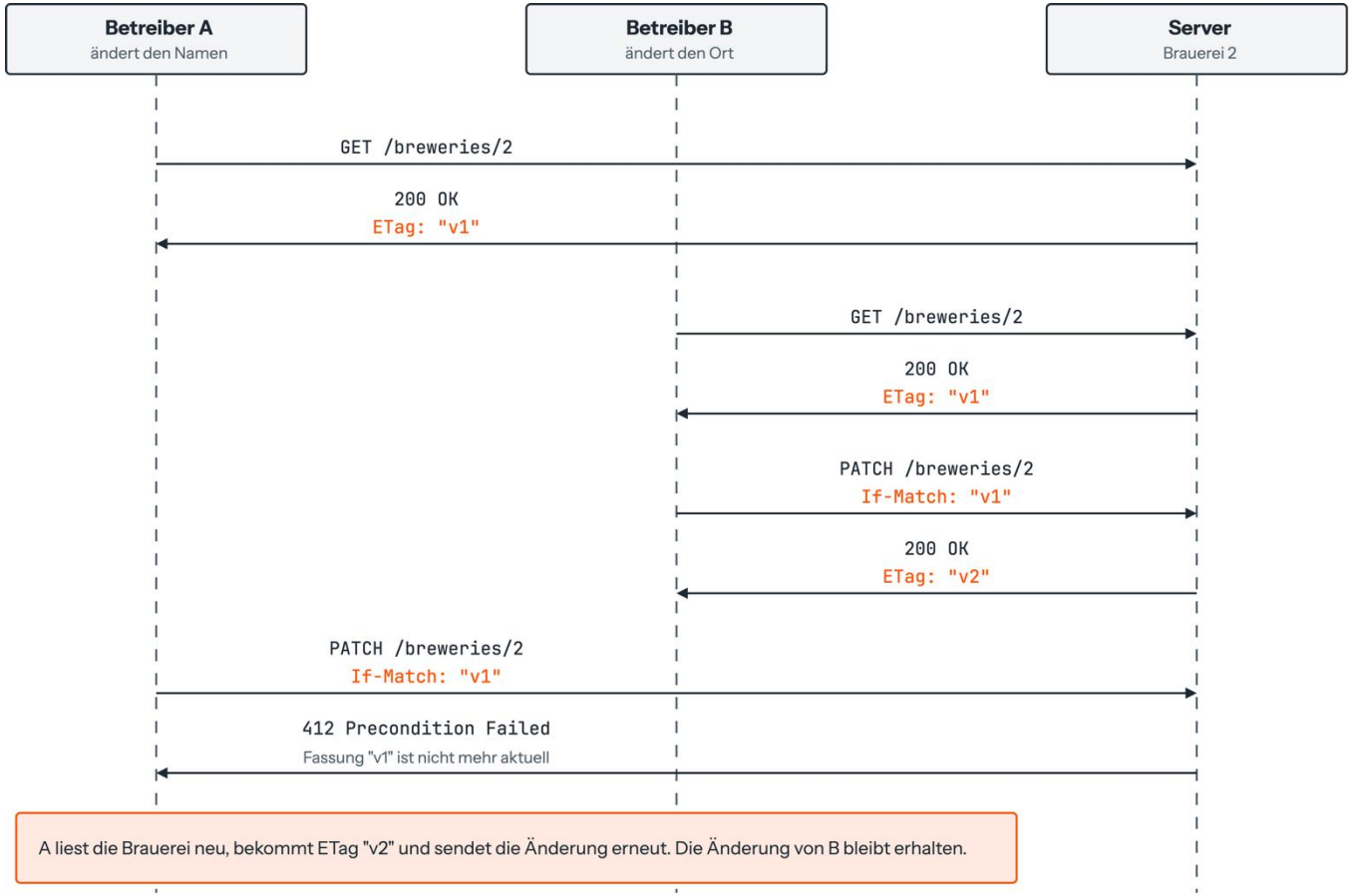


Links fragt der Client mit Accept application/json und bekommt 200 mit Content-Type application/json; mit Accept application/xml bekommt er 406 Not Acceptable. Rechts sendet der Client einen Body mit Content-Type application/json und bekommt 201; mit Content-Type text/plain bekommt er 415 Unsupported Media Type.

PUT ersetzt die ganze Ressource, PATCH ändert nur die gesendeten Felder

Anfrage an /beers/2	Body	Ergebnis
PUT	{"name": "Wildbräu Grafing Helles", "style": "helles", "abv": 5.1, "breweryId": 2}	alle Felder neu; description fehlt danach
PATCH	{"abv": 5.1}	nur abv ändert sich
PATCH	{"description": null}	description wird entfernt
PATCH	{}	nichts ändert sich

ETag und If-Match verhindern, dass eine Änderung eine andere überschreibt



Betreiber A und Betreiber B lesen Brauerei 2 mit ETag v1. B ändert sie mit If-Match v1 und bekommt ETag v2. A sendet seine Änderung mit If-Match v1 und bekommt 412 Precondition Failed. A liest neu und sendet die Änderung erneut; die Änderung von B bleibt erhalten.

Fünf Statuscodes antworten auf Fragen von Format und Fassung

Code	Bedeutung	Header
406 Not Acceptable	kein Format aus Accept lieferbar	Accept
415 Unsupported Media Type	Format des Bodys nicht lesbar	Content-Type, Accept-Patch
428 Precondition Required	Der Server verlangt eine Bedingung	If-Match
412 Precondition Failed	Die Bedingung ist nicht erfüllt, die Fassung ist veraltet	If-Match, ETag
304 Not Modified	Die Kopie des Clients ist aktuell; die Antwort hat keinen Body	If-None-Match, ETag

Live-Demo: JSON lesen, senden und an einem Bier entwerfen

1. GET https://api.openbrewerydb.org/v1/breweries/<id> für die Brauerei Augustiner senden
2. Den Body in den Ansichten Pretty und Raw lesen
3. Dieselbe Anfrage mit Accept: application/xml senden
4. POST https://postman-echo.com/post mit einem JSON-Body senden
5. In der Console von Chrome JSON.stringify aufrufen
6. Ein Bier als JSON entwerfen: Brauerei eingebettet oder als Verweis

Die Open Brewery DB schreibt Schlüssel in snake_case und kennt null

JSON

```
{
  "id": "ab02b540-6a18-4020-8363-d5102897f4e0",
  "name": "Augustiner",
  "brewery_type": "brewpub",
  "address_1": "Arnulfstraße 52",
  "address_2": null,
  …
  "city": "München",
  "state_province": "Bayern",
  "postal_code": "80335",
  …
  "state": "Bayern",
  "street": "Arnulfstraße 52"
}
```

Einheit 4 · JSON und Datenaustausch

FRAGE

Live-Frage: Was antwortet die Open Brewery DB auf Accept: application/xml?

GET https://api.openbrewerydb.org/v1/breweries/<id> der Brauerei Augustiner mit dem Header Accept: application/xml

Auflösung: 200 mit JSON, weil die API Daten nur als JSON liefert

BEISPIEL

- Der Server wählt aus den Formaten, die er kennt.
- Für Daten kennt die Open Brewery DB nur JSON.
- Statt 406 darf ein Server sein Standardformat liefern.
- Der Client prüft Content-Type und entscheidet selbst.

```
HTTP/2 200
content-type: application/json
vary: Accept-Encoding,Origin
x-content-type-options: nosniff

{"id":"ab02b540-6a18-...","name":"Augustiner",...}
```

Einheit 4 · JSON und Datenaustausch

FRAGE

Live-Frage: Derselbe JSON-Body mit Content-Type text/plain: Was liest der Server?

POST https://postman-echo.com/post mit dem Body { "name": "Augustiner", "city": "München" }

Auflösung: einen String, weil Content-Type das Format des Bodys nennt

BEISPIEL

- Postman Echo zeigt, wie es den Body gelesen hat.
- Mit application/json steht im Feld json ein Objekt.
- Mit text/plain ist der Body nur ein String in data.
- Ein Server, der nur JSON annimmt, antwortet mit 415.

```
{
  "data": "{ \"name\": \"Augustiner\", … }",
  "headers": {
    "content-type": "text/plain",
    …
  },
  "json": null
}
```

Einheit 4 · JSON und Datenaustausch

FRAGE

Live-Frage: Was macht JSON.stringify aus einem Datum und aus undefined?

```
JSON.stringify({ name: 'Helles', founded: new Date(2026, 9, 27), note: undefined })
```

Auflösung: Das Datum wird ein String in UTC, das Feld mit undefined fehlt

BEISPIEL

- JSON hat keinen Typ für Zeit; es entsteht ein String.
- Mitternacht in Berlin ist im Winter 23:00 Uhr UTC am Vortag.
- undefined gibt es in JSON nicht; das Feld fällt weg.
- Der Monat zählt in JavaScript ab 0: 9 ist Oktober.

```
> JSON.stringify({ name: 'Helles',
    founded: new Date(2026, 9, 27), note: undefined })
'{"name":"Helles","founded":"2026-10-26T23:00:00.000Z"}'
```

Variante A: Die Brauerei steckt im Bier

JSON

```
{
  "id": 5,
  "name": "Wildbräu Grafing Pils",
  "style": "pils",
  "abv": 4.9,
  "brewery": {
    "id": 2,
    "name": "Wildbräu Grafing",
    "city": "Grafing bei München",
    "country": "Deutschland"
  }
}
```

Die Beer API verweist mit breweryId auf die Brauerei

BEISPIEL

- Variante A wiederholt die Brauerei in jedem ihrer Biere.
- Benennt sich die Brauerei um, ändert A jede Kopie.
- Variante B braucht für den Namen einen Request mehr.
- Das Datenmodell der Beer App wählt Variante B.

```
{
  "id": 5,
  "name": "Wildbräu Grafing Pils",
  "style": "pils",
  "abv": 4.9,
  "breweryId": 2
}
```

Übung

Die Aufgaben stehen vollständig im Skript

Aufgaben dieser Einheit

ÜBUNG

1. Aufgabe 4.1: Lieferschein (Level 5)
2. Aufgabe 4.2: Kiste anlegen und füllen (Level 6 und 7)
3. Aufgabe 4.3: Rezept (Level 8)
4. Aufgabe 4.4: JSON für die Beer App
5. Zusatz 4.5: Frische (Level 9)

SELBSTKONTROLLE

- Capture the Flag fortsetzen: Skript 4.12
- Aufgaben ab Skript 4.13, Hinweise ab 4.24
- Lösung: Ordner der Einheit in der Collection, erst nach der Übung

Review: zwei Wege durch das Capture the Flag und zwei Bierlisten

- Ein Team zeigt Level 7: Was hat das Etikett der Kiste bewahrt?
- Ein Team zeigt Level 8: Warum scheiterte die erste Änderung des Rezepts?
- Zwei Teams vergleichen ihre Bierliste Feld für Feld.
- Wo fehlt ein Feld, und warum?

MERKSATZ

Ab Einheit 5 liefert die Beer API die Bierliste selbst. Dann vergleichst du deine Datei mit ihrer Antwort.

Das solltest du jetzt können

- ✓ Gültiges JSON mit den sechs Datentypen schreiben und Syntaxfehler finden
- ✓ Eine Ressource der Beer App nach dem Datenmodell als JSON darstellen
- ✓ null und ein fehlendes Feld begründet unterscheiden
- ✓ Mit Accept und Content-Type arbeiten und 406 und 415 deuten
- ✓ Mit PATCH einzelne Felder ändern und mit ETag eine veraltete Fassung erkennen

PlanB. Trusted Agentic Operator

DHBW Duale Hochschule Baden-Württemberg Heidenheim

WEB-PROGRAMMIERUNG · EINHEIT 5

API, Backend und das erste NestJS-Projekt

Einheit 5 · API, Backend und das erste NestJS-Projekt

FRAGE

Zehn Jahre Beer API: mit Framework oder selbst geschrieben?

Die regionalen Brauereien betreiben die Beer API über Jahre. Webseite und App sollen dabei immer dieselben Biere zeigen.

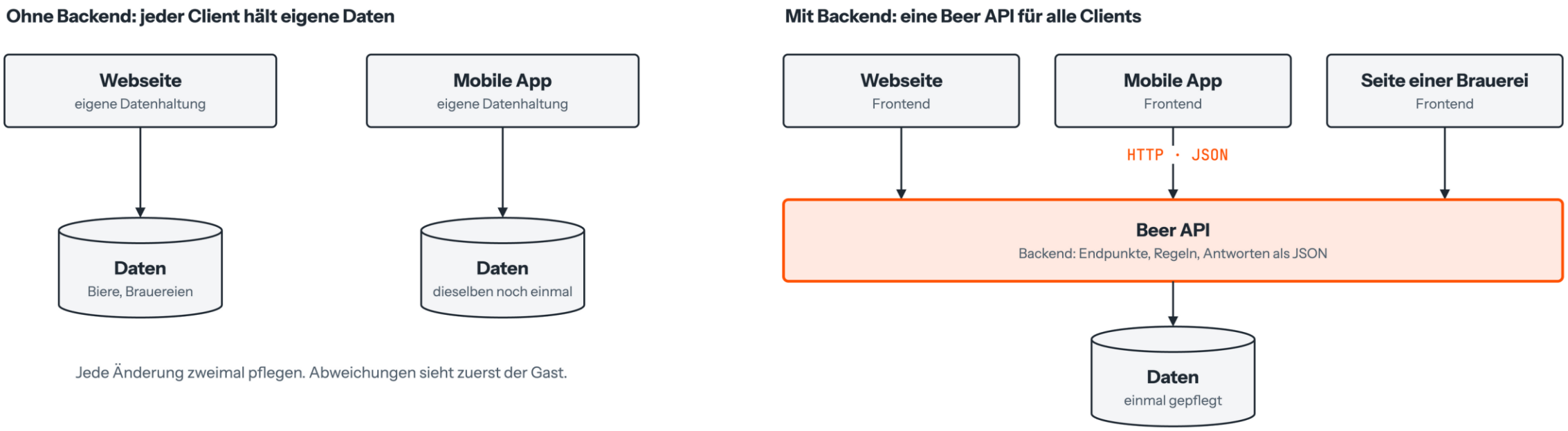
Aufgabe: Als Gast möchte ich alle Biere und ein einzelnes Bier abrufen

- Damit sehen Gäste auf Webseite und App, welche Biere es gibt.
- GET /beers antwortet mit den vier Bieren der Testdaten.
- GET /beers/2 antwortet mit dem Wildbräu Grafing Helles, GET /beers/42 mit 404.
- Zusatz: GET /beers?style=helles liefert nur die zwei hellen Biere.

MERKSATZ

Das Live-Coding zeigt dasselbe an den Brauereien. Die Biere baust du danach selbst.

Webseite und App fragen dieselbe Beer API



Ohne Backend halten Webseite und Mobile App je eigene Daten, jede Änderung wird zweimal gepflegt. Mit Backend fragen Webseite, Mobile App und die Seite einer Brauerei dieselbe Beer API über HTTP mit JSON; die Daten liegen einmal hinter der API.

Eine API legt fest, welche Anfragen das Backend versteht

Begriff	Bedeutung	In der Beer App
API	Vertrag aus Anfragen und Antworten	die Beer API
Backend	speichert Daten, prüft Regeln, antwortet mit JSON	NestJS-Projekt beer-api
Frontend	zeigt Daten an und nimmt Eingaben an	Webseite, Mobile App
Endpunkt	Methode und Pfad, die das Backend beantwortet	GET /beers, GET /beers/2
Framework	Grundstruktur, die den Ablauf vorgibt und eigenen Code aufruft	NestJS

Ein Framework übernimmt, was jede API braucht

Aufgabe	Bedeutung	In NestJS
Routing	Methode und Pfad führen zur richtigen Funktion	@Controller, @Get
Parsing	Werte aus Pfad und Query lesen	@Param, @Query
Antwort	Rückgabewert als JSON mit Statuscode senden	return in der Methode
Fehler	unbekannter Pfad oder fehlende Daten als 404	Cannot GET, NotFoundException
Struktur	feste Orte für Controller, Services, Module	Nest CLI, Konventionen

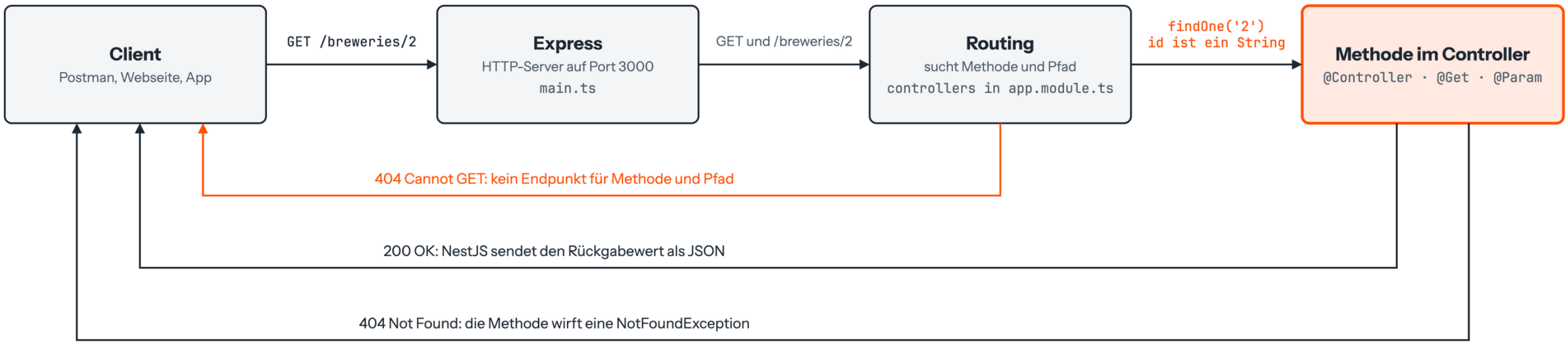
Ein Framework spart eigenen Code und bindet an fremde Releases

- Konventionen: Neue Entwickler finden sich in jedem NestJS-Projekt zurecht.
- Community: Viele Nutzer finden Sicherheitslücken, Updates beheben sie.
- Weniger eigener Code bedeutet weniger Wartung über zehn Jahre.
- Bewerber kennen verbreitete Frameworks schon.
- Preis: Major-Versionen erzwingen Umbauten, Entscheidungen treffen andere.

MERKSATZ

Die Beer API bleibt bei NestJS 11, bis die Bibliothek für die Validierung (Einheit 7) Version 12 unterstützt.

Eine Anfrage läuft über Express und das Routing zur Methode im Controller



Der Client sendet GET /breweries/2 an Express auf Port 3000. Das Routing sucht zu Methode und Pfad eine Methode in den Controllern aus app.module.ts und ruft findOne mit der id als String auf. Findet das Routing keinen Endpunkt, antwortet es mit 404 Cannot GET. Gibt die Methode einen Wert zurück, sendet NestJS ihn als JSON mit 200; wirft sie eine NotFoundException, kommt 404 Not Found.

Drei Dateien des Basisprojekts tragen den Weg einer Anfrage

Datei	Aufgabe
src/main.ts	erzeugt die Anwendung aus AppModule und startet Express auf Port 3000
src/app.module.ts	listet unter controllers alle Controller, die das Routing kennt
src/app.controller.ts	beantwortet GET / mit Hello World

Der erste Controller beantwortet GET / mit einem Objekt

TYPESCRIPT · SRC/APP.CONTROLLER.TS

```typescript
import { Controller, Get } from '@nestjs/common';

@Controller()
export class AppController {
  @Get()
  getHello() {
    return { message: 'Hello World' };
  }
}
```

Decorators verbinden Klasse, Methode und Parameter mit dem Routing

Decorator	steht an	Bedeutung
@Controller('breweries')	Klasse	Pfad /breweries für alle Methoden der Klasse
@Get()	Methode	GET auf den Pfad der Klasse
@Get(':id')	Methode	GET /breweries/2 und jeder andere Wert an dieser Stelle
@Param('id')	Parameter	Wert von :id aus dem Pfad, immer ein String
@Query('style')	Parameter	Wert von ?style= als String, ohne Query undefined

Schritt 1: Die Nest CLI erzeugt den Controller und trägt ihn ein

BASH

```
npx nest generate controller breweries

CREATE src/breweries/breweries.controller.ts (107 bytes)
UPDATE src/app.module.ts (698 bytes)
```

Schritt 2: Ein Typ beschreibt die Brauerei

TYPESCRIPT · SRC/BREWERIES/BREWERIES.CONTROLLER.TS

```typescript
import { Controller, Get, NotFoundException, Param } from '@nestjs/common';

interface Brewery {
  id: number;
  name: string;
  city: string;
  country: string;
}
```

Schritt 3: findAll gibt alle Brauereien zurück

TYPESCRIPT · SRC/BREWERIES/BREWERIES.CONTROLLER.TS

```
@Controller('breweries')
export class BreweriesController {
  @Get()
  findAll(): Brewery[] {
    return breweries;
  }
```

Einheit 5 · API, Backend und das erste NestJS-Projekt

FRAGE

Live-Frage: Was antwortet die API auf GET /Breweries?

GET http://localhost:3000/Breweries

Auflösung: 200 mit allen Brauereien, weil Express Großbuchstaben im Pfad ignoriert

BEISPIEL

- Das Routing vergleicht Pfade ohne Groß- und Kleinschreibung.
- Express nennt dafür die Option caseSensitive, Standard false.
- Ein Webserver für Dateien unterscheidet die Schreibweise meist.
- Clients schreiben Pfade trotzdem so wie dokumentiert.

```
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 340

[{"id":1,"name":"Privatbrauerei Schlumberger",…
```

Einheit 5 · API, Backend und das erste NestJS-Projekt

FRAGE

Live-Frage: Was antwortet die API, wenn die Anfrage nach einer Stadt fragt?

GET http://localhost:3000/breweries?city=München

Auflösung: 200 mit allen Brauereien, weil findAll die Query nicht liest

BEISPIEL

- Express zerlegt die Query, findAll bekommt den Wert aber nicht.
- Erst ein Parameter mit @Query('city') übergibt den Wert.
- Unbekannte Query-Parameter ignoriert die API ohne Fehler.
- Zusatz 5.3 liest einen Query-Parameter selbst.

```
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 340

[{"id":1,"name":"Privatbrauerei Schlumberger",…
```

Schritt 4: findOne sucht die Brauerei oder antwortet mit 404

TYPESCRIPT · SRC/BREWERIES/BREWERIES.CONTROLLER.TS

```
  @Get(':id')
  findOne(@Param('id') id: string): Brewery {
    // Pfadparameter kommen immer als String an
    const brewery = breweries.find((b) => b.id === Number(id));
    if (!brewery) {
      throw new NotFoundException(`Brewery ${id} not found`);
    }
    return brewery;
  }
}
```

Einheit 5 · API, Backend und das erste NestJS-Projekt

FRAGE

Live-Frage: Was antwortet die API auf GET /breweries/abc?

GET http://localhost:3000/breweries/abc

Auflösung: 404 von findOne, weil abc keine Zahl ergibt

BEISPIEL

- Number('abc') ergibt NaN, und NaN gleicht keiner Zahl.
- find findet nichts, findOne wirft die NotFoundException.
- Die Meldung nennt abc: Sie kommt von findOne, nicht vom Routing.
- Passend wäre 400 für eine ungültige id: Einheit 8.

```
HTTP/1.1 404 Not Found
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 72

{
  "message": "Brewery abc not found",
  "error": "Not Found",
  "statusCode": 404
}
```

Fehler finden in vier Schritten: erst eine Spur, dann eine Änderung

Schritt	Frage	Werkzeug
1. Symptom	Welcher Statuscode, welcher Body, welche Meldung?	Postman, roter Test
2. Hypothese	Welche Stelle im Weg der Anfrage erzeugt genau dieses Symptom?	Bild von Folie 5.8
3. Spur	Was bestätigt oder widerlegt die Hypothese, ohne Code zu ändern?	Terminal, Postman, npx eslint src
4. Beheben	Eine Änderung, dann dieselbe Anfrage erneut	Postman

Fehlersuche: GET /breweries antwortet nach dem Speichern mit 404

BEISPIEL

- Bis eben lieferte GET /breweries vier Brauereien.
- Jetzt antwortet dieselbe Anfrage mit 404.
- breweries.controller.ts ist unverändert.
- Welche Stelle im Weg der Anfrage erzeugt Cannot GET?

```
{
  "message": "Cannot GET /breweries",
  "error": "Not Found",
  "statusCode": 404
}
```

Fehlersuche: Im Terminal fehlt die Route, weil der Controller nicht eingetragen ist

- Spur 1: Cannot GET kommt vom Routing, findAll lief also nie.
- Spur 2: Das Terminal zeigt keine Zeile zu BreweriesController, auch keine mit anderem Pfad.
- Spur 3: npx eslint src meldet 'BreweriesController' is defined but never used.
- Ursache: Der Controller fehlt unter controllers in app.module.ts.
- Beheben: Eintrag zurück, GET /breweries antwortet wieder mit 200.

Übung

Die Aufgaben stehen vollständig im Skript

Aufgaben dieser Einheit

ÜBUNG

1. Aufgabe 5.1: Biere auflisten
2. Aufgabe 5.2: Ein Bier abrufen
3. Zusatz 5.3: Biere filtern

SELBSTKONTROLLE

- Basisprojekt und Beispiel: Skript 5.9 bis 5.16
- Aufgaben ab Skript 5.17, Hinweise ab 5.23
- Lösungsstand nach der Übung

Review: zwei Lösungen für Aufgabe 5.2 und Zusatz 5.3

- Was ist gleich, was ist anders?
- Wo wird die id aus dem Pfad in eine Zahl gewandelt?
- Filtert der Zusatz in findAll oder in einer eigenen Methode?
- Was wäre bei 400 Bieren statt vier anders?

MERKSATZ

Daten und Suche stehen heute im Controller. Einheit 6 verschiebt sie in einen Service.

Das solltest du jetzt können

- ✓ Begründen, warum Webseite und App eine gemeinsame API nutzen
- ✓ Den Weg einer Anfrage von Express bis zur Methode im Controller zeigen
- ✓ Einen Controller mit der Nest CLI erzeugen und Endpunkte mit @Get, @Param und @Query bauen
- ✓ Einen Fehler in vier Schritten vom Symptom zur Ursache verfolgen

PlanB. Trusted Agentic Operator

DHBW Duale Hochschule Baden-Württemberg Heidenheim

WEB-PROGRAMMIERUNG · EINHEIT 6

Service und Dependency Injection

Einheit 6 · Service und Dependency Injection

FRAGE

Sieben Zeilen in zwei Controllern greifen auf die Daten zu: Was kostet die Datenbank?

Die regionalen Brauereien lassen die Beer API über Jahre weiterentwickeln und zahlen jede Änderung. Ab Einheit 12 liegen die Daten in einer Datenbank statt in einem Array.

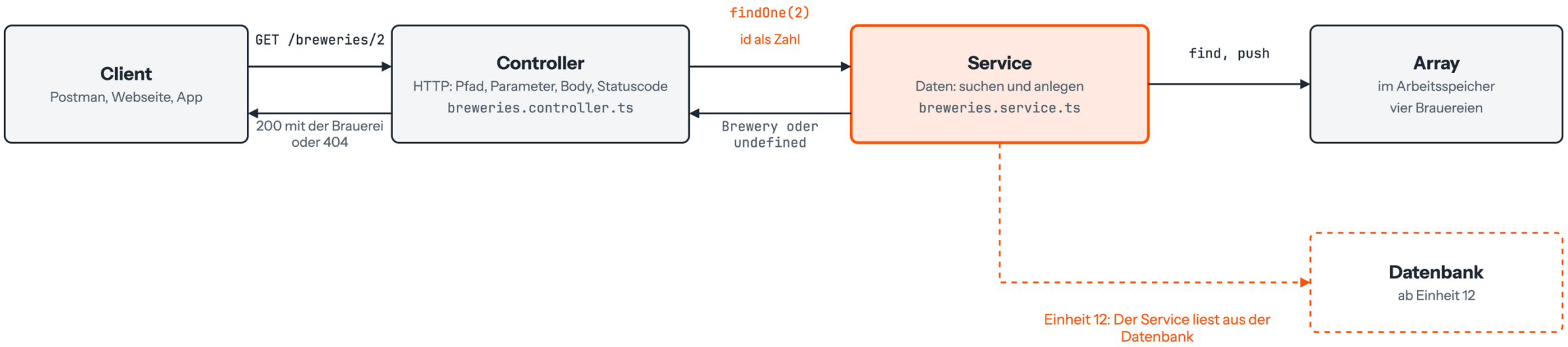
Aufgabe: Als Betreiber möchte ich ein Bier anlegen und löschen

- Damit entspricht die Liste dem Angebot der Brauereien.
- POST /beers legt das Wildbräu Grafing Pils mit der id 5 an.
- Danach liefert GET /beers fünf Biere.
- DELETE /beers/5 entfernt das Bier, danach antwortet GET /beers/5 mit 404.
- Zusatz: GET /beers?style=pils findet auch das neu angelegte Pils.

MERKSATZ

Das Live-Coding zeigt Service und Anlegen an den Brauereien. Die Biere und das Löschen baust du danach selbst.

Der Service hält die Daten an einer Stelle, der Controller beantwortet HTTP



Der Client sendet GET /breweries/2 an den Controller. Der Controller wandelt die id in eine Zahl und ruft findOne(2) im Service auf. Der Service sucht im Array im Arbeitsspeicher und gibt die Brauerei oder undefined zurück. Der Controller antwortet mit 200 oder 404. Ab Einheit 12 liest der Service aus einer Datenbank statt aus dem Array.

Der Controller übersetzt zwischen HTTP und Werten, der Service arbeitet nur mit Werten

Aufgabe	Controller	Service
Pfad und Methode einem Endpunkt zuordnen	@Controller, @Get, @Post	–
Pfadparameter und Body lesen, id in eine Zahl wandeln	@Param, @Body, Number(id)	–
Daten halten, suchen, anlegen, neue id vergeben	–	Array, find, push, nextId
Statuscode der Antwort festlegen	404 mit NotFoundException	gibt Wert oder undefined zurück

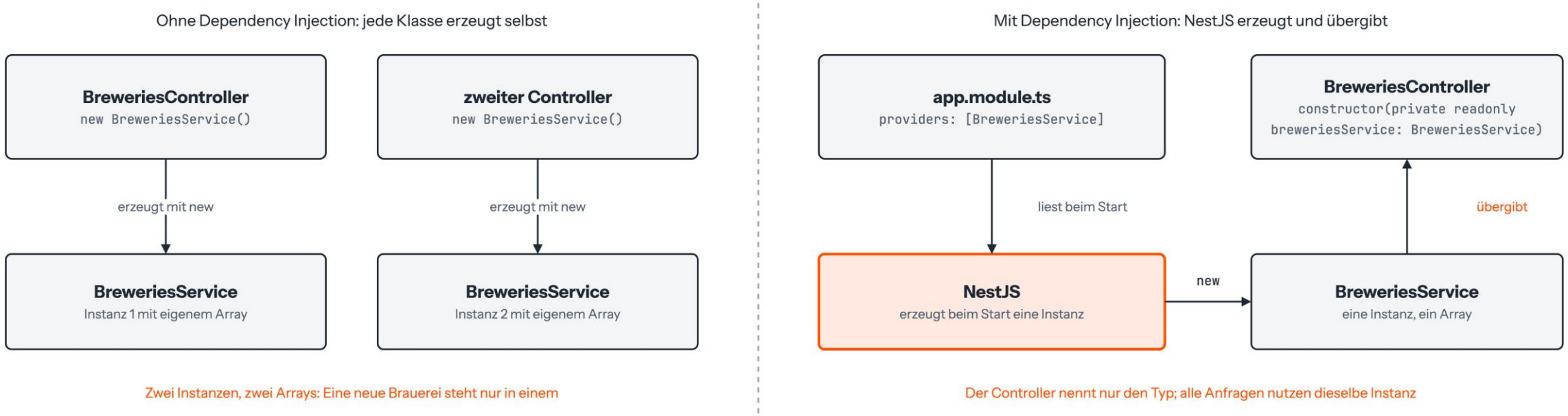
Vereinfachung in dieser Einheit: Der Service kennt kein HTTP

- Der Service liest keine Anfrage und setzt keinen Statuscode.
- Er bekommt Werte wie die Zahl 2 und gibt Werte zurück.
- Ab Einheit 8 wirft der Service selbst eine NotFoundException.
- NestJS übersetzt diese Ausnahme in die Antwort 404.
- Auch ab Einheit 8 liest nur der Controller Request und Response.

MERKSATZ

Heute entscheidet der Controller über 404, damit die Aufteilung klar sichtbar ist.

Mit Dependency Injection erzeugt NestJS den Service und übergibt ihn dem Controller



Ohne Dependency Injection erzeugt jeder Controller seinen Service mit new; zwei Controller hätten zwei Instanzen mit je eigenem Array. Mit Dependency Injection liest NestJS beim Start providers in app.module.ts, erzeugt eine Instanz von BreweriesService und übergibt sie dem Constructor von BreweriesController.

Drei Stellen machen eine Klasse injizierbar

Stelle	Datei	Bedeutung
@Injectable()	breweries.service.ts	markiert die Klasse als Provider
providers: [BreweriesService]	app.module.ts	meldet den Provider bei NestJS an
constructor(private readonly breweriesService: BreweriesService)	breweries.controller.ts	fordert eine Instanz an und legt sie als Feld ab
npx nest generate service breweries	Terminal	erzeugt die Klasse und erledigt die ersten beiden Stellen

Vereinfachung: NestJS erzeugt den Service einmal je Module, alle Anfragen teilen ihn

- Standard ist eine Instanz je Module, das den Service unter providers einträgt.
- Alle Anfragen nutzen diese Instanz und damit dasselbe Array.
- Deshalb findet GET /breweries/5 die Brauerei aus einem POST davor.
- Die Beer API nutzt keinen Scope REQUEST mit einer Instanz je Anfrage.
- In zwei Modulen unter providers ergibt er zwei Instanzen: Einheit 9.

MERKSATZ

Das Array liegt im Arbeitsspeicher des Servers. Es lebt so lange wie der Prozess.

Drei Decorators und ein Typ nehmen Änderungen an

Decorator	steht an	Bedeutung
@Post()	Methode	POST auf den Pfad der Klasse; NestJS antwortet mit 201
@Body()	Parameter	JSON-Body der Anfrage als Objekt
@Delete(':id')	Methode	DELETE mit id im Pfad; die id kommt als String an
Omit<Brewery, 'id'>	Typ	eine Brauerei ohne id, für den Body beim Anlegen

Schritt 1: Die Nest CLI erzeugt den Service und trägt ihn unter providers ein

BASH

```
npx nest generate service breweries

CREATE src/breweries/breweries.service.ts (93 bytes)
UPDATE src/app.module.ts (863 bytes)
```

Schritt 2: Daten und Suche ziehen in den Service

TYPESCRIPT · SRC/BREWERIES/BREWERIES.SERVICE.TS

```
private nextId = 5;

findAll(): Brewery[] {
  return this.breweries;
}

findOne(id: number): Brewery | undefined {
  return this.breweries.find((b) => b.id === id);
}
```

Schritt 3: Der Controller fordert den Service im Constructor an

TYPESCRIPT · SRC/BREWERIES/BREWERIES.CONTROLLER.TS

```
constructor(private readonly breweriesService: BreweriesService) {}

@Get()
findAll(): Brewery[] {
  return this.breweriesService.findAll();
}
```

Schritt 4: findOne fragt den Service und entscheidet über 404

TYPESCRIPT · SRC/BREWERIES/BREWERIES.CONTROLLER.TS

```
@Get(':id')
findOne(@Param('id') id: string): Brewery {
  // Pfadparameter kommen immer als String an
  const brewery = this.breweriesService.findOne(Number(id));
  if (!brewery) {
    throw new NotFoundException(`Brewery ${id} not found`);
  }
  return brewery;
}
```

Schritt 5: create vergibt die nächste id und legt die Brauerei ab

TYPESCRIPT · SRC/BREWERIES/BREWERIES.SERVICE.TS

```
create(data: Omit<Brewery, 'id'>): Brewery {
  const brewery = { id: this.nextId++, ...data };
  this.breweries.push(brewery);
  return brewery;
}
```

Schritt 6: POST /breweries reicht den Body an den Service weiter

TYPESCRIPT · SRC/BREWERIES/BREWERIES.CONTROLLER.TS

```
@Post()
create(@Body() data: Omit<Brewery, 'id'>): Brewery {
  return this.breweriesService.create(data);
}
```

Einheit 6 · Service und Dependency Injection

FRAGE

Live-Frage: Was antwortet die API auf POST /breweries mit einem leeren Objekt?

POST http://localhost:3000/breweries mit dem Body {}

Auflösung: 201 mit einer Brauerei ohne Namen, weil niemand den Body prüft

BEISPIEL

- @Body() übernimmt jedes JSON.
- Der Typ Omit<Brewery, 'id'> gilt nur beim Übersetzen.
- create legt die Brauerei an und vergibt eine id.
- Ab Einheit 7 prüft ein Schema den Body vor dem Service.

```
HTTP/1.1 201 Created
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 8

{"id":5}
```

Einheit 6 · Service und Dependency Injection

FRAGE

Live-Frage: Welche id bekommt eine Brauerei, die im Body die id 1 mitbringt?

POST http://localhost:3000/breweries mit dem Body { "name": "Brauerei Gold Ochsen", "city": "Ulm", "country": "Deutschland", "id": 1 }

Auflösung: id 1, weil die Werte aus dem Body die vergebene id überschreiben

BEISPIEL

- ...data steht in create nach id und überschreibt sie.
- Jetzt haben zwei Brauereien die id 1.
- GET /breweries/1 liefert weiter die Privatbrauerei Schlumberger.
- Einheit 7 entfernt Felder, die das Schema nicht kennt.

```
HTTP/1.1 201 Created
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 75

{
  "id": 1,
  "name": "Brauerei Gold Ochsen",
  "city": "Ulm",
  "country": "Deutschland"
}
```

Einheit 6 · Service und Dependency Injection

FRAGE

Live-Frage: Was antwortet GET /breweries/5, nachdem ein Kommentar im Service geändert wurde?

GET http://localhost:3000/breweries/5

Auflösung: 404, weil der Neustart einen neuen Service mit vier Brauereien erzeugt

BEISPIEL

- npm run start:dev startet nach jeder Änderung einer Datei neu.
- Der neue Prozess erzeugt einen neuen Service.
- Das Array beginnt wieder mit den vier Brauereien aus dem Code.
- Daten, die einen Neustart überstehen: Einheit 12.

```
HTTP/1.1 404 Not Found
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 70

{
  "message": "Brewery 5 not found",
  "error": "Not Found",
  "statusCode": 404
}
```

Fehlersuche: Postman bekommt nach dem Speichern keine Antwort

BEISPIEL

- Bis eben antwortete GET /breweries mit vier Brauereien.
- Jetzt zeigt Postman keine Antwort und keinen Statuscode.
- breweries.controller.ts und breweries.service.ts sind unverändert.
- Welche Stelle verhindert, dass der Server antwortet?

```
GET http://localhost:3000/breweries
connect ECONNREFUSED 127.0.0.1:3000
```

Fehlersuche: Das Terminal nennt den Constructor, weil der Service unter providers fehlt

- Spur 1: Ein Neustart hilft nicht, der Start bricht jedes Mal ab.
- Spur 2: Das Terminal meldet Found 0 errors; TypeScript übersetzt also fehlerfrei.
- Spur 3: Darunter steht Nest can't resolve dependencies of the BreweriesController (?).
- Spur 4: npx eslint src meldet 'BreweriesService' is defined but never used in app.module.ts.
- Ursache: Der Constructor verlangt einen Service, den niemand bei NestJS angemeldet hat.

Übung

Die Aufgaben stehen vollständig im Skript

Aufgaben dieser Einheit

ÜBUNG

1. Aufgabe 6.1: Ein Bier anlegen
2. Aufgabe 6.2: Ein Bier löschen
3. Zusatz 6.3: Biere nach Stil filtern

SELBSTKONTROLLE

- Beispiel zum Übernehmen: Skript 6.7 bis 6.14
- Aufgaben ab Skript 6.15, Hinweise ab 6.20
- Lösungsstand nach der Übung

Review: zwei Lösungen für Aufgabe 6.2

- Was ist gleich, was ist anders?
- Wie entfernt remove das Bier aus dem Array?
- Was antwortet DELETE für eine id, die es nicht gibt?
- Was ändert sich beim Wechsel zur Datenbank, was bleibt?

MERKSATZ

DELETE antwortet heute mit 200 und leerem Body. Einheit 8 macht daraus 204 und 404 für eine unbekannte id.

Das solltest du jetzt können

- ✓ Begründen, welcher Code in den Controller und welcher in den Service gehört
- ✓ Einen Service mit der Nest CLI erzeugen und im Constructor anfordern
- ✓ Endpunkte mit @Post, @Body und @Delete bauen
- ✓ Erklären, warum angelegte Daten einen Neustart nicht überstehen
- ✓ Die Meldung „can't resolve dependencies“ auf einen fehlenden Provider zurückführen

WEB-PROGRAMMIERUNG · EINHEIT 7

DTOs und Validierung mit Zod

Einheit 7 · DTOs und Validierung mit Zod

FRAGE

Ein Bier mit 25 % Alkohol im Jahresbericht: Was kostet der späte Fehler?

Webseite, App und Berichte der regionalen Brauereien lesen dieselben Daten aus der Beer API. Seit Einheit 6 nimmt die API jeden Body an.

Aufgabe: Als Betreiber möchte ich, dass die API nur gültige Biere annimmt

- Damit gelangen keine fehlerhaften Daten in die App.
- Ohne Namen, mit leerem Namen oder unbekanntem Bierstil antwortet POST /beers mit 400.
- Ebenso bei einem Alkoholgehalt außerhalb von 0 bis 20 oder als String.
- Jede Antwort 400 nennt das Feld; das Wildbräu Grafing Pils wird weiter angelegt.

MERKSATZ

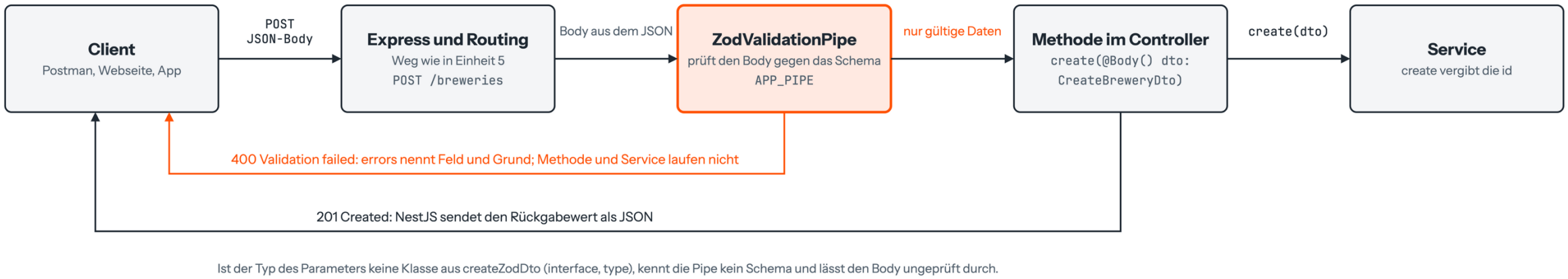
Das Live-Coding zeigt das Schema an den Brauereien. Das Schema für die Biere schreibst du selbst.

Der Server prüft jede Eingabe, auch wenn der Client schon prüft

- **Prüfung im Client**
- Meldet einen Fehler sofort beim Tippen
- Gilt nur für diese eine Webseite oder App
- Postman, curl und fremde Clients umgehen sie

- **Prüfung im Server**
- Letzte Stelle vor dem Speichern
- Gilt für jeden Client, der die API aufruft
- Hier entscheidet sich, was in den Bericht kommt

Eine Pipe prüft Parameter, bevor die Methode im Controller läuft



Ist der Typ des Parameters keine Klasse aus createZodDto (interface, type), kennt die Pipe kein Schema und lässt den Body ungeprüft durch.

Der Client sendet POST /breweries mit einem JSON-Body. Nach Express und Routing prüft die ZodValidationPipe den Body gegen das Schema. Ist er ungültig, antwortet sie mit 400 Validation failed; Methode und Service laufen nicht. Ist er gültig, ruft NestJS die Methode im Controller auf, die den Service aufruft, und antwortet mit 201. Ist der Typ des Parameters keine Klasse aus createZodDto, lässt die Pipe den Body ungeprüft durch.

Ein DTO beschreibt, welche Daten ein Endpunkt als Eingabe annimmt

- DTO steht für Data Transfer Object.
- CreateBreweryDto beschreibt den Body von POST /breweries.
- Die id fehlt im DTO, weil der Service sie vergibt.
- Das DTO ist deshalb nicht dasselbe wie die gespeicherte Brewery.
- Konvention: Datei create-brewery.dto.ts im Ordner dto der Ressource.

MERKSATZ

Zod prüft Daten zur Laufzeit gegen ein Schema. Mit nestjs-zod entsteht das DTO aus diesem Schema.

Ein Zod-Schema setzt sich aus Bausteinen für Felder und Grenzen zusammen

Baustein	Bedeutung	ungültig wäre
z.object({ … })	ein Objekt mit diesen Feldern	ein String, ein fehlender Body
z.string().min(1)	String mit mindestens einem Zeichen	"" oder 42
z.number().min(1).max(100)	Zahl von 1 bis 100, Grenzen eingeschlossen	0, 101 oder "50"
z.number().int()	ganze Zahl	2.5
z.enum(['klein', 'groß'])	genau einer der aufgezählten Werte	"mittel"
.optional()	das Feld darf fehlen	–

Ein Typ verschwindet beim Kompilieren, eine Klasse aus createZodDto trägt das Schema

- **Typ aus dem Schema**
- z.infer<typeof CreateBrewerySchema>
- Hat dieselben Felder wie das Schema
- Gilt nur beim Kompilieren, im JavaScript fehlt er

- **Klasse aus dem Schema**
- extends createZodDto(CreateBrewerySchema)
- Hat den Typ und zusätzlich das Schema
- Bleibt zur Laufzeit erhalten, die Pipe prüft sie

Die Pipe ruft parse auf und macht aus dem ZodError eine Antwort 400

- **parse(data)**
- Gibt die gültigen Daten zurück
- Wirft bei ungültigen Daten einen ZodError
- Die ZodValidationPipe ruft parse auf

- **safeParse(data)**
- Wirft nie
- Ergebnis { success, data } oder { success, error }
- Passt, wenn der eigene Code danach entscheidet

Eine Antwort 400 der Pipe nennt je Fehler das Feld und den Grund

Feld	Bedeutung	Beispiel
statusCode	Statuscode, wie in der Statuszeile	400
message	immer derselbe Text der Pipe	Validation failed
errors	ein Eintrag je Fehler; ein Body kann mehrere haben	[…]
path	Feld im Body; leer heißt: der ganze Body	["name"]
code	Art des Fehlers	invalid_type, too_small, invalid_value
message im Eintrag	Grund in Worten	Invalid input: expected string, received number

partial() macht jedes Feld optional und eignet sich für Änderungen mit PATCH

- Schema.partial() erzeugt dasselbe Schema, in dem jedes Feld fehlen darf.
- Ein fehlendes Feld bleibt beim Ändern unverändert.
- Ein vorhandenes Feld prüft weiter die Regel aus dem ursprünglichen Schema.
- @Patch(':id') steht an der Methode für PATCH auf ein einzelnes Element.

MERKSATZ

PATCH ändert nur die gesendeten Felder, PUT ersetzt das ganze Element (Folie 4.10).

Schritt 1: Die Pipe ist seit dem Basisprojekt für alle Endpunkte registriert

TYPESCRIPT · SRC/APP.MODULE.TS

```
providers: [
  // Prüft Body, Query und Pfadparameter gegen die Zod-Schemas der DTOs
  { provide: APP_PIPE, useClass: ZodValidationPipe },
  // Prüft Antworten gegen Zod-Schemas, wenn ein Endpunkt eines angibt
  { provide: APP_INTERCEPTOR, useClass: ZodSerializerInterceptor },
  BreweriesService,
  BeersService,
],
```

Schritt 2: Schema und DTO für eine neue Brauerei

TYPESCRIPT · SRC/BREWERIES/DTO/CREATE-BREWERY.DTO.TS

```typescript
import { createZodDto } from 'nestjs-zod';
import { z } from 'zod';

// Gültige Daten für eine neue Brauerei; die id vergibt der Service
export const CreateBrewerySchema = z.object({
  name: z.string().min(1),
  city: z.string().min(1),
  country: z.string().min(1),
});

// Klasse aus dem Schema: Nur eine Klasse prüft die ZodValidationPipe
export class CreateBreweryDto extends createZodDto(CreateBrewerySchema) {}
```

Schritt 3: Controller und Service nehmen das DTO statt Omit<Brewery, 'id'>

TYPESCRIPT · SRC/BREWERIES/BREWERIES.CONTROLLER.TS

```
@Post()
create(@Body() dto: CreateBreweryDto): Brewery {
  return this.breweriesService.create(dto);
}
```

Schritt 4: Ein Name als Zahl ergibt 400, und der Service läuft nicht

BEISPIEL

- Der Body hat die Zahl 42 als name.
- path nennt das Feld name.
- code invalid_type: Der Wert hat den falschen Typ.
- Ein leeres Objekt ergibt drei Einträge: name, city, country.

```
{
 "statusCode": 400,
 "message": "Validation failed",
 "errors": [{
  "expected": "string",
  "code": "invalid_type",
  "path": ["name"],
  "message":
  "Invalid input: expected string, received number"
 }]
}
```

Einheit 7 · DTOs und Validierung mit Zod

FRAGE

Live-Frage: Was antwortet die API auf eine gültige Brauerei mit einem zusätzlichen Feld?

POST http://localhost:3000/breweries mit { "name": "Brauerei Gold Ochsen", "city": "Ulm", "country": "Deutschland", "founded" : 1516 }

Auflösung: 201 ohne founded, weil Zod unbekannte Felder entfernt

BEISPIEL

- z.object übernimmt nur die Felder des Schemas.
- Unbekannte Felder entfernt Zod ohne Meldung.
- Auch die id 1 aus Live-Frage 6.19 erreicht den Service nicht mehr.
- Ab Einheit 13: .strict() lehnt unbekannte Felder mit 400 ab.

```
HTTP/1.1 201 Created
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 75

{
  "id": 6,
  "name": "Brauerei Gold Ochsen",
  "city": "Ulm",
  "country": "Deutschland"
}
```

Einheit 7 · DTOs und Validierung mit Zod

FRAGE

Live-Frage: Ist ein Name aus drei Leerzeichen gültig?

POST http://localhost:3000/breweries mit { "name": "   ", "city": "Ulm", "country": "Deutschland" }

Auflösung: 201, weil min(1) auch Leerzeichen als Zeichen zählt

BEISPIEL

- min(1) zählt Zeichen; ein Leerzeichen ist ein Zeichen.
- Die neue Brauerei heißt " ".
- .trim() vor .min(1) entfernt Leerzeichen am Rand vor der Prüfung.
- Ob ein Feld das braucht, entscheidet seine Regel im Datenmodell.

```
HTTP/1.1 201 Created
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 58

{
  "id": 7,
  "name": "  ",
  "city": "Ulm",
  "country": "Deutschland"
}
```

Einheit 7 · DTOs und Validierung mit Zod

FRAGE

Live-Frage: Was antwortet die API jetzt auf GET /breweries/abc?

GET http://localhost:3000/breweries/abc

Auflösung: weiter 404, weil die Pipe nur Parameter mit einer DTO-Klasse prüft

BEISPIEL

- @Param('id') hat den Typ string, keine Klasse aus createZodDto.
- Die Pipe lässt den Pfadparameter deshalb ungeprüft durch.
- Number('abc') ergibt NaN, findOne findet keine Brauerei.
- Eine Pipe, die Pfadparameter in Zahlen wandelt, zeigt Einheit 8.

```
HTTP/1.1 404 Not Found
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 72

{
  "message": "Brewery abc not found",
  "error": "Not Found",
  "statusCode": 404
}
```

Fehlersuche: POST /breweries mit leerem Objekt legt wieder eine Brauerei an

BEISPIEL

- Bis eben antwortete dieselbe Anfrage mit 400 und drei Einträgen.
- Jetzt kommt 201 mit einer Brauerei, die nur eine id hat.
- Das Terminal meldet keinen Fehler.
- Welche Stelle lässt den Body ungeprüft durch?

```
POST http://localhost:3000/breweries
Content-Type: application/json

{}

HTTP/1.1 201 Created
{"id":5}
```

Fehlersuche: Die Pipe prüft nichts, weil der Parameter von @Body() nur einen Typ hat

- Spur 1: CreateBrewerySchema enthält für jedes Feld weiter min(1).
- Spur 2: Das Terminal meldet Found 0 errors; TypeScript übersetzt fehlerfrei.
- Spur 3: Der Parameter von @Body() hat den Typ CreateBreweryInput.
- Spur 4: CreateBreweryInput ist ein type aus z.infer, keine Klasse.
- Ursache: Der Typ fehlt zur Laufzeit; ohne Klasse kennt die Pipe kein Schema.

Übung

Die Aufgaben stehen vollständig im Skript

Aufgaben dieser Einheit

ÜBUNG

1. Aufgabe 7.1: Pflichtfelder und Typen prüfen
2. Aufgabe 7.2: Bierstil und Alkoholgehalt prüfen
3. Zusatz 7.3: Ein Bier ändern

SELBSTKONTROLLE

- Beispiel zum Übernehmen: Skript 7.7 bis 7.12
- Aufgaben ab Skript 7.13, Hinweise ab 7.20
- Lösungsstand nach der Übung

Review: zwei Lösungen für die Aufgaben 7.1 und 7.2

- Was ist gleich, was ist anders?
- Welches Feld nennt die Antwort bei abv "5.2", und warum?
- Wie legt das Schema die Grenzen 0 und 20 fest?
- Welche Kosten aus der Frage zum Einstieg vermeidet das Schema?

MERKSATZ

Einheit 8 bringt alle Fehler der API in ein Format, auch 404 und die Antwort der Pipe.

Das solltest du jetzt können

- ✓ Begründen, warum der Server Eingaben prüft, auch wenn der Client prüft
- ✓ Ein Zod-Schema mit Pflichtfeldern, Aufzählung und Grenzen schreiben
- ✓ Aus einem Schema ein DTO erzeugen und an einem Endpunkt verwenden
- ✓ In einer Antwort 400 das fehlerhafte Feld und den Grund benennen
- ✓ Erklären, warum ein TypeScript-Typ allein nichts prüft

PlanB. Trusted Agentic Operator

DHBW Duale Hochschule Baden-Württemberg Heidenheim

WEB-PROGRAMMIERUNG · EINHEIT 8

Fehlerbehandlung und Statuscodes im Backend

Einheit 8 · Fehlerbehandlung und Statuscodes im Backend

FRAGE

Zwei Meldungen auf GET /breweries/abc: Welche spart ein Support-Ticket?

Meldung A: 404 „Brewery abc not found“. Meldung B: 400 „Validation failed (numeric string is expected)“. Webseite, App und die Webseiten der Brauereien zeigen, was die Beer API meldet.

Aufgabe: Als Betreiber möchte ich passende Statuscodes und verständliche Meldungen

- Damit zeigen Webseite und App Fehler an, und Gäste fragen seltener beim Support nach.
- DELETE /beers/5 antwortet mit 204 ohne Body, ein zweites Mal mit 404.
- GET /beers/abc antwortet mit 400, weil abc keine Zahl ist.
- Jede Fehlerantwort hat statusCode, message und error und keinen Stacktrace.

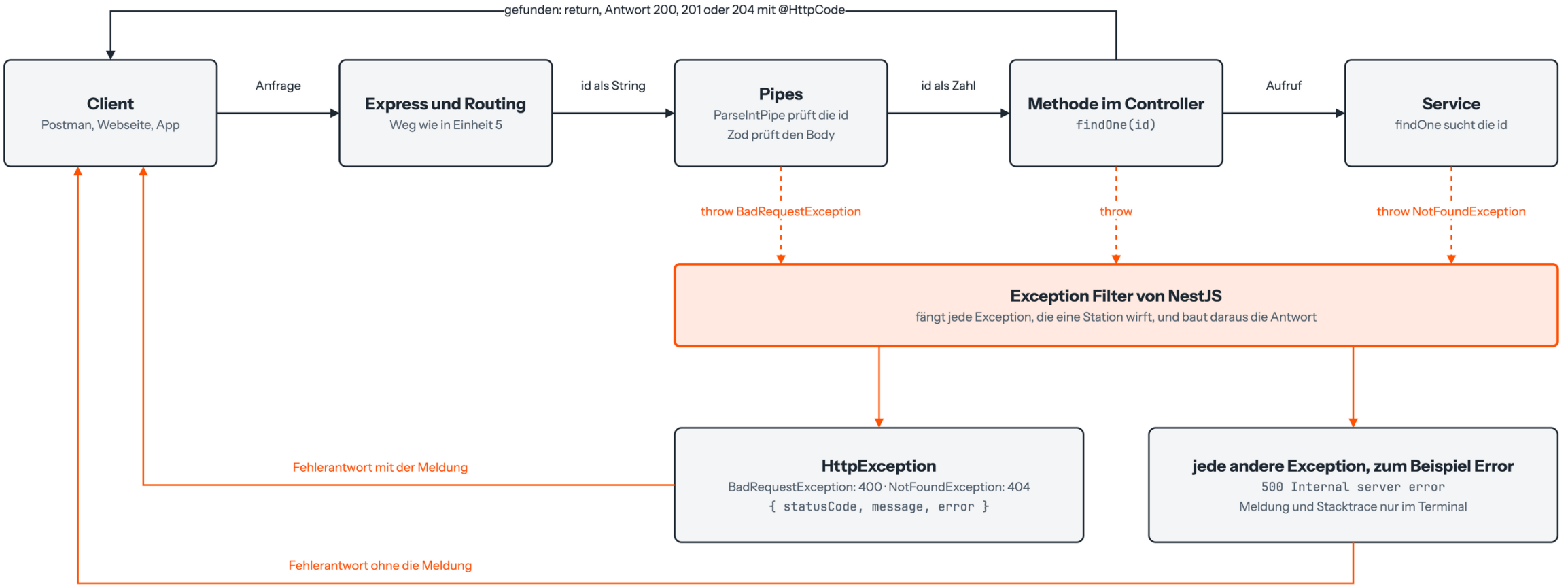
MERKSATZ

Das Live-Coding zeigt die Fehler an den Brauereien. Die Fehler der Biere baust du selbst.

Jede Situation der Beer API hat einen Statuscode, der sagt, wer handeln muss

Situation in der Beer API	Code	Wer handelt
Brauerei angelegt; die Beer API sendet sie zurück	201	niemand
Brauerei gelöscht; die Antwort hat keinen Body	204	niemand
id ist keine ganze Zahl, oder der Body ist ungültig	400	der Client korrigiert die Anfrage
Anfrage richtig, aber die Brauerei mit dieser id fehlt	404	der Client prüft die id
Fehler im Code des Servers	500	der Betreiber

Jede Station kann eine Exception werfen; NestJS baut daraus die Antwort



Der Client sendet eine Anfrage. Sie läuft über Express und Routing, die Pipes und die Methode im Controller bis zum Service. Findet der Service die id, gibt die Methode das Ergebnis zurück, und NestJS antwortet mit 200, 201 oder 204. Wirft eine Station eine Exception, fängt sie der Exception Filter von NestJS. Aus einer HttpException wie BadRequestException oder NotFoundException wird eine Antwort mit statusCode, message und error. Aus jeder anderen Exception wird 500 Internal server error; Meldung und Stacktrace stehen nur im Terminal.

NestJS hat für die üblichen Fehler eine Exception mit festem Statuscode

Exception	Code	Situation in der Beer API
BadRequestException	400	Body oder Pfadparameter ungültig; wirft meist eine Pipe
NotFoundException	404	Bier oder Brauerei mit dieser id fehlt
HttpException	frei wählbar	Oberklasse der Exceptions darüber
Error oder eine andere Klasse	500	NestJS kennt keinen passenden Code

Eine unbekannte Exception wird zur 500, und der Stacktrace bleibt im Terminal

- Der Body einer 500 enthält nur statusCode und „Internal server error“.
- Die eigentliche Meldung und der Stacktrace stehen im Terminal des Servers.
- Ein Stacktrace listet die Funktionsaufrufe bis zum Fehler mit Dateien und Zeilen.
- In einer Antwort zeigt er Pfade, Bibliotheken und den Aufbau des Servers.

MERKSATZ

Der Server läuft nach einer 500 weiter und beantwortet die nächste Anfrage.

findOne im Service wirft die 404, damit jeder Aufrufer dieselbe Antwort bekommt

- Bisher prüft jeder Controller selbst auf undefined und wirft die 404.
- Wirft findOne im Service, erben GET und DELETE dieselbe 404.
- Der Rückgabetyp wird Brewery statt Brewery | undefined.
- Kein Aufrufer muss mehr auf undefined prüfen.

MERKSATZ

Einheit 6 hat vereinfacht: Der Service kennt kein HTTP. Ab heute wirft er eine NotFoundException.

ParseIntPipe prüft die id vor der Methode, @HttpCode setzt den Code für Erfolg

- **@Param('id', ParseIntPipe)**
- Macht aus dem String eine ganze Zahl
- Keine ganze Zahl: 400, die Methode läuft nicht
- Number(id) im Controller entfällt

- **@HttpCode(HttpStatus.NO_CONTENT)**
- Ohne Angabe: POST 201, alle anderen 200
- Setzt 204 für ein erfolgreiches DELETE
- Wirft die Methode, gilt der Code der Exception

Die Pipe aus Einheit 7 antwortet in einem anderen Format als die NotFoundException

- **ZodValidationPipe, Einheit 7**
- Felder statusCode, message, errors
- message ist immer „Validation failed“
- kein Feld error

- **NotFoundException**
- Felder message, error, statusCode
- message nennt die Ursache
- error ist „Not Found“

Schritt 1: findOne im Service wirft die 404

TYPESCRIPT · SRC/BREWERIES/BREWERIES.SERVICE.TS

```
findOne(id: number): Brewery {
  const brewery = this.breweries.find((b) => b.id === id);
  if (!brewery) {
    // Jeder Aufrufer bekommt dieselbe 404
    throw new NotFoundException(`Brewery ${id} not found`);
  }
  return brewery;
}
```

Schritt 2: Der Controller prüft nur noch, ob die id eine ganze Zahl ist

TYPESCRIPT · SRC/BREWERIES/BREWERIES.CONTROLLER.TS

```
@Get(':id')
findOne(@Param('id', ParseIntPipe) id: number): Brewery {
  // Keine ganze Zahl: ParseIntPipe antwortet mit 400
  return this.breweriesService.findOne(id);
}
```

Einheit 8 · Fehlerbehandlung und Statuscodes im Backend

FRAGE

Live-Frage: Was antwortet die API auf GET /breweries/2.5?

GET http://localhost:3000/breweries/2.5

Auflösung: 400, weil ParseIntPipe den Text prüft, bevor sie umwandelt

BEISPIEL

- Erlaubt sind nur Ziffern, davor höchstens ein Minus.
- 2.5 besteht die Prüfung nicht.
- Die Methode und findOne laufen nicht.
- Für Kommazahlen gibt es ParseFloatPipe; jede id der Beer API ist ganzzahlig.

```
HTTP/1.1 400 Bad Request
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 99

{
 "message":
 "Validation failed (numeric string is expected)",
 "error": "Bad Request",
 "statusCode": 400
}
```

Einheit 8 · Fehlerbehandlung und Statuscodes im Backend

FRAGE

Live-Frage: Was antwortet die API auf GET /breweries/-1?

GET http://localhost:3000/breweries/-1

Auflösung: 404, weil die Pipe die Form prüft und erst der Service die Daten

BEISPIEL

- -1 ist eine ganze Zahl und besteht die Pipe.
- Erst findOne sucht und findet keine Brauerei.
- Ob eine id existiert, weiß nur der Service.
- Eine untere Grenze für die id prüft die Beer API nicht.

```
HTTP/1.1 404 Not Found
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 71

{
  "message": "Brewery -1 not found",
  "error": "Not Found",
  "statusCode": 404
}
```

Schritt 3: Löschen antwortet mit 204

TYPESCRIPT · SRC/BREWERIES/BREWERIES.CONTROLLER.TS

```
@Delete(':id')
@HttpCode(HttpStatus.NO_CONTENT)
remove(@Param('id', ParseIntPipe) id: number): Brewery {
  return this.breweriesService.remove(id);
}
```

Einheit 8 · Fehlerbehandlung und Statuscodes im Backend

FRAGE

Live-Frage: Was steht im Body der Antwort auf DELETE /breweries/3?

DELETE http://localhost:3000/breweries/3 – die Methode gibt das gelöschte Augustiner-Bräu zurück.

Auflösung: Der Body ist leer, weil eine Antwort 204 nie einen Body hat

BEISPIEL

- 204 No Content heißt: keine Daten in der Antwort.
- Express sendet den Rückgabewert der Methode nicht.
- Die Antwort hat deshalb auch keinen Content-Type.
- Wer das gelöschte Element zurückgeben will, antwortet mit 200.

```
HTTP/1.1 204 No Content
X-Powered-By: Express
ETag: W/"4c-uYrhk8DubXNt1vNbEGA5etD8tlg"
Date: Sun, 25 Oct 2026 16:49:30 GMT
Connection: keep-alive
Keep-Alive: timeout=5
```

Schritt 4: Validierungsfehler im Format aller anderen Fehler

TYPESCRIPT · SRC/APP.MODULE.TS

```
// Validierungsfehler im Format aller anderen Fehler
const ValidationPipe = createZodValidationPipe({
  createValidationException: (error) =>
    new BadRequestException(
      (error as ZodError).issues.map(
        (issue) => `${issue.path.join('.')}: ${issue.message}`,
      ),
    ),
});
```

Ein leerer Body ergibt jetzt dieselbe Form wie eine 404

BEISPIEL

- POST /breweries mit {} antwortet weiter mit 400.
- message ist hier eine Liste, bei einer 404 ein Text.
- error und statusCode stehen wie bei jeder Exception.
- Eine App liest ab jetzt ein Format für alle Fehler.

```
{
  "message": [
    "name: Invalid input: expected string, …",
    "city: Invalid input: expected string, …",
    "country: Invalid input: expected string, …"
  ],
  "error": "Bad Request",
  "statusCode": 400
}
```

Fehlersuche: GET /breweries/9 antwortet plötzlich mit 500

BEISPIEL

- Bis eben antwortete dieselbe Anfrage mit 404 und „Brewery 9 not found“.
- Jetzt kommt 500 ohne die Meldung.
- GET /breweries/2 antwortet weiter mit 200.
- Welche Stelle erzeugt die 500?

```
GET http://localhost:3000/breweries/9

HTTP/1.1 500 Internal Server Error
{"statusCode":500,"message":"Internal server error"}
```

Fehlersuche: Error ist keine HttpException, deshalb kennt NestJS keinen Statuscode

- Spur 1: GET /breweries/2 antwortet mit 200; der Server läuft.
- Spur 2: Terminal: ERROR [ExceptionsHandler] Error: Brewery 9 not found.
- Spur 3: Die erste Zeile des Stacktrace zeigt breweries.service.ts, Zeile 45.
- Spur 4: Dort steht throw new Error statt einer HttpException.
- Ursache: Für Error kennt NestJS keinen Code und antwortet mit 500.

Übung

Die Aufgaben stehen vollständig im Skript

Aufgaben dieser Einheit

ÜBUNG

1. Aufgabe 8.1: Situation und Statuscode zuordnen
2. Aufgabe 8.2: Fehler der Biere erkennbar machen
3. Zusatz 8.3: Ein Bier mit Fehlern ändern

SELBSTKONTROLLE

- Beispiel zum Übernehmen: Skript 8.6 bis 8.12
- Aufgaben ab Skript 8.13, Hinweise ab 8.20
- Lösungsstand nach der Übung

Review: zwei Lösungen für die Aufgaben 8.1 und 8.2

- Was ist gleich, was ist anders?
- Welche Zeile der Tabelle war umstritten, und was antwortet die API?
- An welcher Stelle prüft die Lösung, ob ein Bier existiert?
- Welche Meldung aus der Frage zum Einstieg spart jetzt ein Ticket?

MERKSATZ

Einheit 9 prüft, ob die Brauerei eines neuen Biers existiert.

Das solltest du jetzt können

- ✓ Zu einer Situation den passenden Statuscode nennen und begründen
- ✓ Mit NotFoundException, ParseIntPipe und @HttpCode die passende Antwort erzeugen
- ✓ Erklären, warum eine Antwort 500 keinen Stacktrace enthält und wo er steht
- ✓ Begründen, warum findOne im Service die NotFoundException wirft
- ✓ Prüfen, ob zwei Fehlerantworten dasselbe Format haben

WEB-PROGRAMMIERUNG · EINHEIT 9

Module und Projektstruktur

Einheit 9 · Module und Projektstruktur

FRAGE

Zwei Teams, eine API: Was muss das Team der Biere über die Brauereien wissen?

Der Betreiber lässt die Beer API von zwei Teams weiterbauen: eines für die Brauereien, eines für Biere und Bewertungen. Jede Abstimmung zwischen den Teams verzögert Änderungen, die die Brauereien bestellen.

Aufgabe: Als Betreiber möchte ich, dass jedes Bier zu einer existierenden Brauerei gehört

- Damit sehen Gäste, woher ein Bier kommt.
- POST /beers mit "breweryId": 3 legt ein Bier des Augustiner-Bräu an.
- POST /beers mit "breweryId": 42 wird abgelehnt; eine Brauerei 42 gibt es nicht.
- Du wählst 400 oder 404 und begründest die Wahl im Review.

MERKSATZ

Das Live-Coding zeigt die Brauereien. Die Biere baust du selbst.

Heute nimmt die API ein Bier einer Brauerei an, die es nicht gibt

HTTP

```
POST http://localhost:3000/beers
{"name":"Augustiner Dunkel","style":"dunkel","abv":5.6,"breweryId":42}

HTTP/1.1 201 Created
Content-Type: application/json; charset=utf-8

{"id":5,"name":"Augustiner Dunkel","style":"dunkel","abv":5.6,
 "breweryId":42}
```

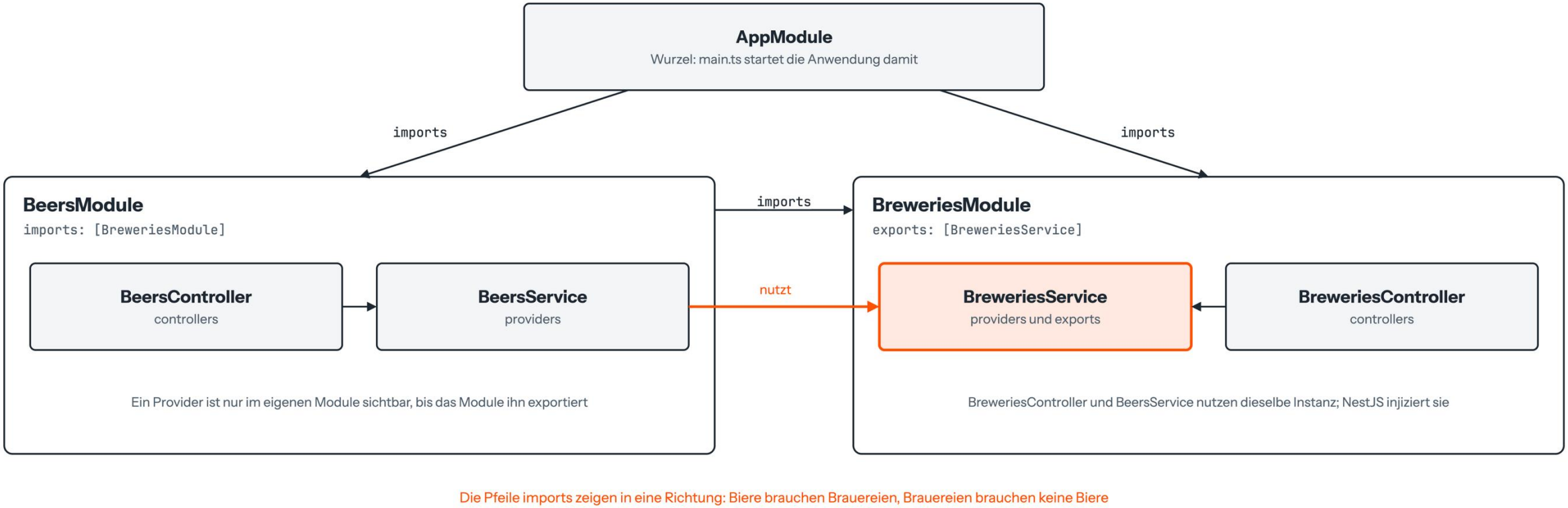
Conway’s Law: Die Struktur eines Systems folgt der Kommunikation der Organisation

- Zwei Teams, die wenig miteinander reden, bauen zwei getrennte Teile.
- Die Grenze zwischen den Teilen ist eine Absprache zwischen den Teams.
- In NestJS steht diese Absprache in den exports eines Module.
- Was ein Module nicht exportiert, ändert sein Team ohne Rückfrage.

Ein Module ordnet die Klassen einer Ressource in vier Listen

Liste in @Module	Bedeutung	Im BreweriesModule
controllers	Endpunkte, die das Module anbietet	BreweriesController
providers	Klassen, die NestJS im Module erzeugt und injiziert	BreweriesService
exports	Provider, die andere Module nutzen dürfen	BreweriesService
imports	Module, deren Exporte dieses Module nutzt	keine

Module importieren die Exporte anderer Module; die Pfeile zeigen in eine Richtung



Die Pfeile imports zeigen in eine Richtung: Biere brauchen Brauereien, Brauereien brauchen keine Biere

AppModule ist die Wurzel und importiert BeersModule und BreweriesModule. BeersModule enthält BeersController und BeersService, BreweriesModule enthält BreweriesController und BreweriesService. BreweriesModule exportiert den BreweriesService. BeersModule importiert BreweriesModule; deshalb kann der BeersService den BreweriesService nutzen, NestJS injiziert ihn. BreweriesController und BeersService nutzen dieselbe Instanz. Ein Provider ist nur im eigenen Module sichtbar, bis das Module ihn exportiert. Die Pfeile imports zeigen in eine Richtung: Biere brauchen Brauereien,

Jede Ressource bekommt einen Ordner mit ihrem Module

```
src/
 ├── main.ts
 ├── app.module.ts
 ├── app.controller.ts
 ├── breweries/
 │    ├── breweries.module.ts
 │    ├── breweries.controller.ts
 │    ├── breweries.service.ts
 │    └── dto/
 │         └── create-brewery.dto.ts
 └── beers/
```

Wo ein Service steht, entscheidet, wer ihn bekommt

Eintrag für BreweriesService	Bedeutung	Ergebnis
providers und exports im BreweriesModule	sichtbar für jedes Module, das BreweriesModule importiert	BeersService bekommt dieselbe Instanz
providers ohne exports, BeersService verlangt ihn	im BeersModule nicht verfügbar	Der Server startet nicht
zusätzlich unter providers im BeersModule	das BeersModule erzeugt eine zweite Instanz	neue Brauereien fehlen dort

Fehlt ein benötigter Provider, bricht NestJS den Start ab und nennt die Stelle

```
ERROR [ExceptionHandler] UnknownDependenciesException [Error]:
Nest can't resolve dependencies of the BeersService (?).
Please make sure that the argument BreweriesService
at index [0] is available in the BeersModule module.
...
```

Ohne forwardRef verhindern zwei Module, die sich gegenseitig importieren, den Start

- Das BeersModule importiert das BreweriesModule für die Prüfung der Brauerei.
- Importiert das BreweriesModule zusätzlich das BeersModule, entsteht eine zirkuläre Abhängigkeit.
- NestJS bricht den Start mit einer UndefinedModuleException ab.
- Ein Endpunkt, der beide Ressourcen braucht, gehört in das abhängige Module.

MERKSATZ

forwardRef() löst eine zirkuläre Abhängigkeit auf. Die Beer API braucht es nicht, weil ihre Abhängigkeiten in eine Richtung zeigen.

Schritt 1: Die Nest CLI erzeugt das Module und trägt es in AppModule ein

```
$ npx nest generate module breweries
CREATE src/breweries/breweries.module.ts (86 bytes)
UPDATE src/app.module.ts (1369 bytes)
```

Schritt 2: Controller und Service ziehen in das BreweriesModule

TYPESCRIPT · SRC/BREWERIES/BREWERIES.MODULE.TS

```typescript
import { Module } from '@nestjs/common';
import { BreweriesController } from './breweries.controller';
import { BreweriesService } from './breweries.service';

@Module({
  controllers: [BreweriesController],
  providers: [BreweriesService],
  // Andere Module dürfen BreweriesService injizieren
  exports: [BreweriesService],
})
export class BreweriesModule {}
```

Schritt 3: AppModule gibt Controller und Service der Brauereien ab

TYPESCRIPT · SRC/APP.MODULE.TS

```
@Module({
  imports: [BreweriesModule],
  controllers: [AppController, BeersController],
  providers: [
    // Prüft Body, Query und Pfadparameter gegen die Zod-Schemas der DTOs
    { provide: APP_PIPE, useClass: ValidationPipe },
    // Prüft Antworten gegen Zod-Schemas, wenn ein Endpunkt eines angibt
    { provide: APP_INTERCEPTOR, useClass: ZodSerializerInterceptor },
    BeersService,
  ],
})
export class AppModule {}
```

Das Terminal zeigt eine Zeile je Module und je Controller

```
LOG [InstanceLoader] BreweriesModule dependencies initialized
LOG [InstanceLoader] AppModule dependencies initialized
LOG [RoutesResolver] AppController {/}:
LOG [RoutesResolver] BeersController {/beers}:
LOG [RoutesResolver] BreweriesController {/breweries}:
…
```

Einheit 9 · Module und Projektstruktur

FRAGE

Live-Frage: Was antwortet GET /breweries, wenn exports im BreweriesModule fehlt?

GET http://localhost:3000/breweries – vorher die Zeile exports: [BreweriesService] löschen.

Auflösung: 200, weil exports nur regelt, was andere Module bekommen

BEISPIEL

- Controller und Service stehen im selben Module.
- Innerhalb eines Module ist jeder Provider sichtbar.
- Noch verlangt kein anderes Module den BreweriesService.
- Fehlt der Export später, startet der Server nicht (Folie 9.10).

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Content-Length: 340

[{"id":1,"name":"Privatbrauerei Schlumberger",…},
 …
 {"id":4,"name":"Edelweissbrauerei Farny",…}]
```

Einheit 9 · Module und Projektstruktur

FRAGE

Live-Frage: Mit einer zweiten Instanz in AppModule: Was liefert GET /breweries/5 nach dem POST?

POST http://localhost:3000/breweries mit der Brauerei Gold Ochsen aus Ulm, danach GET /breweries/5.

Auflösung: 200, weil der Controller beide Male die Instanz seines Module nutzt

BEISPIEL

- Es gibt jetzt zwei Instanzen des BreweriesService.
- Der BreweriesController bekommt die aus dem BreweriesModule.
- POST und GET laufen über dieselbe Instanz.
- Die Instanz in AppModule nutzt niemand.

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Content-Length: 75

{"id":5,"name":"Brauerei Gold Ochsen","city":"Ulm",
 "country":"Deutschland"}
```

Einheit 9 · Module und Projektstruktur

FRAGE

Live-Frage: Der BreweriesController zieht zurück in AppModule: Was antwortet GET /breweries/3?

GET http://localhost:3000/breweries/3 – der BreweriesService bleibt im BreweriesModule.

Auflösung: 200, weil AppModule das BreweriesModule importiert und dessen Export bekommt

BEISPIEL

- Der Controller in AppModule verlangt den BreweriesService.
- AppModule importiert das BreweriesModule, und das exportiert den Service.
- Ohne exports startet der Server jetzt nicht.
- Genau so bekommt in der Übung der BeersService den BreweriesService.

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Content-Length: 76

{"id":3,"name":"Augustiner-Bräu","city":"München",
 "country":"Deutschland"}
```

Fehlersuche: GET /breweries antwortet plötzlich mit 404

BEISPIEL

- Bis eben antwortete dieselbe Anfrage mit 200 und vier Brauereien.
- Der Server startet ohne Fehlermeldung.
- GET /beers antwortet weiter mit 200.
- Welche Stelle erzeugt die 404?

```
GET http://localhost:3000/breweries

HTTP/1.1 404 Not Found
{"message":"Cannot GET /breweries","error":"Not Found",
 "statusCode":404}
```

Fehlersuche: NestJS erzeugt nur Module, die AppModule über imports erreicht

- Spur 1: breweries.module.ts ist vollständig.
- Spur 2: Terminal ohne „BreweriesModule dependencies initialized“ und ohne BreweriesController.
- Spur 3: npx eslint src meldet „'BreweriesModule' is defined but never used“.
- Ursache: Niemand importiert das BreweriesModule; der Compiler meldet das nicht.
- Beheben: BreweriesModule wieder unter imports in AppModule eintragen.

Übung

Die Aufgaben stehen vollständig im Skript

Aufgaben dieser Einheit

ÜBUNG

1. Aufgabe 9.1: Die Biere bekommen ein eigenes Module
2. Aufgabe 9.2: Nur Biere existierender Brauereien anlegen
3. Zusatz 9.3: Die Brauerei eines Biers ändern
4. Zusatz 9.4: Die Biere einer Brauerei abrufen

SELBSTKONTROLLE

- Beispiel zum Übernehmen: Skript 9.6 bis 9.10
- Aufgaben ab Skript 9.11, Hinweise ab 9.19
- Lösungsstand nach der Übung

Review: zwei Lösungen für Aufgabe 9.2

- 400 oder 404 für eine fehlende Brauerei: Welche Begründung überzeugt?
- Steht die Prüfung der Brauerei im Controller oder im Service?
- Was muss das Team der Biere über die Brauereien wissen?
- Welches Team darf den BreweriesService ändern, ohne zu fragen?

MERKSATZ

Einheit 10 zählt an derselben Regel alle Fälle auf, bevor Code entsteht.

Das solltest du jetzt können

- ✓ Ein Module mit der Nest CLI erzeugen und Controller und Service hineinverschieben
- ✓ Einen Service exportieren und in einem anderen Module injizieren
- ✓ Die Meldung „can't resolve dependencies“ auf einen fehlenden Import oder Export zurückführen
- ✓ Eine Regel umsetzen, die den Service eines anderen Module braucht
- ✓ Begründen, warum die Biere von den Brauereien abhängen und nicht umgekehrt

PlanB. Trusted Agentic Operator

DHBW Duale Hochschule Baden-Württemberg Heidenheim

WEB-PROGRAMMIERUNG · EINHEIT 10

Anforderungen in Code übersetzen

Einheit 10 · Anforderungen in Code übersetzen

FRAGE

Biere filtern und sortieren: Woran erkennt die Brauerei, dass es fertig ist?

Als Brauerei möchte ich Biere filtern und sortieren und die API anhand einer Dokumentation anbinden, damit ich die Daten auf meiner eigenen Webseite zeige.

Aufgabe: Als Gast möchte ich ein Bier mit 1 bis 5 Sternen bewerten

- Damit sehen andere Gäste, was gut ankommt.
- POST /beers/1/ratings mit 4 Sternen legt eine Bewertung an.
- Eine zweite Bewertung desselben Gasts ersetzt seine erste.
- GET /beers/1 zeigt den Durchschnitt mit einer Nachkommastelle, ohne Bewertung null.

MERKSATZ

Die Fälle zu jeder Regel stehen als Kommentar über der Methode, bevor der Code entsteht.

Ein Akzeptanzkriterium ist eine Anfrage mit erwarteter Antwort

- Konkret: Es nennt Werte aus den Beispieldaten statt „funktioniert“.
- Prüfbar: Statuscode und Inhalt der Antwort stehen fest.
- Vollständig: Fehlerfälle gehören dazu, ebenso der normale Weg.
- Verbindlich: Was nicht als Kriterium dasteht, gilt nicht als vereinbart.

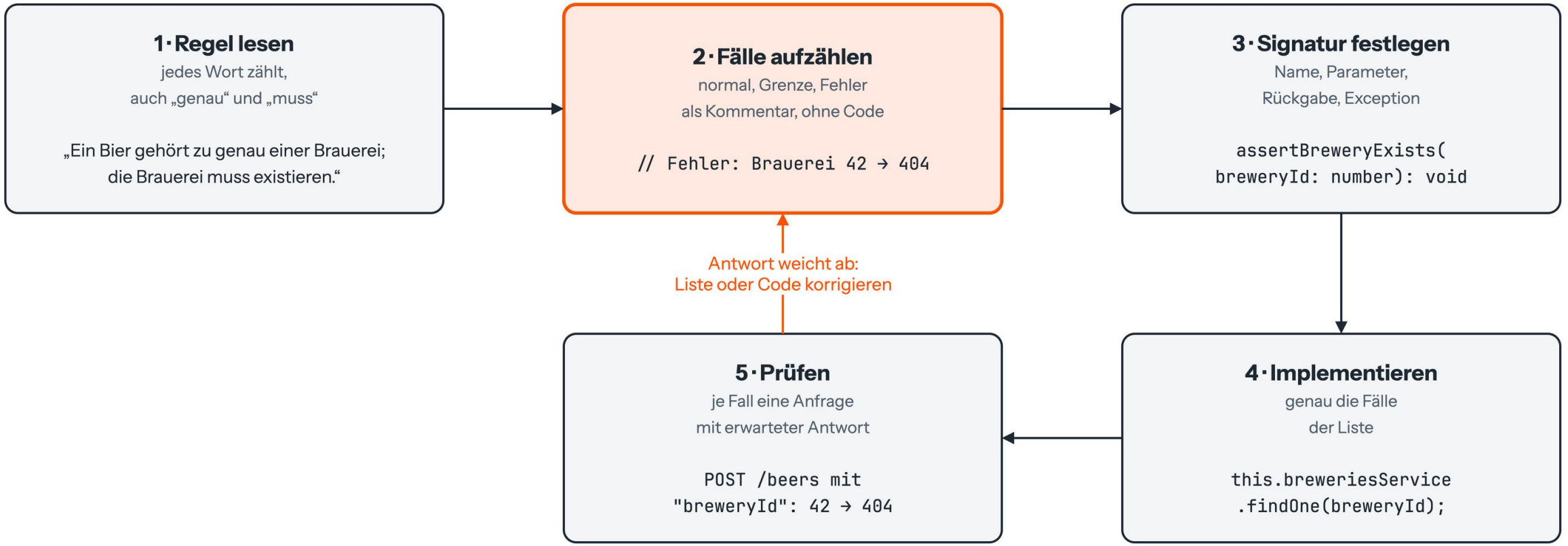
MERKSATZ

„Bewerten funktioniert“ wird zu: POST /beers/42/ratings antwortet mit 404.

Eine User Story nennt Rolle, Funktion und Nutzen; die Kriterien legen die Details fest

Teil	Bedeutung	In der Aufgabe dieser Einheit
Rolle	wer etwas will	Als Gast
Funktion	was die Rolle tun kann	ein Bier mit 1 bis 5 Sternen bewerten
Nutzen	warum es sich für den Auftraggeber lohnt	andere Gäste sehen, was gut ankommt
Akzeptanzkriterium	Anfrage und erwartete Antwort	POST /beers/42/ratings → 404
Geschäftsregel	gilt ab ihrer Story für alle folgenden	Ein User bewertet ein Bier höchstens einmal.

Fünf Schritte führen von der Regel zur geprüften Methode

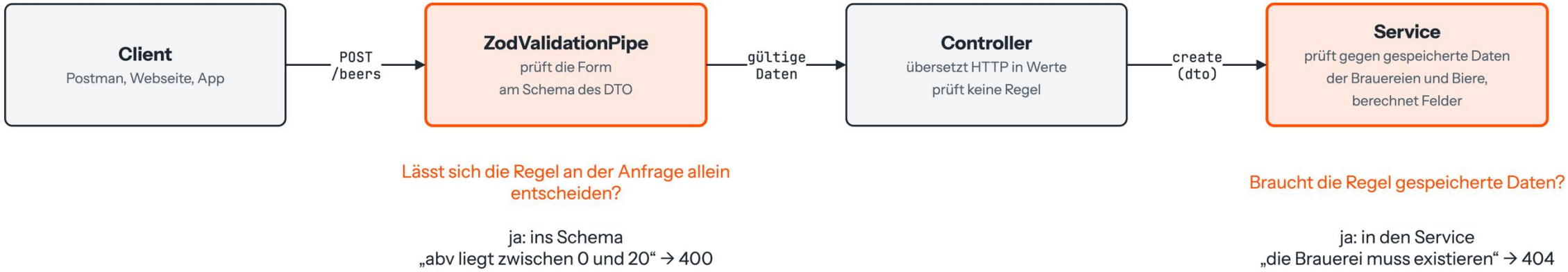


Fünf Schritte im Kreis, je Schritt ein Beispiel an der Regel für Brauereien. 1 Regel lesen, jedes Wort zählt, auch „genau“ und „muss“: „Ein Bier gehört zu genau einer Brauerei; die Brauerei muss existieren.“ 2 Fälle aufzählen: normal, Grenze, Fehler, als Kommentar, ohne Code: „Fehler: Brauerei 42 → 404“. 3 Signatur festlegen: Name, Parameter, Rückgabe, Exception: assertBreweryExists(breweryId: number): void. 4 Implementieren: genau die Fälle der Liste: this.breweriesService.findOne(breweryId). 5 Prüfen: je Fall eine Anfrage mit erwarteter Antwort: POST /beers mit "breweryId": 42 → 404. Weicht eine

Jede Regel hat normale Fälle, Grenzen und Fehler

Art	Bedeutung	An der Regel für Brauereien
Normal	der übliche Weg	Brauerei 3 existiert → 201
Grenze	ein Wert am Rand, ein eben angelegter oder ein leerer Eintrag	eben angelegte Brauerei → 201; breweryId 0 → 404
Fehler	was abgelehnt wird, mit Statuscode	Brauerei 42 → 404

Das Schema prüft die Form, der Service prüft gegen gespeicherte Daten



Stünde eine Regel im Controller, bräuchte jeder Endpunkt mit derselben Regel eine eigene Kopie, zum Beispiel POST und PATCH.

Weg einer Anfrage POST /beers von links nach rechts: Client, ZodValidationPipe prüft die Form am Schema des DTO, Controller übersetzt HTTP in Werte und prüft keine Regel, Service prüft gegen gespeicherte Daten der Brauereien und Biere und berechnet Felder. Unter der Pipe die Frage „Lässt sich die Regel an der Anfrage allein entscheiden?“, ja: ins Schema, mit dem Beispiel „abv liegt zwischen 0 und 20“ → 400. Unter dem Service die Frage „Braucht die Regel gespeicherte Daten?“, ja: in den Service, mit dem Beispiel „die Brauerei muss existieren“ → 404. Stünde eine Regel im Controller, bräuchte jeder

Die Signatur legt fest, was eine Methode braucht und liefert

- Der Name sagt, was geprüft wird: assertBreweryExists.
- Parameter: nur, was die Prüfung braucht, breweryId: number.
- Rückgabe void: Die Methode kehrt zurück oder wirft eine Exception.
- Exception: NotFoundException mit 404, wenn die Brauerei fehlt.

MERKSATZ

Die Signatur ist eine Entscheidung vor dem Code und passt in eine Zeile.

Ein berechnetes Feld entsteht bei jeder Antwort neu

- Gespeichert wird, was der Client schickt oder der Server vergibt.
- Ein berechnetes Feld leitet der Service aus gespeicherten Daten ab.
- Ein gespeicherter Wert müsste bei jeder Änderung der Grundlage nachgeführt werden.
- Vergisst eine Stelle das, widerspricht der Wert seinen Daten.

MERKSATZ

Im Datenmodell steht beim Durchschnitt der Bewertungen: wird berechnet, nicht gespeichert.

Schritt 1: Jedes Wort der Regel zählt, auch „genau“ und „muss“

- Regel: Ein Bier gehört zu genau einer Brauerei; die Brauerei muss existieren.
- Wann wird geprüft: beim Anlegen, beim Ändern, beim Löschen?
- Was passiert, wenn breweryId fehlt oder 0 ist?
- „Genau einer“ erfüllt schon das Datenmodell: Ein Bier hat ein Feld breweryId.

Schritt 2: Die Fälle stehen als Kommentar über der neuen Methode

TYPESCRIPT · SRC/BEERS/BEERS.SERVICE.TS

```
// R2: Ein Bier gehört zu genau einer Brauerei;
// die Brauerei muss existieren.
// Normal: Brauerei 3 existiert → 201
// Grenze: Brauerei eben mit POST /breweries angelegt → 201
// Grenze: breweryId 0 → 404, es gibt keine Brauerei 0
// Fehler: Brauerei 42 → 404
// Fehler: breweryId fehlt bei POST → 400 (Schema)
// Normal: breweryId fehlt bei PATCH → keine Prüfung
```

Schritt 3: Signatur und Rumpf der Prüfung

TYPESCRIPT · SRC/BEERS/BEERS.SERVICE.TS

```
private assertBreweryExists(breweryId: number): void {
  // findOne wirft 404, wenn die Brauerei fehlt. 404 statt 400, weil
  // die Anfrage gültig ist und nur die Brauerei nicht existiert
  this.breweriesService.findOne(breweryId);
}
```

Schritt 4: update prüft nur, wenn breweryId im Body steht

TYPESCRIPT · SRC/BEERS/BEERS.SERVICE.TS

```
update(id: number, dto: UpdateBeerDto): Beer {
  const beer = this.findOne(id);
  // R2 auch beim Ändern, bevor ein Feld überschrieben wird
  if (dto.breweryId !== undefined) {
    this.assertBreweryExists(dto.breweryId);
  }
  // Nur die Felder aus dem Body überschreiben
  Object.assign(beer, dto);
  return beer;
}
```

Schritt 5: Jeder Fall wird eine Anfrage mit erwarteter Antwort

Fall	Anfrage	Antwort
Normal: Brauerei 3	POST /beers mit "breweryId": 3	201
Grenze: eben angelegt	POST /breweries, dann POST /beers mit "breweryId": 5	201
Grenze: breweryId 0	PATCH /beers/3 mit "breweryId": 0	404, Brewery 0 not found
Fehler: Brauerei 42	POST /beers mit "breweryId": 42	404, Brewery 42 not found
Fehler: ohne breweryId	POST /beers ohne das Feld	400, breweryId: Invalid input
Normal: PATCH ohne breweryId	PATCH /beers/2 mit "abv": 5.1	200, keine Prüfung

Einheit 10 · Anforderungen in Code übersetzen

FRAGE

Live-Frage: Was antwortet GET /beers/2, nachdem die Brauerei 2 gelöscht ist?

DELETE http://localhost:3000/breweries/2, danach GET http://localhost:3000/beers/2

Auflösung: 200, weil die Liste den Fall „Brauerei wird gelöscht“ nicht enthielt

BEISPIEL

- DELETE antwortet mit 204; die Brauerei ist weg.
- Das Bier zeigt weiter auf die Brauerei 2.
- Die Regel wird nur beim Anlegen und Ändern eines Biers geprüft.
- Ab Einheit 12 verhindert die Datenbank solche Verweise.

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Content-Length: 81

{"id":2,"name":"Wildbräu Grafing Helles",
 "style":"helles","abv":5,"breweryId":2}
```

Einheit 10 · Anforderungen in Code übersetzen

FRAGE

Live-Frage: Ein Bier verletzt zwei Regeln: Welchen Statuscode gibt die API zurück?

POST http://localhost:3000/beers mit "abv": 25 und "breweryId": 42

Auflösung: 400, weil das Schema prüft, bevor der Service läuft

BEISPIEL

- Die Pipe prüft den Body vor dem Controller.
- abv 25 verletzt eine Form; die Pipe lehnt ab.
- Der Service läuft nie; die Brauerei wird nicht geprüft.
- Mit abv 5.6 folgt 404: Brewery 42 not found.

```
HTTP/1.1 400 Bad Request
Content-Type: application/json; charset=utf-8

{"message":
  ["abv: Too big: expected number to be <=20"],
 "error":"Bad Request","statusCode":400}
```

Einheit 10 · Anforderungen in Code übersetzen

FRAGE

Live-Frage: Was antwortet PATCH /beers/2 mit einem leeren Body?

PATCH http://localhost:3000/beers/2 mit {} und Content-Type application/json

Auflösung: 200 mit dem Bier, weil im Schema für PATCH jedes Feld fehlen darf

BEISPIEL

- Das Schema für PATCH macht alle Felder optional.
- Ein leeres Objekt ist damit gültig.
- update überschreibt kein Feld und liefert das Bier.
- Ob {} erlaubt sein soll, entscheidet der Auftraggeber.

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Content-Length: 81

{"id":2,"name":"Wildbräu Grafing Helles",
 "style":"helles","abv":5,"breweryId":2}
```

Fehlersuche: PATCH mit "breweryId": 0 antwortet mit 200

BEISPIEL

- Erwartet: 404, eine Brauerei 0 gibt es nicht.
- Der Fall steht im Kommentar über der Methode.
- Der Server startet ohne Fehlermeldung.
- Welche Stelle lässt die 0 durch?

```
PATCH http://localhost:3000/beers/3
{"breweryId":0}

HTTP/1.1 200 OK
{"id":3,"name":"Augustiner Lagerbier Hell",
 "style":"helles","abv":5.2,"breweryId":0}
```

Fehlersuche: 0 ist in JavaScript falsy, die Bedingung überspringt die Prüfung

- Spur 1: POST /beers mit "breweryId": 0 antwortet mit 404; Schema und Prüfung arbeiten.
- Spur 2: PATCH mit "breweryId": 42 antwortet mit 404; nur die 0 fehlt.
- Spur 3: Compiler und ESLint melden nichts.
- Ursache: if (dto.breweryId) ist für undefined und für 0 falsch.
- Beheben: dto.breweryId !== undefined; gefunden hat den Fehler die Anfrage zum Fall.

Übung

Die Aufgaben stehen vollständig im Skript

Aufgaben dieser Einheit

ÜBUNG

1. Aufgabe 10.1: Ein Bier bewerten
2. Aufgabe 10.2: Eine zweite Bewertung ersetzt die erste
3. Aufgabe 10.3: Den Durchschnitt anzeigen
4. Zusatz 10.4: Fälle, die die Story nicht nennt

SELBSTKONTROLLE

- Beispiel zum Übernehmen: Skript 10.7 bis 10.10
- Aufgaben ab Skript 10.11, Hinweise ab 10.21
- Lösungsstand nach der Übung

Review: zwei Lösungen für die Bewertungen

- Welche Fälle stehen im Kommentar, welche fehlen?
- Wo steht die Regel zu den Sternen, wo die zur zweiten Bewertung?
- Wer prüft, dass es das Bier gibt, bevor die Bewertung entsteht?
- Hätten eure Kriterien aus dem Einstieg die Grenzfälle enthalten?

MERKSATZ

Einheit 11 vergleicht eure Kriterien zum Filtern und Sortieren mit denen des Auftraggebers.

Das solltest du jetzt können

- ✓ Ein Akzeptanzkriterium als Anfrage mit erwarteter Antwort formulieren
- ✓ Zu einer Regel normale Fälle, Grenzen und Fehler aufzählen, bevor du Code schreibst
- ✓ Entscheiden, ob eine Regel ins Schema oder in den Service gehört
- ✓ Eine Signatur festlegen, bevor du implementierst
- ✓ Ein berechnetes Feld in jeder Antwort ausgeben, ohne es zu speichern

WEB-PROGRAMMIERUNG · EINHEIT 11

REST-Design und API-Dokumentation

Einheit 11 · REST-Design und API-Dokumentation

FRAGE

GitHub unterstützt jede Version seiner API 24 Monate weiter: Was hat GitHub davon?

Die Brauereien binden die Beer API an ihre eigenen Webseiten an. Sie brauchen eine Beschreibung, auf die sie sich verlassen können.

Aufgabe: Brauereien filtern und sortieren Biere und binden die API über eine Dokumentation an

- Als Brauerei möchte ich Biere filtern, sortieren und die API dokumentiert anbinden.
- Damit zeige ich die Daten auf meiner eigenen Webseite.
- GET /beers?style=helles&breweryId=3 liefert nur das Augustiner Lagerbier Hell.
- Sortiert nach Durchschnitt stehen Biere ohne Bewertung am Ende.
- Swagger UI zeigt alle Endpunkte mit Schemas; Postman importiert die Beschreibung.

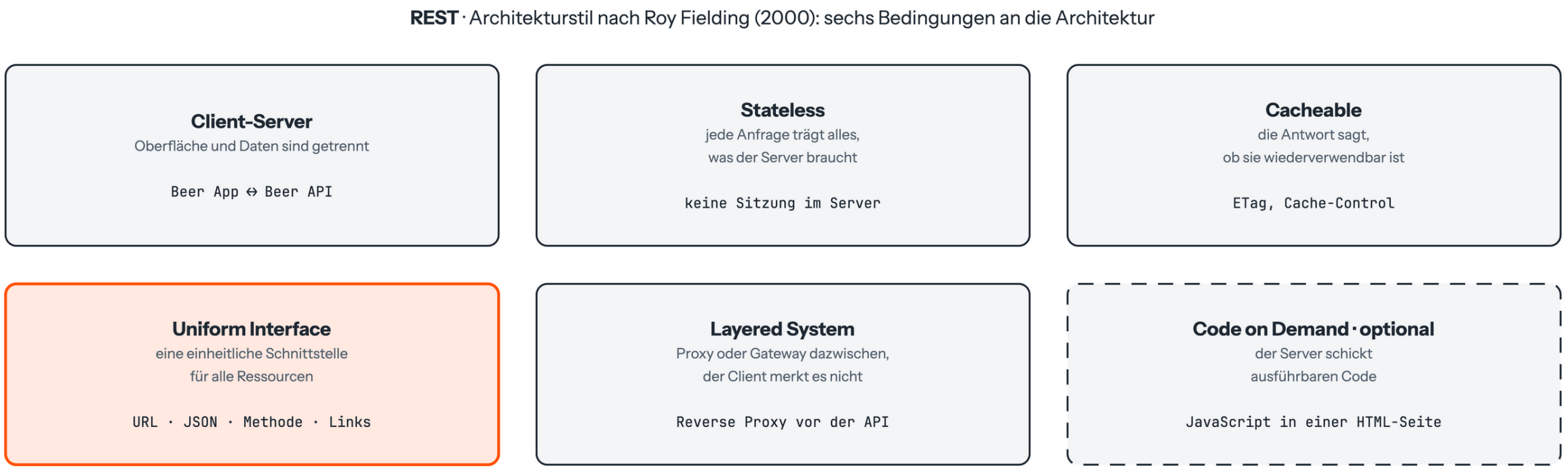
MERKSATZ

Query-Parameter kommen als Strings an; ein Schema prüft sie wie einen Body.

Eine Version ist eine Zusage: Breaking Changes kommen nur in einer neuen Version

Änderung an der Beer API	Breaking Change	Folge für die Webseite einer Brauerei
averageRating heißt künftig rating	ja	Der Durchschnitt fehlt auf der Webseite.
abv kommt als String "5.2" statt als Zahl	ja	Rechnen und Vergleichen liefern andere Ergebnisse.
neuer Pflichtparameter ?country	ja	Bisherige Anfragen antworten mit 400.
neues Feld description in der Antwort	nein	Die Webseite ignoriert das Feld.
neuer optionaler Filter ?style	nein	Bisherige Anfragen bleiben gültig.

REST ist ein Architekturstil aus sechs Constraints, einer davon optional



Sechs Karten in zwei Reihen unter der Überschrift „REST · Architekturstil nach Roy Fielding (2000): sechs Bedingungen an die Architektur“. Client-Server: Oberfläche und Daten sind getrennt, Beer App und Beer API. Stateless: jede Anfrage trägt alles, was der Server braucht, keine Sitzung im Server. Cacheable: die Antwort sagt, ob sie wiederverwendbar ist, ETag und Cache-Control. Uniform Interface, hervorgehoben: eine einheitliche Schnittstelle für alle Ressourcen, URL, JSON, Methode, Links. Layered System: Proxy oder Gateway dazwischen, der Client merkt es nicht, Reverse Proxy vor der API. Code

Das Uniform Interface hat vier Teile; der vierte fehlt der Beer API

Teil	Bedeutung	In der Beer API
Identifikation	Jede Ressource hat eine URL.	/beers/2
Repräsentation	Der Client ändert die Ressource über eine Darstellung.	JSON im Body von PATCH /beers/2
Selbstbeschreibende Nachricht	Methode, Statuscode und Header reichen zum Verstehen.	Content-Type: application/json, 404
Hypermedia (HATEOAS)	Links in der Antwort führen zum nächsten Schritt.	fehlt

Stateless: Der Server merkt sich zwischen zwei Anfragen nichts über den Client

- Jede Anfrage trägt Methode, Pfad, Query-Parameter und Body.
- Der Server führt keine Sitzung je Client.
- Die gespeicherten Biere sind Zustand der Ressourcen.
- Ab Einheit 13 trägt jede geschützte Anfrage ein Token mit der Identität.

MERKSATZ

Hinter einem Load Balancer beantwortet jeder Server jede Anfrage, wenn er dieselben Daten hat.

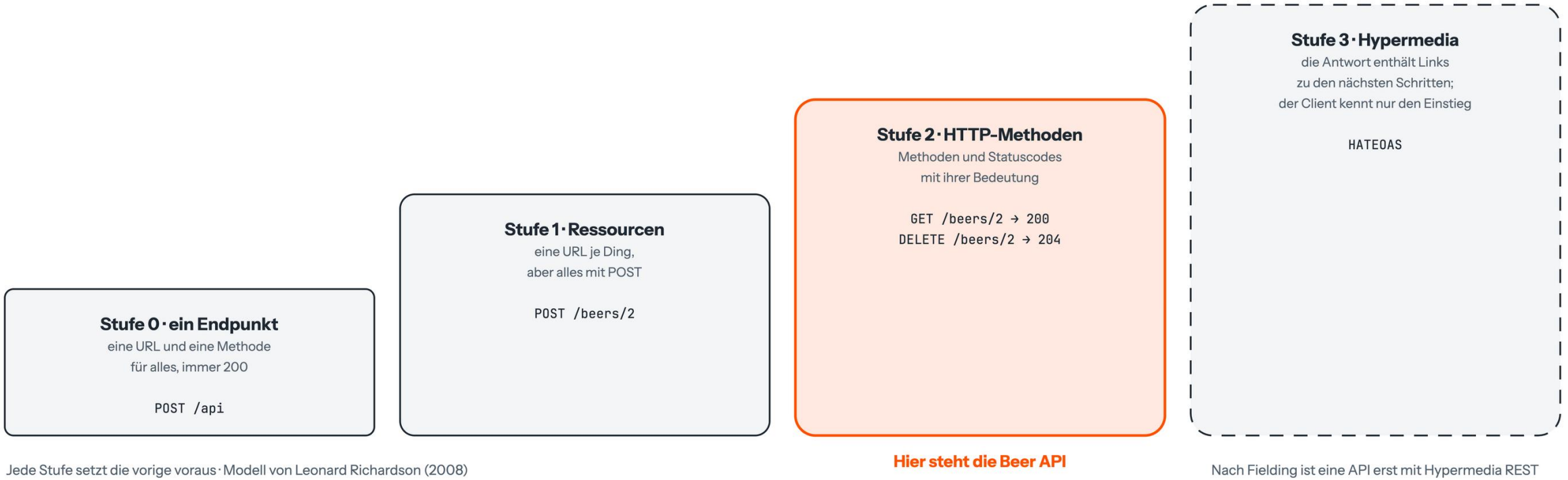
URLs benennen Ressourcen mit Substantiven im Plural; Query-Parameter wählen aus

Anfrage	Bedeutung	Regel
GET /beers	Sammlung der Biere	Substantiv im Plural
GET /beers/2	ein Element der Sammlung	Die id benennt die Ressource und steht im Pfad.
POST /beers/1/ratings	Bewertung, die zu Bier 1 gehört	Hierarchie nur für Zugehörigkeit
GET /beers?style=helles&breweryId=3	dieselbe Sammlung, gefiltert	Mehrere Filter verkleinern die Liste.
GET /beers?sort=averageRating&order=desc	dieselbe Sammlung, sortiert	Sortierung im Query-String
GET /getBeers, POST /beers/1/rate	Verb im Pfad	vermeiden: Die Methode ist das Verb.

Die Methode sagt, was mit der Ressource geschieht

Methode	Bedeutung	sicher · idempotent	In der Beer API
GET	liest eine Ressource	ja · ja	GET /beers, GET /beers/2
POST	legt eine Ressource in einer Sammlung an	nein · nein	POST /beers, POST /beers/1/ratings
PATCH	ändert einzelne Felder	nein · nein	PATCH /beers/2
DELETE	löscht eine Ressource	nein · ja	DELETE /beers/5

Das Richardson Maturity Model hat vier Stufen; die Beer API steht auf Stufe 2



Treppe aus vier Stufen von links nach rechts. Stufe 0, ein Endpunkt: eine URL und eine Methode für alles, immer 200, POST /api. Stufe 1, Ressourcen: eine URL je Ding, aber alles mit POST, POST /beers/2. Stufe 2, HTTP-Methoden, hervorgehoben mit „Hier steht die Beer API“: Methoden und Statuscodes mit ihrer Bedeutung, GET /beers/2 → 200, DELETE /beers/2 → 204. Stufe 3, Hypermedia, gestrichelt: die Antwort enthält Links zu den nächsten Schritten; der Client kennt nur den Einstieg, HATEOAS. Darunter: Jede Stufe setzt die vorige voraus, Modell von Leonard Richardson (2008); nach Fielding ist eine API

HATEOAS: Der Client folgt Links aus der Antwort, statt URLs selbst zu bauen

- Prinzip aus dem Web: Ein Mensch klickt Links, statt Adressen zu tippen.
- Der Client kennt nur den Einstieg; Antworten nennen die nächsten Schritte.
- Der Server kann URLs ändern, ohne Clients zu brechen.
- In der Praxis selten: Clients werden für bekannte Abläufe gebaut.
- Reine URL-Felder in JSON nennen weder Methode noch Body.

MERKSATZ

Die Praxis sind JSON-APIs auf Stufe 2. Welche URLs es gibt, beschreibt OpenAPI.

OpenAPI beschreibt eine HTTP-API maschinenlesbar

- Ein JSON- oder YAML-Dokument nennt alle Pfade und Methoden.
- Parameter im Pfad und im Query-String stehen mit Typ und erlaubten Werten.
- Body und Antwort haben je Statuscode ein Schema.
- Swagger UI zeigt das Dokument an; Postman importiert es als Collection.

MERKSATZ

Die Beer API liefert ihre Beschreibung unter /api-json, Swagger UI unter /api.

Die Beschreibung entsteht beim Start aus Decorators und DTO-Klassen

Im Code	In der Beschreibung	An den Brauereien
DTO am Parameter mit @Body	Schema des Body	CreateBreweryDto
DTO am Parameter mit @Query	Query-Parameter mit erlaubten Werten	city, sort, order
@ZodResponse	Antwort mit Statuscode und Schema	200 mit BreweryDto
@ApiNotFoundResponse	Fehlerantwort mit Schema	404 mit ErrorDto
@ApiTags	Gruppe in Swagger UI	breweries
TypeScript-Typ allein	nichts	Brewery aus dem Service

Schritt 1: Ein Schema legt fest, welche Query-Parameter erlaubt sind

TYPESCRIPT · SRC/BREWERIES/DTO/BREWERY-QUERY.DTO.TS

```
export const BreweryQuerySchema = z.object({
  city: z.string().optional(),
  sort: z.enum(['name', 'city']).optional(),
  order: z.enum(['asc', 'desc']).default('asc'),
});

export class BreweryQueryDto extends createZodDto(BreweryQuerySchema) {}
```

Schritt 2: Der Service filtert und sortiert eine Kopie der Liste

TYPESCRIPT · SRC/BREWERIES/BREWERIES.SERVICE.TS

```
findAll(query: BreweryQueryDto): Brewery[] {
  // filter liefert eine neue Liste; sort ändert nur sie
  const breweries = this.breweries.filter(
    (brewery) => !query.city || brewery.city === query.city,
  );
  if (query.sort) {
    const field = query.sort;
    const direction = query.order === 'desc' ? -1 : 1;
    const byField = (a: Brewery, b: Brewery) =>
      a[field].localeCompare(b[field]) * direction;
    breweries.sort(byField);
  }
  return breweries;
}
```

Einheit 11 · REST-Design und API-Dokumentation

FRAGE

Live-Frage: Keine Brauerei steht in Ulm. Was antwortet die API?

GET http://localhost:3000/breweries?city=Ulm

Auflösung: 200 mit einer leeren Liste, weil es die Ressource /breweries gibt

BEISPIEL

- Der Filter wählt aus der Liste aus; er benennt keine Ressource.
- Die Liste ist leer, die Sammlung existiert.
- 404 hieße: Die Ressource gibt es nicht.
- Der Client verarbeitet immer eine Liste, auch eine leere.

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Content-Length: 2

[]
```

Einheit 11 · REST-Design und API-Dokumentation

FRAGE

Live-Frage: Was antwortet die API, wenn city einen Tippfehler hat?

GET http://localhost:3000/breweries?citty=München

Auflösung: 200 mit allen vier Brauereien, weil z.object unbekannte Schlüssel entfernt

BEISPIEL

- Das Schema entfernt citty ohne Fehler.
- city fehlt; der Filter läuft nicht.
- Die Antwort sieht richtig aus, der Tippfehler bleibt unbemerkt.
- Mit .strict() am Schema antwortet der Server mit 400.

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Content-Length: 340

[{"id":1,"name":"Privatbrauerei Schlumberger", …},
 {"id":2, …}, {"id":3, …}, {"id":4, …}]
```

Schritt 3: Ein Schema beschreibt die Antwort mit id

TYPESCRIPT · SRC/BREWERIES/DTO/BREWERY.DTO.TS

```
// Antwort: die Felder aus dem Body und die id, die der Service vergibt
export const BrewerySchema = z.object({
  id: z.number().int(),
  ...CreateBrewerySchema.shape,
});

export class BreweryDto extends createZodDto(BrewerySchema) {}
```

Schritt 4: Decorators beschreiben Antwort und Fehler je Endpunkt

TYPESCRIPT · SRC/BREWERIES/BREWERIES.CONTROLLER.TS

```
@Get()
@ZodResponse({ status: 200, type: [BreweryDto] })
@ApiBadRequestResponse({ type: ErrorDto })
findAll(@Query() query: BreweryQueryDto) {
  return this.breweriesService.findAll(query);
}

@Get(':id')
@ZodResponse({ status: 200, type: BreweryDto })
@ApiBadRequestResponse({ type: ErrorDto })
@ApiNotFoundResponse({ type: ErrorDto })
findOne(@Param('id', ParseIntPipe) id: number) {
  // Keine ganze Zahl: ParseIntPipe antwortet mit 400
  return this.breweriesService.findOne(id);
}
```

Schritt 5: Mit der Adresse des Servers läuft der Import in Postman

TYPESCRIPT · SRC/MAIN.TS

```
const config = new DocumentBuilder()
  .setTitle('Beer API')
  // Adresse des Servers: Der Import in Postman setzt daraus baseUrl
  .addServer('http://localhost:3000')
  .build();
```

Fehlersuche: Swagger UI zeigt für GET /breweries/{id} keine Antwort 200

BEISPIEL

- Erwartet: 200 mit BreweryDto, 400 und 404.
- Swagger UI zeigt nur 400 und 404.
- Die Anfrage selbst antwortet wie immer.
- Der Server startet ohne Fehlermeldung.

```
Swagger UI, GET /breweries/{id}, Responses:
  400   ErrorDto
  404   ErrorDto

GET http://localhost:3000/breweries/2
HTTP/1.1 200 OK
{"id":2,"name":"Wildbräu Grafing", …}
```

Fehlersuche: Ohne Decorator für die Antwort kennt die Beschreibung keine 200

- Spur 1: Neu laden ändert nichts; Swagger UI liest /api-json bei jedem Laden.
- Spur 2: /api-json nennt für GET /breweries/{id} nur 400 und 404.
- Spur 3: GET /breweries hat 200 mit Schema; Compiler und ESLint melden nichts.
- Ursache: An findOne fehlt @ZodResponse; der Typ aus dem Service existiert zur Laufzeit nicht.
- Beheben: Decorator zurück; /api-json nennt wieder 200 mit BreweryDto_Output.

Übung

Die Aufgaben stehen vollständig im Skript

Aufgaben dieser Einheit

ÜBUNG

1. Aufgabe 11.1: Biere filtern
2. Aufgabe 11.2: Biere sortieren
3. Aufgabe 11.3: Die API dokumentieren
4. Zusatz 11.4: Zwei öffentliche APIs einordnen

SELBSTKONTROLLE

- Beispiel zum Übernehmen: Skript 11.8 bis 11.14
- Aufgaben ab Skript 11.15, Hinweise ab 11.25
- Lösungsstand nach der Übung

Review: zwei Lösungen für Filter, Sortierung und Dokumentation

- Wie wird aus "2" im Query-String die Zahl 2?
- Wie stehen Biere ohne Bewertung am Ende, auch bei aufsteigender Reihenfolge?
- Was müsste eine Brauerei aus Swagger UI noch wissen?
- Auf welcher Stufe stehen GitHub und die Open Brewery DB?

MERKSATZ

Die Beschreibung unter /api-json ist jetzt der Vertrag mit den Brauereien.

Das solltest du jetzt können

- ✓ Die REST-Constraints nennen und an der Beer API zeigen
- ✓ URLs für Ressourcen, Filter und Sortierung entwerfen
- ✓ Eine API nach dem Richardson Maturity Model einordnen
- ✓ Endpunkte so beschreiben, dass Swagger UI Anfrage, Antwort und Fehler zeigt
- ✓ Eine OpenAPI-Beschreibung in Postman importieren und die Anfragen ausführen

WEB-PROGRAMMIERUNG · EINHEIT 12

Datenbank-Anbindung

Einheit 12 · Datenbank-Anbindung

FRAGE

Jeder Neustart löscht alle neuen Daten der Beer API: Welcher Verlust wäre am teuersten?

Die regionalen Brauereien betreiben die App. Gäste bewerten Biere, der Betreiber pflegt Brauereien und Biere.

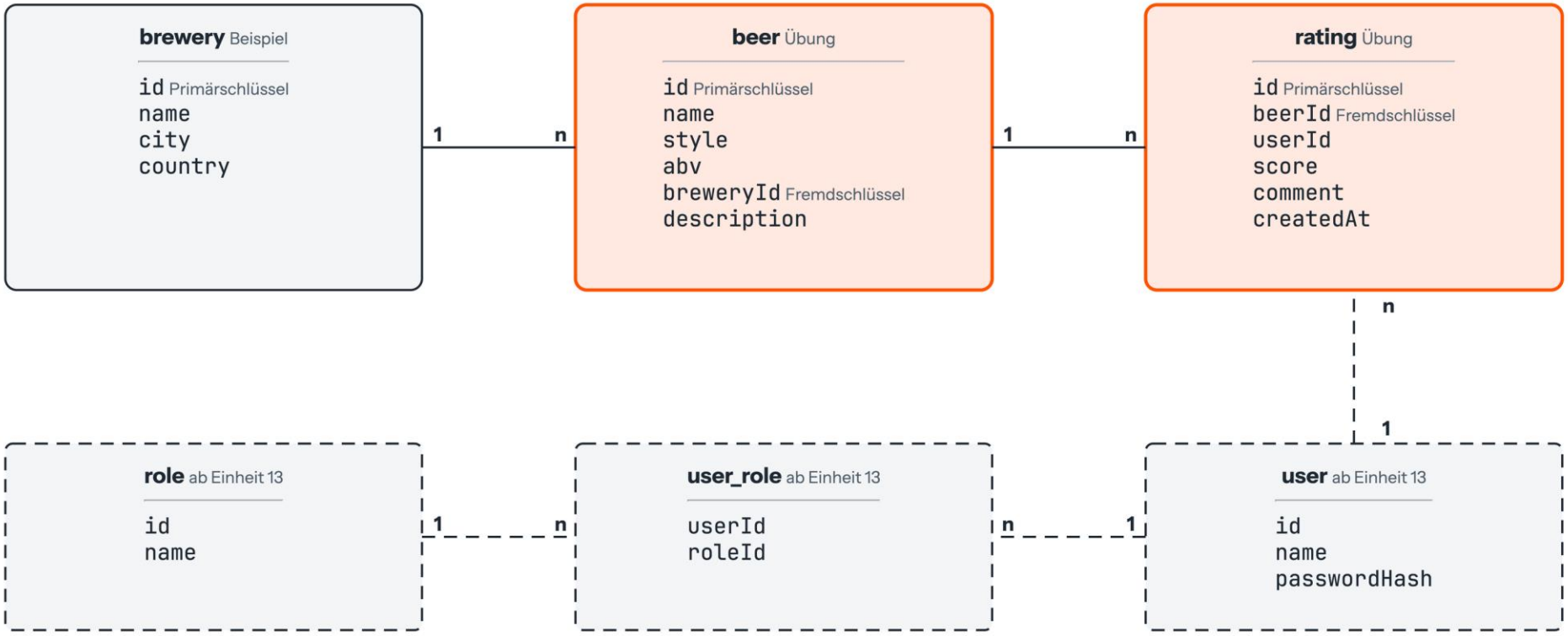
Aufgabe: Biere, Brauereien und Bewertungen überstehen einen Neustart des Servers

- Als Betreiber möchte ich, dass Biere, Brauereien und Bewertungen einen Neustart überstehen.
- Damit gehen keine Daten verloren.
- Nach einem Neustart sind neue Daten erhalten und gelöschte Biere bleiben gelöscht.
- Alles, was die Beer API bis Einheit 11 konnte, funktioniert unverändert.

MERKSATZ

Eine Entity beschreibt eine Tabelle. Erst eine Migration legt die Tabelle in der Datenbank an.

Das ER-Modell der Beer App hat sechs Tabellen; drei davon entstehen heute



Ein Bier gehört zu genau einer Brauerei, eine Bewertung zu genau einem Bier. Durchgezogen: ab heute in beer.db · gestrichelt: ab Einheit 13

ER-Modell mit sechs Tabellen in zwei Reihen. Oben, durchgezogen: brewery (Beispiel) mit id als Primärschlüssel, name, city, country; beer (Übung, hervorgehoben) mit id als Primärschlüssel, name, style, abv, breweryId als Fremdschlüssel, description; rating (Übung, hervorgehoben) mit id als Primärschlüssel, beerId als Fremdschlüssel, userId, score, comment, createdAt. brewery zu beer 1 zu n, beer zu rating 1 zu n. Unten, gestrichelt, ab Einheit 13: role mit id und name, user_role mit userId und roleId, user mit id, name und passwordHash; role zu user_role 1 zu n, user zu user_role 1 zu n, user zu

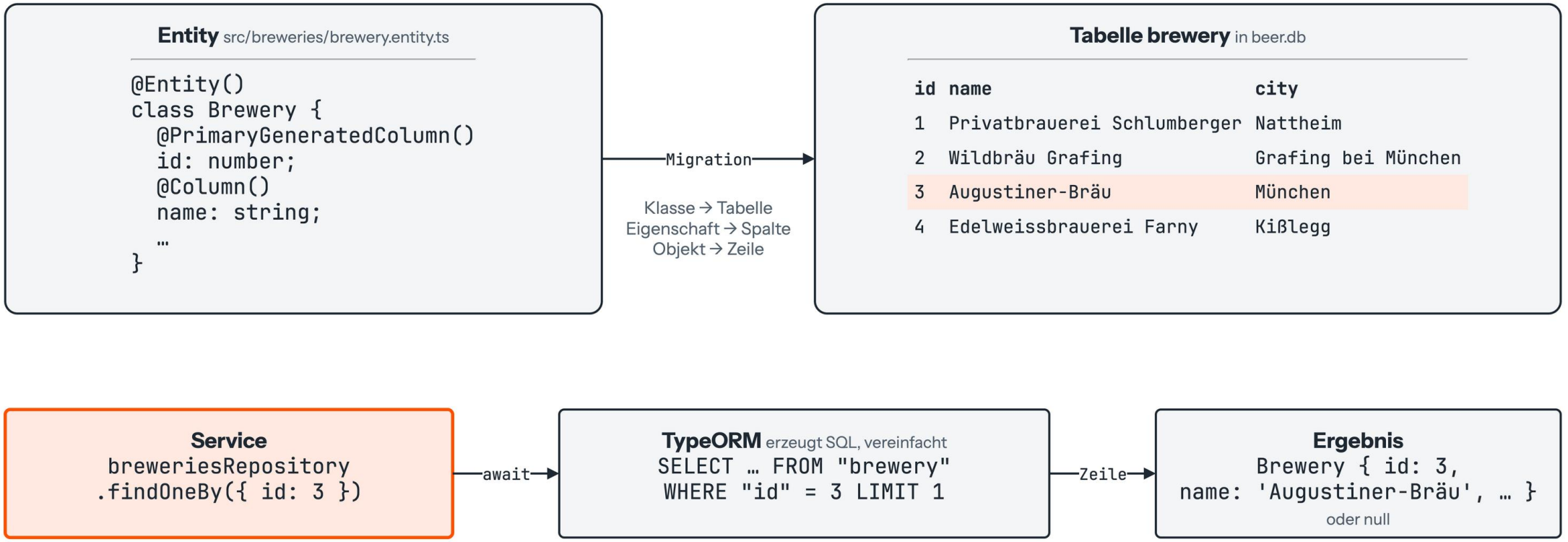
Bewertungen sind der teuerste Verlust, weil nur die App sie kennt

Tabelle	Wer schreibt sie	Nach einem Verlust
brewery	der Betreiber, wenige Zeilen	aus den Unterlagen der Brauereien nachpflegen
beer	der Betreiber, wenige Zeilen	aus den Unterlagen der Brauereien nachpflegen
rating	viele Gäste, jede Zeile einmal	verloren; der Kern der App fehlt
user (ab Einheit 13)	jeder Gast bei der Registrierung	jeder Gast muss sich neu registrieren

SQLite speichert die ganze Datenbank in einer Datei

- Eine relationale Datenbank speichert Tabellen mit Zeilen und Spalten.
- Der Primärschlüssel id macht jede Zeile eindeutig.
- Ein Fremdschlüssel wie breweryId zeigt auf eine Zeile einer anderen Tabelle.
- SQLite läuft im Prozess des Servers; die Datenbank ist die Datei beer.db.
- beer.db steht nicht im Git-Repository; jede Kopie erzeugt ihre eigene.

Ein ORM übersetzt die Klasse Brewery in die Tabelle brewery



Der Code bleibt TypeScript: Das ORM übersetzt Klassen in Tabellen und Methoden des Repository in SQL.

Oben links die Entity in src/breweries/brewery.entity.ts: @Entity() über class Brewery, darin @PrimaryGeneratedColumn() über id: number und @Column() über name: string. Ein Pfeil „Migration legt an“ führt zur Tabelle brewery in beer.db mit den Spalten id, name, city und den vier Brauereien; die Zeile 3, Augustiner-Bräu, München, ist hervorgehoben. Zwischen beiden: Klasse → Tabelle, Eigenschaft → Spalte, Objekt → Zeile. Unten drei Kästen: Service ruft breweriesRepository.findOneBy({ id: 3 }) auf, Pfeil „await“ zu TypeORM, das SQL erzeugt, vereinfacht SELECT … FROM "brewery" WHERE "id" =

Das Repository liest und schreibt eine Tabelle; jede Methode liefert ein Promise

Methode	Bedeutung	Ergebnis
find({ where, order })	Zeilen lesen, gefiltert und sortiert	Liste, auch leer
findOneBy({ id })	eine Zeile lesen	Objekt oder null
save(objekt ohne id)	Zeile einfügen; die Datenbank vergibt die id	gespeichertes Objekt mit id
save(objekt mit id)	vorhandene Zeile ändern	gespeichertes Objekt
remove(objekt)	Zeile löschen	gelöschtes Objekt

Eine Migration bringt jede Kopie der Datenbank auf denselben Stand



migration:run legt beer.db an, wenn die Datei fehlt, und führt nur offene Migrationen aus.

Ablauf in fünf Kästen. Entities (Brewery, Beer, Rating: wie die Tabellen sein sollen) führen zu migration:generate, hervorgehoben: vergleicht die Entities mit den Tabellen in beer.db. Pfeil „erzeugt“ zur Migration im Git-Repository mit up: CREATE TABLE und down: DROP TABLE. Pfeil zu migration:run, hervorgehoben: führt jede Migration aus, die noch nicht gelaufen ist. Pfeil „ändert“ zu beer.db, nicht im Repository: Tabellen brewery, beer, rating und Tabelle migrations, die festhält, was gelaufen ist. Links: Jede Kopie des Projekts ergibt mit npm install und npm run migration:run dieselben Tabellen.

Vier Befehle verwalten das Datenbankschema der Beer API

Befehl	Wirkung	Wann
npm run migration:generate -- src/migrations/Name	Unterschied zwischen Entities und beer.db als Migration	nach einer Änderung an einer Entity
npm run migration:run	alle Migrationen ausführen, die noch nicht gelaufen sind	nach generate und nach jedem git pull
npm run typeorm -- migration:show	gelaufene Migrationen mit [X], offene mit [ ]	wenn eine Tabelle zu fehlen scheint
npm run db:reset	alle Tabellen löschen und alle Migrationen ausführen	vor jedem Lauf der Collection

Ein Fremdschlüssel lässt die Datenbank prüfen, dass die Brauerei eines Biers existiert

- @ManyToOne(() => Brewery) an Beer erzeugt in der Migration einen Fremdschlüssel.
- TypeORM schaltet die Prüfung in SQLite ein; dann lehnt SQLite Biere ohne existierende Brauerei ab.
- Das schließt die Lücke aus Einheit 10: Eine gelöschte Brauerei ließ ihre Biere zurück.
- Der Server meldet die Ablehnung als 500, bis der Service sie übersetzt.

MERKSATZ

Die Regel „Ein Bier gehört zu einer existierenden Brauerei“ steht dann im Service und in der Datenbank.

Schritt 1: Drei Pakete verbinden NestJS mit SQLite

BASH

```
npm install @nestjs/typeorm@12.0.2 typeorm@1.1.2
npm install better-sqlite3@12.11.1
```

Schritt 2: Eine Beschreibung der Verbindung gilt für Server und CLI

TYPESCRIPT · SRC/DATA-SOURCE.TS

```typescript
// Eine Beschreibung der Verbindung für den Server (AppModule) und die CLI
export const dataSourceOptions: DataSourceOptions = {
  type: 'better-sqlite3',
  database: 'beer.db',
  entities: [Brewery],
  // .ts für die CLI im Ordner src, .js für den Server im Ordner dist
  migrations: [__dirname + '/migrations/*.{ts,js}'],
};

// Die CLI liest die Verbindung aus dem Default-Export
export default new DataSource(dataSourceOptions);
```

Schritt 3: Die Entity Brewery beschreibt die Tabelle brewery

TYPESCRIPT · SRC/BREWERIES/BREWERY.ENTITY.TS

```
// Die Klasse beschreibt die Tabelle brewery, ein Objekt eine Zeile
@Entity()
export class Brewery {
  // Die Datenbank vergibt die id beim Einfügen
  @PrimaryGeneratedColumn()
  id: number;

  @Column()
  name: string;

  @Column()
  city: string;

  @Column()
  country: string;
}
```

Schritt 4: Die CLI erzeugt die Migration und führt sie aus

```
$ npm run migration:generate -- src/migrations/CreateBreweries
Migration …/src/migrations/1792672856555-CreateBreweries.ts
  has been generated successfully.

$ npm run migration:run
Migration CreateBreweries1792672856555 has been executed
  successfully.
```

Schritt 4: Die Beispieldaten kommen als zweite Migration in die Datenbank

TYPESCRIPT · SRC/MIGRATIONS/1792672866842-SEEDBREWERIES.TS

```
public async up(queryRunner: QueryRunner): Promise<void> {
  await queryRunner.manager.insert(Brewery, [
    {
      id: 1,
      name: 'Privatbrauerei Schlumberger',
      city: 'Nattheim',
      country: 'Deutschland',
    },
```

Schritt 5: Der Service bekommt das Repository und filtert in der Datenbank

TYPESCRIPT · SRC/BREWERIES/BREWERIES.SERVICE.TS

```
// Daten in beer.db: Ein Neustart ändert nichts
constructor(
  @InjectRepository(Brewery)
  private readonly breweriesRepository: Repository<Brewery>,
) {}

findAll(query: BreweryQueryDto): Promise<Brewery[]> {
  // undefined im where wirft in TypeORM 1: nur gesetzte Filter übergeben
  return this.breweriesRepository.find({
    where: query.city ? { city: query.city } : {},
    order: query.sort ? { [query.sort]: query.order } : {},
  });
}
```

Schritt 5: save fügt ohne id ein und ändert mit id

TYPESCRIPT · SRC/BREWERIES/BREWERIES.SERVICE.TS

```
create(dto: CreateBreweryDto): Promise<Brewery> {
  // Ohne id fügt save eine Zeile ein; die Datenbank vergibt die id
  return this.breweriesRepository.save(dto);
}

async update(id: number, dto: UpdateBreweryDto): Promise<Brewery> {
  const brewery = await this.findOne(id);
  // Nur die Felder aus dem Body überschreiben
  Object.assign(brewery, dto);
  // Mit id ändert save die vorhandene Zeile
  return this.breweriesRepository.save(brewery);
}
```

Einheit 12 · Datenbank-Anbindung

FRAGE

Live-Frage: Brauerei 5 ist gelöscht. Welche id bekommt die nächste neue Brauerei?

DELETE http://localhost:3000/breweries/5, danach POST http://localhost:3000/breweries mit Brauerei Gold Ochsen

Auflösung: 201 mit id 6, weil SQLite eine gelöschte id nie wieder vergibt

BEISPIEL

- @PrimaryGeneratedColumn erzeugt AUTOINCREMENT in SQLite.
- SQLite merkt sich die höchste je vergebene id.
- Ein Link auf Brauerei 5 zeigt nie auf eine andere Brauerei.
- npm run db:reset setzt den Zähler zurück.

```
HTTP/1.1 201 Created
Content-Type: application/json; charset=utf-8

{"id":6,"name":"Brauerei Gold Ochsen",
 "city":"Ulm","country":"Deutschland"}
```

Einheit 12 · Datenbank-Anbindung

FRAGE

Live-Frage: Brauerei 5 und Bier 5 anlegen, Server neu starten. Was antworten beide?

GET http://localhost:3000/breweries/5 und GET http://localhost:3000/beers/5

Auflösung: Brauerei 200, Bier 404, weil Biere noch im Arbeitsspeicher liegen

BEISPIEL

- Die Brauerei steht in beer.db.
- Der neue Prozess liest dieselbe Datei.
- Das Bier lag im Array des BeersService.
- Biere und Bewertungen umstellen ist die Übung.

```
GET /breweries/5 → HTTP/1.1 200 OK
{"id":5,"name":"Brauerei Gold Ochsen", …}

GET /beers/5 → HTTP/1.1 404 Not Found
{"message":"Beer 5 not found",
 "error":"Not Found","statusCode":404}
```

Fehlersuche: GET /breweries antwortet mit 500, obwohl der Code unverändert ist

BEISPIEL

- Erwartet: 200 mit den vier Brauereien.
- Die Antwort nennt keinen Grund.
- Der Server läuft weiter.
- Die Lage entspricht einem frischen Clone ohne Migrationen.

```
GET http://localhost:3000/breweries

HTTP/1.1 500 Internal Server Error
{"statusCode":500,
 "message":"Internal server error"}
```

Fehlersuche: Die Tabelle fehlt, weil keine Migration gelaufen ist

BEISPIEL

- Spur 1: Das Terminal von start:dev nennt die fehlende Tabelle.
- Spur 2: migration:show zeigt beide Migrationen als offen.
- Ursache: Die Entity beschreibt die Tabelle; angelegt hat sie niemand.
- Beheben: npm run migration:run; GET /breweries antwortet wieder mit 200.

```
ERROR [ExceptionsHandler] QueryFailedError:
  SqliteError: no such table: brewery

$ npm run typeorm -- migration:show
[ ] CreateBreweries1792672856555
[ ] SeedBreweries1792672866842
```

Übung

Die Aufgaben stehen vollständig im Skript

Aufgaben dieser Einheit

ÜBUNG

1. Aufgabe 12.1: Biere in der Datenbank speichern
2. Aufgabe 12.2: Bewertungen in der Datenbank speichern
3. Zusatz 12.3: Eine Brauerei mit Bieren behalten

SELBSTKONTROLLE

- Beispiel zum Übernehmen: Skript 12.9 bis 12.21
- Aufgaben ab Skript 12.22, Hinweise ab 12.33
- Lösungsstand nach der Übung

Review: zwei Lösungen für Biere und Bewertungen in der Datenbank

- Wo sortiert ihr nach averageRating: in der Datenbank oder im Service?
- Was geschieht mit den Bewertungen eines gelöschten Biers?
- Welche Felder einer Antwort haben sich durch die Datenbank geändert?
- Warum passt im Zusatz 409 besser als 400 oder 500?

MERKSATZ

Die Bewertungen stehen jetzt in beer.db. Ein Backup ist eine Kopie dieser Datei.

Das solltest du jetzt können

- ✓ Eine Entity für eine Ressource schreiben
- ✓ Eine Migration erzeugen, lesen und ausführen
- ✓ Einen Service von einem Array auf ein Repository umstellen
- ✓ Mit @ManyToOne einen Fremdschlüssel anlegen und erklären, was er verhindert
- ✓ Die Datenbank mit einem Befehl auf die Beispieldaten zurücksetzen

PlanB. Trusted Agentic Operator

DHBW Duale Hochschule Baden-Württemberg Heidenheim

WEB-PROGRAMMIERUNG · EINHEIT 13

Authentifizierung

Einheit 13 · Authentifizierung

FRAGE

Zwei Bewertungen, zwei userIds, ein Gast: Login selbst bauen oder kaufen?

Heute übernimmt die Beer API jede userId aus dem Body. Ein Gast bewertet Bier 1 als User 1 mit 5 und als User 2 mit 1; der Durchschnitt sinkt auf 3. Die regionalen Brauereien brauchen Bewertungen, die echten Gästen gehören.

Aufgabe: Gäste registrieren sich, melden sich an und bewerten nur in eigenem Namen

- Als Gast möchte ich mich registrieren und anmelden, damit nur ich in meinem Namen bewerte.
- Eine Registrierung legt einen User mit der Rolle user an; ein vergebener Name ergibt 409.
- Bewerten geht nur mit gültigem Token; die userId stammt aus dem Token.
- Alle GET-Anfragen bleiben ohne Anmeldung möglich.

MERKSATZ

Login und Guard zeigt das Beispiel an den Brauereien. Die Übung ergänzt die Registrierung und schützt die Bewertungen mit dem Guard.

Ein Identity Provider übernimmt fünf Aufgaben; die Beer API baut heute zwei davon

Aufgabe	Bedeutung	Beer API	Identity Provider
Passwort als Hash speichern	Ein Leak gibt keine Passwörter preis	ja, mit bcrypt	ja
Token ausstellen und prüfen	Jede Anfrage nennt den Gast	ja, als JWT	ja
Passwort zurücksetzen	Ein Gast hat sein Passwort vergessen	nein	ja
Zweiter Faktor (MFA)	Ein Passwort allein genügt nicht	nein	ja
Sperre nach Fehlversuchen	Raten von Passwörtern wird gebremst	nein	ja

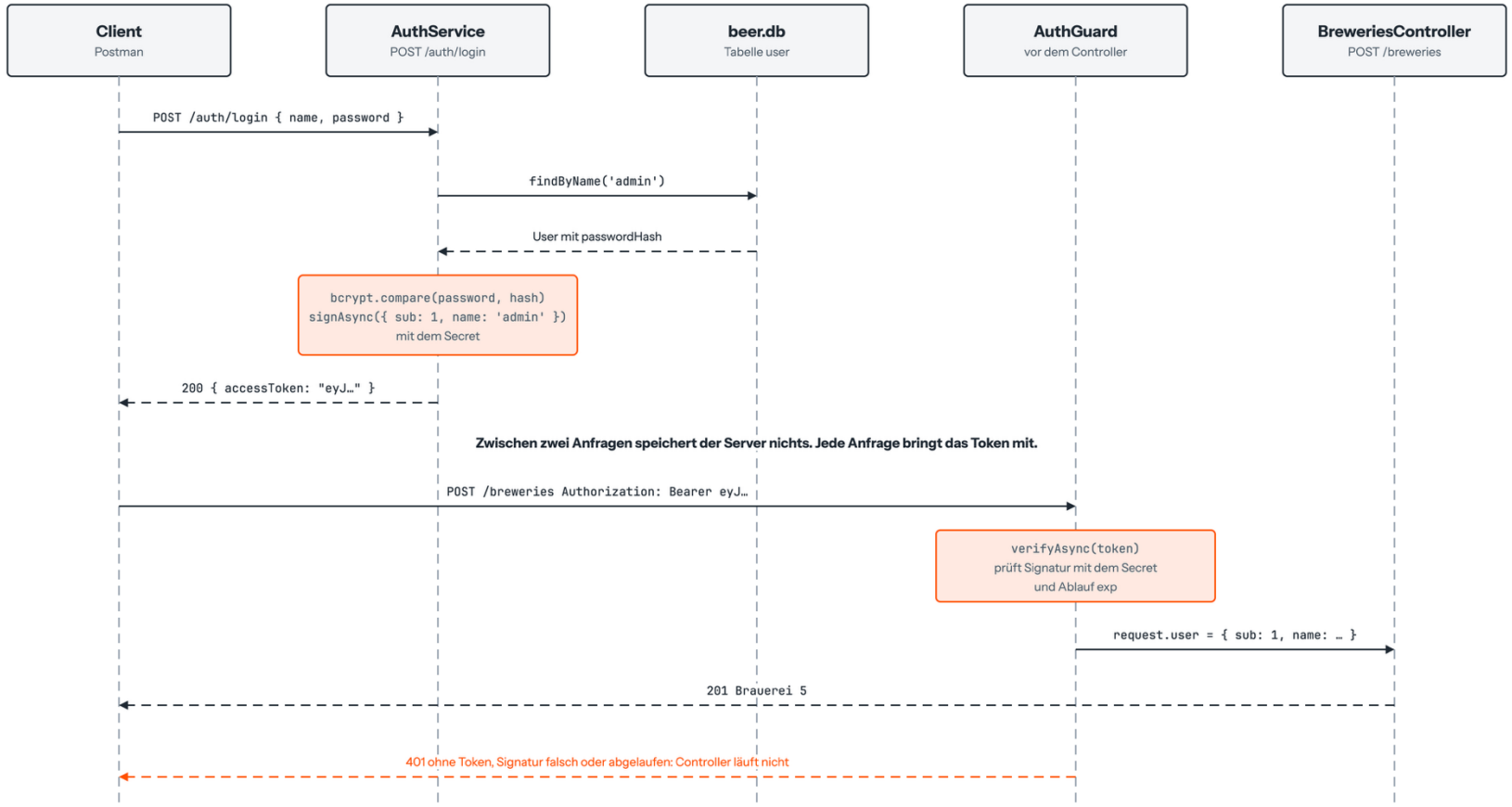
Authentifizierung stellt fest, wer anfragt; Autorisierung prüft, was diese Person darf

Frage	Statuscode	Bedeutung	Einheit
Wer stellt die Anfrage?	401 Unauthorized	nicht angemeldet, Token fehlt oder ist ungültig	13, heute
Darf diese Person das?	403 Forbidden	angemeldet, aber ohne das nötige Recht	14

Drei Wege der Anmeldung; die Beer API meldet Gäste mit Passwort und Token an

Weg	Wer meldet sich an	Was der Client sendet	Beispiel
API Key	ein Programm	ein festes Geheimnis im Header	ein Partner ruft Biere ab
Login mit Passwort und Token	eine Person in der eigenen App	das Token aus dem Login	die Beer API ab heute
OAuth 2.0 und OpenID Connect	eine Person bei einem Identity Provider	ein Token des Identity Providers	„Mit Google anmelden“

Nach dem Login trägt jede Anfrage ein signiertes Token, das der Guard prüft



Sequenz mit fünf Beteiligten: Client (Postman), AuthService (POST /auth/login), beer.db (Tabelle user), AuthGuard (vor dem Controller), BreweriesController (POST /breweries). Der Client sendet POST /auth/login mit name und password. Der AuthService ruft findByName('admin') und bekommt den User mit passwordHash. Hervorgehoben: bcrypt.compare(password, hash) und signAsync({ sub: 1, name: 'admin' }) mit dem Secret. Antwort 200 mit accessToken. Darunter: Zwischen zwei Anfragen speichert der Server nichts. Jede Anfrage bringt das Token mit. Der Client sendet POST

Die Datenbank speichert nur einen Hash des Passworts mit Salt und Kostenfaktor

admin123
↓ bcrypt mit Kostenfaktor 10 und zufälligem Salt
$2b$10$o6W5KznaQaytXGZ1itv87ODpeic6pzT91PoVsjnx5FyGIMimNJW/q

$2b	Algorithmus bcrypt
10	Kostenfaktor
o6W5KznaQaytXGZ1itv87O	Salt, 22 Zeichen
DpeicpzT91PoVsjnx5FyGIMimNJW/q	Hash, 31 Zeichen

bcrypt ist absichtlich langsam, damit Raten nach einem Leak teuer wird

- Gleiches Passwort, neues Salt: Jeder Aufruf ergibt einen anderen Hash.
- compare liest Salt und Kostenfaktor aus dem gespeicherten Hash und vergleicht.
- Kostenfaktor 10 braucht 69 ms je Hash, Kostenfaktor 12 braucht 270 ms.
- Schnelle Hashfunktionen wie SHA-256 eignen sich deshalb nicht für Passwörter.

MERKSATZ

Ein Passwort-Leak ist eine meldepflichtige Datenpanne (DSGVO Art. 33). Ein Hash verlangsamt das Raten; kein Passwort steht im Klartext in der Datenbank.

Ein JWT besteht aus Header, Payload und Signatur, getrennt durch Punkte

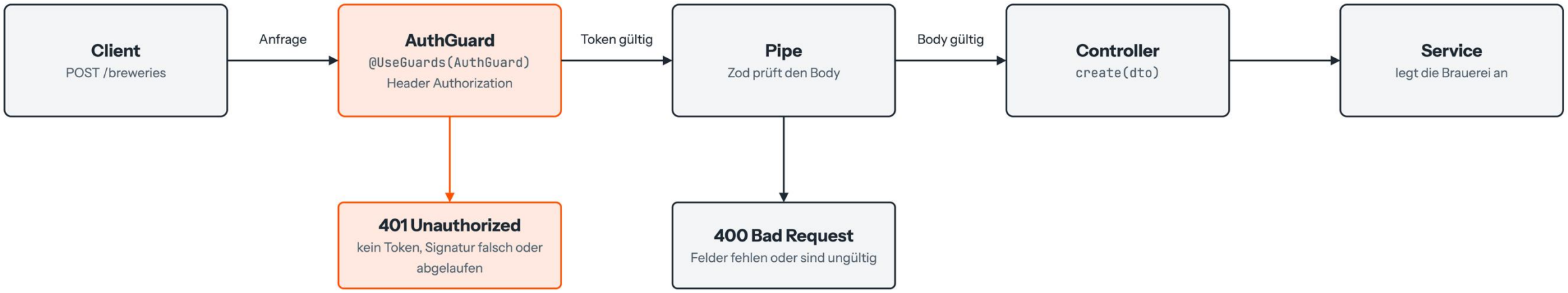
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
.
eyJzdWIiOjEsIm5hbWUiOiJhZG1pbiIsImlhdCI6MTc5Mjk1NDk5MSwiZXhwIjoxNzkyOTU4NTkxfQ
.
wyB2bBZtkSutEhTRtxqen4lAlb6GO-62YZC0rUs4f4M

Header {"alg":"HS256","typ":"JWT"}
Payload {"sub":1,"name":"admin","iat":1792954991,"exp":1792958591}

Header und Payload sind nur kodiert; die Signatur kann nur der Server berechnen

Teil	Inhalt	Bedeutung
Header	alg: HS256	Verfahren der Signatur
Payload: sub, name	1, admin	id und Name des Users
Payload: iat, exp	Sekunden seit 1970	ausgestellt am, gültig bis: eine Stunde später
Signatur	HMAC aus Header, Payload und Secret	Ein geänderter Payload passt nicht mehr: 401
Kodierung	Base64URL ohne Schlüssel	Jeder liest den Payload; kein Passwort hinein

Ein Guard entscheidet vor der Pipe, ob die Anfrage den Controller erreicht



Der Guard läuft vor der Pipe: Ohne Token und mit leerem Body antwortet der Server mit 401.

Weg einer Anfrage in fünf Kästen von links nach rechts: Client mit POST /breweries, Pfeil „Anfrage“ zum hervorgehobenen AuthGuard mit @UseGuards(AuthGuard), Header Authorization; Pfeil „Token gültig“ zur Pipe, Zod prüft den Body; Pfeil „Body gültig“ zum Controller mit create(dto); Pfeil zum Service, legt die Brauerei an. Vom AuthGuard nach unten in Orange: 401 Unauthorized, kein Token, Signatur falsch oder abgelaufen. Von der Pipe nach unten: 400 Bad Request, Felder fehlen oder sind ungültig. Darunter: Der Guard läuft vor der Pipe: Ohne Token und mit leerem Body antwortet der

Schritt 1: Zwei Pakete signieren Tokens und hashen Passwörter

BASH

```
npm install @nestjs/jwt@12.0.3 bcryptjs@3.0.3
```

Schritt 2: Die Entity User speichert Namen und Hash, nie das Passwort

TYPESCRIPT · SRC/USERS/USER.ENTITY.TS

```
@Entity()
export class User {
  @PrimaryGeneratedColumn()
  id: number;

  // Anmeldename; die Datenbank lehnt einen zweiten gleichen Namen ab
  @Column({ unique: true })
  name: string;

  // Nur der Hash, nie das Passwort; erscheint in keiner Antwort
  @Column()
  passwordHash: string;
}
```

Schritt 3: Eine Migration legt die Beispiel-User mit Hash an

TYPESCRIPT · SRC/MIGRATIONS/1792675025666-SEEDUSERS.TS

```
  await queryRunner.manager.insert(User, [
   {
    id: 1,
    name: 'admin',
    passwordHash:
     '$2b$10$o6W5KznaQaytXGZ1itv87ODpeic6pzT91PoVsjnx5FyGIMimNJW/q',
   },
```

Schritt 4: Das AuthModule stellt JwtService bereit, der Login signiert id und Namen

TYPESCRIPT · SRC/AUTH/AUTH.MODULE.TS

```
  JwtModule.register({
    // JwtService in jedem Module, ohne Import
    global: true,
    // Bis Einheit 14 im Code; danach aus einer Umgebungsvariable
    secret: 'beer-api-secret-einheit-13',
    signOptions: { expiresIn: '1h' },
  }),
```

Einheit 13 · Authentifizierung

FRAGE

Live-Frage: Login als nobody, den es nicht gibt. Welcher Code, welche Meldung?

POST http://localhost:3000/auth/login mit { "name": "nobody", "password": "admin123" }

Auflösung: 401 mit derselben Meldung, weil eine eigene Antwort Namen verriete

BEISPIEL

- Ein 404 bestätigte, dass es den Namen nicht gibt.
- User Enumeration: erst Namen sammeln, dann Passwörter raten.
- Die Zeit verrät den Namen trotzdem: 1 ms gegen 69 ms.
- Die Registrierung antwortet mit 409 und verrät Namen ebenfalls.

```
HTTP/1.1 401 Unauthorized

{"message":"Invalid name or password",
 "error":"Unauthorized","statusCode":401}
```

Schritt 5: Der AuthGuard liest das Token aus dem Header und prüft es

TYPESCRIPT · SRC/AUTH/AUTH.GUARD.TS

```
// Läuft vor dem Controller; eine Exception beendet die Anfrage
async canActivate(context: ExecutionContext): Promise<boolean> {
  const request = context.switchToHttp().getRequest<AuthRequest>();
  // Header "Authorization: Bearer <token>"
  const header = request.headers.authorization ?? '';
  const [type, token] = header.split(' ');
  if (type !== 'Bearer' || !token) {
    throw new UnauthorizedException();
  }
  try {
    // Prüft Signatur und Ablauf (exp)
    request.user = await this.jwtService.verifyAsync(token);
  } catch {
    throw new UnauthorizedException();
  }
  return true;
```

Schritt 6: Ein Decorator schützt POST /breweries

TYPESCRIPT · SRC/BREWERIES/BREWERIES.CONTROLLER.TS

```
// Nur mit gültigem Token; ohne Token antwortet der Guard mit 401
@Post()
@UseGuards(AuthGuard)
@ApiBearerAuth()
@ZodResponse({ status: 201, type: BreweryDto })
@ApiBadRequestResponse({ type: ErrorDto })
@ApiUnauthorizedResponse({ type: ErrorDto })
create(@Body() dto: CreateBreweryDto) {
```

Einheit 13 · Authentifizierung

FRAGE

Live-Frage: admin meldet sich ein zweites Mal an. Gilt das erste Token noch?

POST http://localhost:3000/breweries mit dem Token aus dem ersten Login und Brauhaus Riegele

Auflösung: 201, weil der Server keine Tokens speichert und jedes bis exp gilt

BEISPIEL

- Der Guard prüft nur Signatur und Ablauf.
- Ein zweiter Login stellt ein weiteres Token aus.
- Ein gestohlenes Token gilt deshalb bis zu seinem Ablauf.
- Kurze Laufzeiten begrenzen diesen Schaden.

```
HTTP/1.1 201 Created

{"id":6,"name":"Brauhaus Riegele",
 "city":"Augsburg","country":"Deutschland"}
```

Einheit 13 · Authentifizierung

FRAGE

Live-Frage: GET /breweries mit ungültigem Token. Welcher Code?

GET http://localhost:3000/breweries mit Authorization: Bearer kein-gueltiges-token

Auflösung: 200, weil an GET /breweries kein Guard den Header liest

BEISPIEL

- Der Guard steht nur an POST /breweries.
- Ein Header, den kein Code liest, ändert nichts.
- Alle GET-Anfragen bleiben ohne Anmeldung möglich.
- Ein Guard am ganzen Controller schützte auch GET.

```
HTTP/1.1 200 OK

[{"id":1,
  "name":"Privatbrauerei Schlumberger", …}, …]
```

Fehlersuche: Ein frisches Token wird nach wenigen Sekunden mit 401 abgelehnt

BEISPIEL

- Erwartet: 201 mit jedem Token aus dem Login.
- Direkt nach dem Login antwortet der Server mit 201.
- Vier Sekunden später antwortet er mit 401.
- TypeScript und ESLint melden nichts.

```
POST /auth/login          → 200, accessToken
POST /breweries, sofort   → 201
POST /breweries, 4 s später → 401
{"message":"Unauthorized","statusCode":401}
```

Fehlersuche: expiresIn als String ohne Einheit zählt Millisekunden

BEISPIEL

- Spur: Der Payload zeigt exp drei Sekunden nach iat.
- Ursache: '3600' bedeutet 3600 Millisekunden, abgerundet 3 Sekunden.
- Die Zahl 3600 oder der String '1h' bedeuten eine Stunde.
- Beheben: '1h' eintragen, neu anmelden; der Test ist grün.

jwt.io, Payload:
{"sub":1,"name":"admin",
"iat":1792955055,"exp":1792955058}

Collection, Ordner Beispiel:
expected 3 to deeply equal 3600

Übung

Die Aufgaben stehen vollständig im Skript

Aufgaben dieser Einheit

ÜBUNG

1. Aufgabe 13.1: Registrieren und anmelden
2. Aufgabe 13.2: Bewerten nur mit Token
3. Zusatz 13.3: Das Tor im Capture the Flag

SELBSTKONTROLLE

- Beispiel zum Übernehmen: Skript 13.9 bis 13.23
- Aufgaben ab Skript 13.24, Hinweise ab 13.33
- Lösungsstand nach der Übung

Review: zwei Lösungen für Registrierung und Bewertungen mit Token

- Wo prüft ihr den vergebenen Namen: im Service oder in der Datenbank?
- Wie verschwindet passwordHash aus der Antwort der Registrierung?
- Welche Antworten verraten, ob ein Name existiert?
- Was hätte ein Identity Provider euch abgenommen?

MERKSATZ

Die Beer API weiß jetzt, wer bewertet. Wer Brauereien und Biere ändern darf, regelt Einheit 14.

Das solltest du jetzt können

- ✓ Erklären, warum Passwörter nur als Hash mit Salt gespeichert werden
- ✓ Ein JWT ausstellen und Header, Payload und Signatur lesen
- ✓ Einen Endpunkt mit einem Guard schützen
- ✓ In Postman ein Token holen und als Bearer senden
- ✓ 401 begründen und von 403 abgrenzen

WEB-PROGRAMMIERUNG · EINHEIT 14

Autorisierung und sichere Grundeinstellungen

Einheit 14 · Autorisierung und sichere Grundeinstellungen

FRAGE

Ohne Anmeldung benennt heute jeder eine Brauerei um: Was kostet das den Betreiber?

Am Stand der Einheit 13 antwortet PATCH /breweries/1 ohne Token mit 200, DELETE mit 204. Mit dem Token eines Gastes legt jeder Brauereien an. Die regionalen Brauereien brauchen Stammdaten, die nur der Betreiber pflegt.

Aufgabe: Nur der Betreiber ändert Stammdaten und vergibt Rollen

- Biere und Brauereien anlegen, ändern und löschen darf nur die Rolle admin.
- Mit der Rolle user antwortet der Server darauf mit 403, ohne Token mit 401.
- Bewerten bleibt jedem angemeldeten User erlaubt.
- Rollen vergibt nur ein admin; eine Registrierung mit einem Feld roles ergibt 400.

MERKSATZ

Als Betreiber möchte ich, dass nur der Betreiber Stammdaten und Rollen ändert, damit niemand die App manipuliert oder sich selbst Rechte gibt.

Ein Gast mit zu vielen Rechten kostet Arbeit, Verfügbarkeit und Vertrauen

Posten	Bedeutung	Gegenmaßnahme
Wiederherstellen	Backup einspielen, jede Änderung seit dem Backup von Hand prüfen	nur admin ändert Stammdaten
Ausfall	Auf einer Veranstaltung zeigt die App falsche Brauereien	403 für alle anderen
Vertrauen	Brauereien und Gäste zweifeln an den Daten der App	Rollen vergibt nur ein admin
Meldung und Haftung	Datenpanne mit personenbezogenen Daten: Meldung binnen 72 Stunden	keine Antwort enthält passwordHash

Welche Felder der Beer API führen auf eine Person zurück?

User	id · name · passwordHash
UserRole	userId · roleId
Rating	id · beerId · userId · score · comment · createdAt
Beer	id · name · style · abv · breweryId · description
Brewery	id · name · city · country

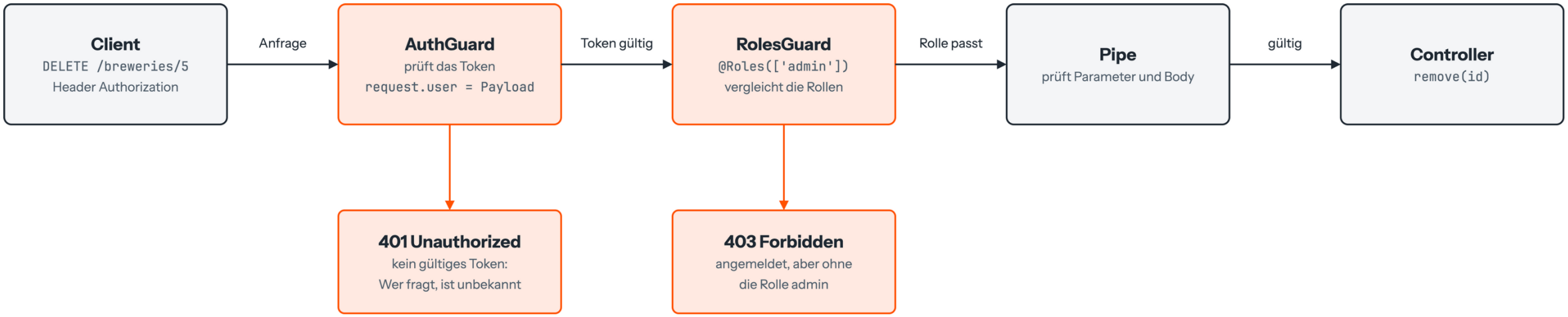
Personenbezogen ist jedes Feld, das auf eine Person zurückführt

Feld	Personenbezogen	Bedeutung
User.name, passwordHash	ja	Name eines Gastes; der Hash gehört zu genau einer Person
jede userId	ja	verweist auf einen User, auch in UserRole und Rating
Rating.comment	ja	Freitext kann Namen, Orte oder Gesundheitsangaben enthalten
Rating.createdAt	ja, mit userId	wann eine Person welches Bier bewertet
Beer, Brewery	nein	Stammdaten ohne Bezug zu einer Person

RBAC: Rechte hängen an Rollen, Rollen an Usern

Begriff	Bedeutung	Beer API
Rolle	Gruppe von Rechten	admin (Betreiber), user (Gäste)
Zuordnung	welcher User welche Rolle hat	Tabelle user_role seit Einheit 13
Recht am Endpunkt	welche Rolle eine Anfrage braucht	Decorator @Roles(['admin'])
Prüfung	Rolle des Users gegen Recht des Endpunkts	RolesGuard nach dem AuthGuard
Ablehnung	angemeldet, aber ohne passende Rolle	403 Forbidden

Erst stellt der AuthGuard fest, wer fragt; dann prüft der RolesGuard die Rolle



@UseGuards(AuthGuard, RolesGuard): NestJS ruft die Guards von links nach rechts auf. Der erste, der ablehnt, beendet die Anfrage.

Weg einer Anfrage in fünf Kästen von links nach rechts: Client mit DELETE /breweries/5 und Header Authorization; Pfeil „Anfrage“ zum hervorgehobenen AuthGuard, prüft das Token, request.user = Payload; Pfeil „Token gültig“ zum hervorgehobenen RolesGuard mit @Roles und der Rolle admin, vergleicht die Rollen; Pfeil „Rolle passt“ zur Pipe, prüft Parameter und Body; Pfeil „gültig“ zum Controller mit remove(id). Vom AuthGuard nach unten in Orange: 401 Unauthorized, kein gültiges Token, wer fragt, ist unbekannt. Vom RolesGuard nach unten in Orange: 403 Forbidden, angemeldet, aber ohne die Rolle

OWASP Top 10:2025, A01 bis A05: drei davon betreffen die Beer API heute

Kategorie	Bedeutung	Beer API
A01 Broken Access Control	User handeln außerhalb ihrer Rechte	RolesGuard je Endpunkt, Schema lehnt fremde Felder ab (heute)
A02 Security Misconfiguration	unsichere Einstellungen, Standardkonten, Secrets im Code	Secret in .env, Beispiel-User nur in der Entwicklung (heute)
A03 Software Supply Chain Failures	Lücken in Paketen und Werkzeugen	npm audit meldet bekannte Lücken in den Paketen
A04 Cryptographic Failures	Daten ungeschützt gespeichert oder übertragen	Passwort als bcrypt-Hash, Token im Betrieb nur über HTTPS (Einheit 13)
A05 Injection	eine Eingabe wird Teil eines Befehls	TypeORM übergibt Werte als Parameter (heute)

OWASP Top 10:2025, A06 bis A10: die Beer API berücksichtigt drei, zwei sind offen

Kategorie	Bedeutung	Beer API
A06 Insecure Design	Schutz fehlt schon im Entwurf	teilweise: Regeln und Akzeptanzkriterien je Story (Einheit 10)
A07 Authentication Failures	Anmeldung lässt sich umgehen oder erraten	gleiche Meldung beim Login; keine Sperre nach Fehlversuchen (Einheit 13)
A08 Software or Data Integrity Failures	Code oder Daten ungeprüft übernommen	Prüfsummen der Pakete in package-lock.json
A09 Security Logging and Alerting Failures	Angriffe bleiben unbemerkt	abgelehnte Anfragen werden nicht protokolliert
A10 Mishandling of Exceptional Conditions	Fehler führen in einen unsicheren Zustand	500 ohne Details, ein Fehlerformat (Einheit 8)

Mass Assignment: Der Client schickt ein Feld, das er nicht setzen darf

- Beispiel: "roles": ["admin"] bei der Registrierung.
- Übernimmt der Server den Body ungeprüft, hat sich die Person Rechte gegeben.
- Das Schema beschreibt genau die Felder, die der Client senden darf.
- Zod entfernt unbekannte Felder; mit .strict() antwortet der Server mit 400.

MERKSATZ

Ein abgelehntes Feld macht den Versuch sichtbar, für den Client und im Test.

Injection entsteht, wenn Code SQL aus Text und Eingabe zusammensetzt

SQL aus Text zusammengesetzt

- city = ' OR '1'='1
- Bedingung: city = '' OR '1'='1'
- Die Bedingung ist für jede Zeile wahr.
- Die Eingabe ändert den Befehl.

Wert als Parameter

- Der SQL-Text enthält den Platzhalter ?.
- Der Server sendet den Wert getrennt vom Befehl.
- Die Datenbank sucht genau diesen Text.
- Zod prüft Typ und Form eines Werts.

Entwicklung und Betrieb bekommen verschiedene Werte aus der Umgebung

Wert	Bedeutung	Entwicklung	Betrieb
JWT_SECRET	signiert und prüft jedes Token	eigener Wert in .env	lang, zufällig, von der Plattform gesetzt
NODE_ENV	nennt die Umgebung	nicht gesetzt	production
Beispiel-User	admin und gast mit bekanntem Passwort	aus der Migration	keine, solange NODE_ENV=production gesetzt ist
.env	Datei mit Werten für die lokale Entwicklung	nicht im Repository	entfällt

Schritt 1: Ein Paket liest .env; die Vorlage kommt ins Repository

BASH

```
npm install @nestjs/config@12.0.1
```

Schritt 2: Das JwtModule holt das Secret aus dem ConfigService

TYPESCRIPT · SRC/APP.MODULE.TS

```
// Liest .env; ConfigService in jedem Module, ohne Import
ConfigModule.forRoot({ isGlobal: true }),
```

Schritt 3: Die Migration legt Beispiel-User nur in der Entwicklung an

TYPESCRIPT · SRC/MIGRATIONS/1792675025666-SEEDUSERS.TS

```
  await queryRunner.manager.insert(Role, [
    { id: 1, name: 'admin' },
    { id: 2, name: 'user' },
  ]);
  // Bekannte Passwörter nur in der Entwicklung; im Betrieb gilt
  // NODE_ENV=production, dort gibt es keine Beispiel-User
  if (process.env.NODE_ENV === 'production') {
    return;
  }
  await queryRunner.manager.insert(User, [
```

Schritt 4: Der Login schreibt die Rollen aus der Datenbank in den Payload

TYPESCRIPT · SRC/USERS/USERS.SERVICE.TS

```typescript
async findRoles(userId: number): Promise<string[]> {
  const userRoles = await this.userRolesRepository.find({
    where: { userId },
    relations: { role: true },
  });
  return userRoles.map((userRole) => userRole.role.name);
}
```

Schritt 5: Ein Decorator nennt die Rolle, der RolesGuard vergleicht

TYPESCRIPT · SRC/AUTH/ROLES.DECORATOR.TS

```
// @Roles(['admin']): Rollen, von denen der User eine braucht
export const Roles = Reflector.createDecorator<string[]>();
```

Schritt 6: Zwei Guards und eine Rolle schützen DELETE /breweries/:id

TYPESCRIPT · SRC/BREWERIES/BREWERIES.CONTROLLER.TS

```
@Delete(':id')
@UseGuards(AuthGuard, RolesGuard)
@Roles(['admin'])
@ApiBearerAuth()
```

Drei Anfragen, drei Antworten an DELETE /breweries/5

BEISPIEL

- Ohne Token: 401 vom AuthGuard.
- Token von gast: 403 vom RolesGuard.
- Token von admin: 204, die Brauerei ist gelöscht.
- GET /breweries bleibt ohne Token offen.

```
ohne Token  → 401
{"message":"Unauthorized","statusCode":401}

gast        → 403
{"message":"Forbidden resource",
 "error":"Forbidden","statusCode":403}

admin       → 204 (kein Body)
```

Einheit 14 · Autorisierung und sichere Grundeinstellungen

FRAGE

Live-Frage: lena bekommt die Rolle admin. Was antwortet ihr bisheriges Token?

POST http://localhost:3000/breweries mit dem Token, das lena vor der neuen Rolle bekommen hat

Auflösung: 403, weil der RolesGuard die Rollen aus dem Token liest

BEISPIEL

- Das Token nennt nur die Rolle user.
- Ein neuer Login schreibt admin in den Payload.
- Eine entzogene Rolle gilt ebenso bis zum Ablauf weiter.
- Kurze Laufzeiten begrenzen diese Verzögerung.

altes Token → 403 Forbidden

neuer Login, Payload:
{"sub":3,"name":"lena",
"roles":["admin","user"],…}

neues Token → 201 Created

Einheit 14 · Autorisierung und sichere Grundeinstellungen

FRAGE

Live-Frage: mallory registriert sich mit dem Feld roles. Welcher Statuscode?

POST http://localhost:3000/auth/register mit { "name": "mallory", "password": "mallory123", "roles": ["admin"] }

Auflösung: 201, weil Zod das Feld roles still entfernt; mallory bleibt user

BEISPIEL

- Zod entfernt roles still; der Service sieht das Feld nie.
- Die Registrierung vergibt immer die Rolle user.
- Der Versuch bleibt für Client und Test unsichtbar.
- Aufgabe 14.2 verlangt deshalb 400.

```
POST /auth/register → 201
{"id":4,"name":"mallory"}

Payload nach dem Login:
{"sub":4,"name":"mallory",
 "roles":["user"],…}

POST /breweries     → 403
```

Einheit 14 · Autorisierung und sichere Grundeinstellungen

FRAGE

Live-Frage: GET /breweries?city=' OR '1'='1. Was antwortet der Server?

GET http://localhost:3000/breweries?city=%27%20OR%20%271%27%3D%271

Auflösung: 200 mit leerer Liste, weil TypeORM den Wert als Parameter sendet

BEISPIEL

- Die Datenbank sucht eine Stadt mit genau diesem Namen.
- Keine Brauerei liegt in „' OR '1'='1“.
- Rohes SQL aus Text und Eingabe öffnet die Lücke.
- Auch dort gehören Werte in Parameter.

```
GET /breweries?city=… → 200
[]

Log von TypeORM, gekürzt:
WHERE (("Brewery"."city" = ?))
-- PARAMETERS: ["' OR '1'='1"]
```

Fehlersuche: admin löscht Brauerei 5, und der Server antwortet mit 500

BEISPIEL

- Erwartet: 204 mit dem Token von admin.
- PATCH mit demselben Token antwortet mit 200.
- Ohne Token kommt ebenfalls 500 statt 401.
- TypeScript und ESLint melden nichts.

```
DELETE /breweries/5 mit admin  → 500
{"statusCode":500,
 "message":"Internal server error"}

PATCH /breweries/5 mit admin   → 200
DELETE /breweries/5 ohne Token → 500
```

Fehlersuche: Der RolesGuard lief vor dem AuthGuard und las request.user leer

BEISPIEL

- Spur: Das Terminal nennt den RolesGuard und das Feld roles.
- Ursache: request.user setzt erst der AuthGuard.
- Die Guards laufen in der Reihenfolge der Liste.
- Beheben: @UseGuards(AuthGuard, RolesGuard); der Test ist grün.

Terminal von start:dev:
ERROR [ExceptionsHandler] TypeError:
Cannot read properties of undefined
(reading 'roles')
    at RolesGuard.canActivate

Collection, Ordner Beispiel:
DELETE /breweries/5 ohne Token
expected 500 to deeply equal 401

Übung

Die Aufgaben stehen vollständig im Skript

Aufgaben dieser Einheit

ÜBUNG

1. Aufgabe 14.1: Biere schützen
2. Aufgabe 14.2: Rollen vergibt nur der admin

SELBSTKONTROLLE

- Beispiel zum Übernehmen: Skript 14.8 bis 14.17
- Aufgaben ab Skript 14.18, Hinweise ab 14.27
- Lösungsstand nach der Übung

Review: zwei Lösungen für Rechte an Bieren und Rollen

- Wie sieht eure Vorlage „Endpunkt und Rolle“ aus Skript 14.19 aus?
- 401 oder 403: Woran erkennt ihr den Unterschied in Postman?
- Nach POST /users/2/roles: Wer kennt die neue Rolle zuerst, /users/me oder das Token?
- Welche Felder gibt /users/me zurück, welche nicht?

MERKSATZ

Die Beer API schützt jetzt Stammdaten und Rollen. Einheit 15 zeigt neue Bewertungen live auf einem Dashboard.

Das solltest du jetzt können

- ✓ Einen Endpunkt mit AuthGuard, RolesGuard und @Roles auf eine Rolle beschränken
- ✓ 401 und 403 an einem Endpunkt unterscheiden und begründen
- ✓ Mass Assignment erkennen und mit .strict() verhindern
- ✓ Ein Secret aus dem Code in eine Umgebungsvariable verschieben
- ✓ Die zehn Kategorien der OWASP Top 10 der Beer API zuordnen

PlanB. Trusted Agentic Operator

DHBW Duale Hochschule Baden-Württemberg Heidenheim

WEB-PROGRAMMIERUNG · EINHEIT 15

WebSockets

Einheit 15 · WebSockets

FRAGE

200 Bildschirme fragen alle fünf Sekunden nach dem Ranking: Wer zahlt diese Last?

Auf Veranstaltungen zeigt ein Bildschirm am Stand die beliebtesten Biere. Die regionalen Brauereien wollen ein Ranking, das neue Bewertungen sofort zeigt. Rechtfertigt „live“ die Komplexität?

Aufgabe: Das Dashboard zeigt die fünf besten Biere ohne Neuladen

- Die Seite /dashboard zeigt die fünf Biere mit der höchsten Durchschnittsbewertung.
- Biere ohne Bewertung fehlen im Ranking.
- Nach einer Bewertung zeigen zwei offene Fenster dasselbe neue Ranking.
- Der Server sendet das Ranking nur nach einer neuen Bewertung.

MERKSATZ

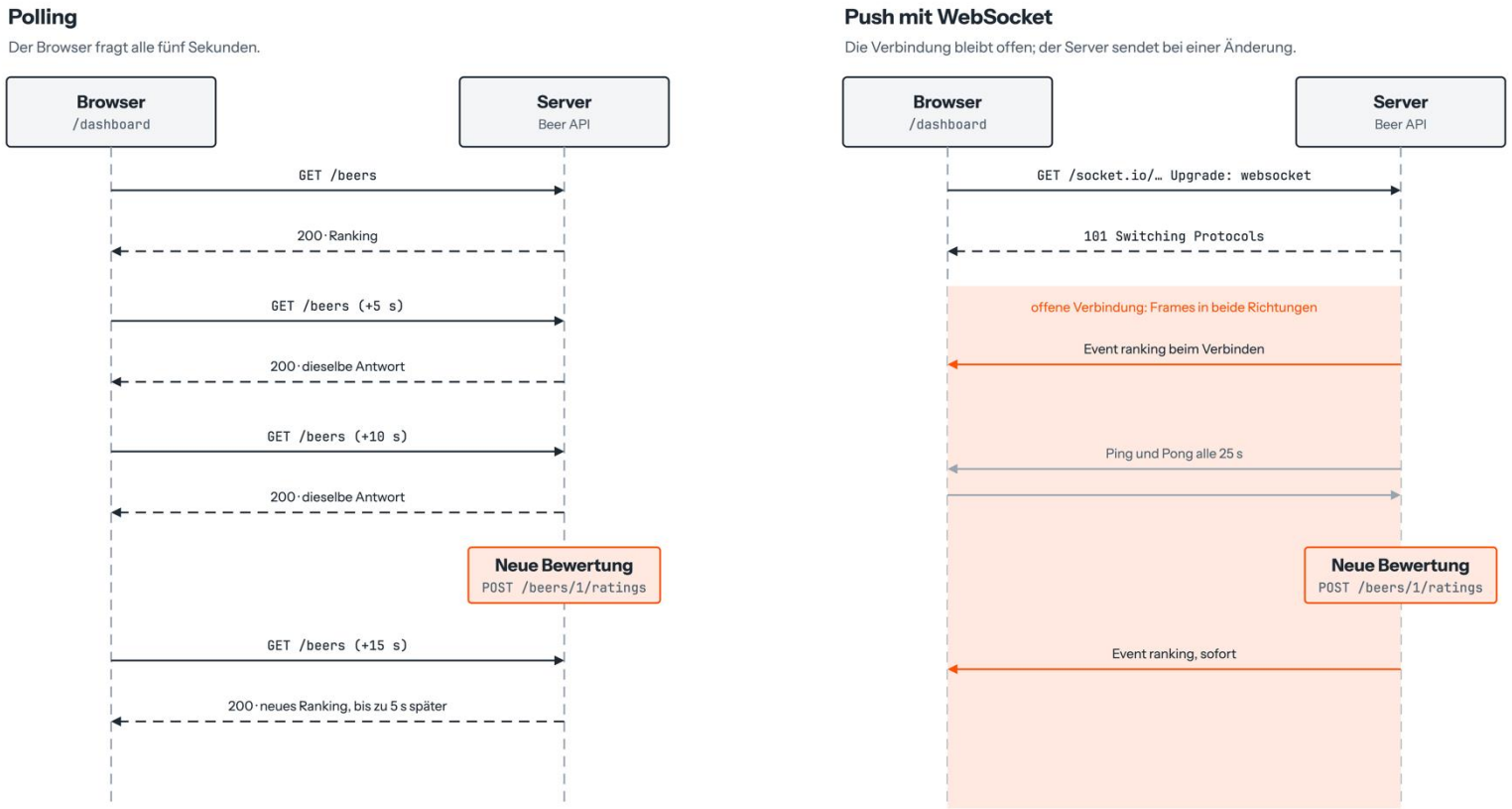
Als Betreiber möchte ich die fünf bestbewerteten Biere live sehen, damit ich auf Veranstaltungen zeige, was gerade gut ankommt.

Polling fragt in festen Abständen, auch wenn sich nichts geändert hat

JAVASCRIPT · PUBLIC/DASHBOARD.HTML

```
async function poll() {
  const url = '/beers?sort=averageRating&order=desc';
  const response = await fetch(url);
  const beers = await response.json();
  // Biere ohne Bewertung fehlen, die besten fünf bleiben
  const rated = beers.filter((b) => b.averageRating !== null);
  render(rated.slice(0, 5));
}
poll();
setInterval(poll, 5000);
```

Bei Push hält der Browser eine Verbindung offen, und der Server meldet sich



Polling erzeugt Anfragen je Fenster und Zeit, auch ohne Änderung. Push sendet je Änderung einen Frame an jedes verbundene Fenster. Vereinfacht: socket.io beginnt mit einigen Anfragen über HTTP und wechselt dann.

Zwei Sequenzen nebeneinander zwischen Browser und Server. Links Polling: Der Browser sendet GET /beers sofort, nach 5 und nach 10 Sekunden und bekommt jedes Mal 200, zweimal dieselbe Antwort; nach einer neuen Bewertung kommt das neue Ranking erst mit der Anfrage nach 15 Sekunden, bis zu 5 Sekunden später. Rechts Push mit WebSocket: Der Browser sendet GET /socket.io/… mit Upgrade: websocket, der Server antwortet 101 Switching Protocols. Danach ist die Verbindung offen, Frames laufen in beide Richtungen: das Event ranking beim Verbinden, Ping und Pong alle 25 Sekunden, und

Bei 200 Bildschirmen liest Polling das Ranking 2 400-mal je Minute, Push 30-mal

Je Minute	Polling alle 5 Sekunden	Push
Anfragen ohne Bewertung	2 400	keine, nur Ping und Pong
Ranking lesen bei 30 Bewertungen	2 400	30
Nachrichten an die Bildschirme	2 400 Antworten	6 000 Frames
Offene Verbindungen	keine	200
Verzögerung nach einer Bewertung	bis 5 Sekunden	sofort

Eine WebSocket-Verbindung beginnt als HTTP-Anfrage

HTTP

```
GET /socket.io/?EIO=4&transport=websocket&sid=… HTTP/1.1
Connection: Upgrade
Upgrade: websocket

HTTP/1.1 …
Connection: Upgrade
Upgrade: websocket
```

Ein Frame von socket.io trägt den Namen eines Events und die Daten als JSON

Frame	Richtung	Bedeutung
2 und 3	Server, dann Client	Ping und Pong alle 25 Sekunden: Die Verbindung steht
42["breweryCreated",{…}]	Server an Client	Event: 4 heißt Nachricht, 2 heißt Event; danach Name und Daten
420["breweries"]	Client an Server	Event mit Nummer 0: Der Client erwartet eine Antwort
430[[{…}]]	Server an Client	Antwort auf das Event mit Nummer 0 (Acknowledgement)

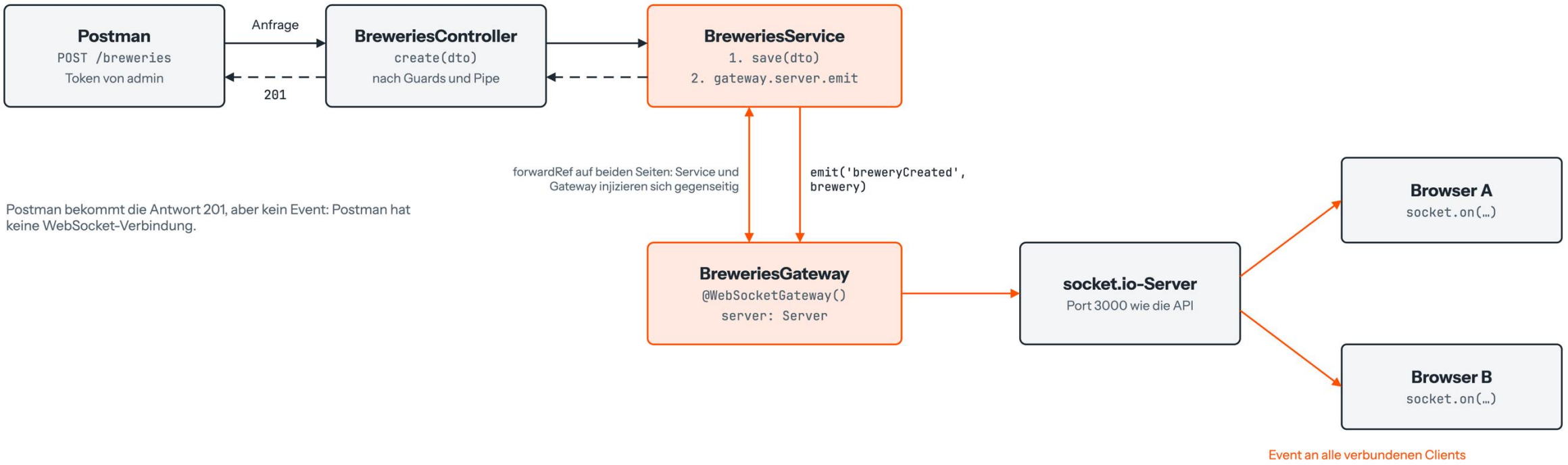
socket.io ergänzt WebSocket um Events, Antworten und automatisches Neuverbinden

- Jede Nachricht trägt den Namen eines Events und Daten.
- Der Empfänger kann ein Event beantworten (Acknowledgement).
- Nach einer Trennung verbindet sich der Client selbst neu.
- Den Client für den Browser liefert der Server unter /socket.io/socket.io.js.
- socket.io hat ein eigenes Format in den Frames; ein reiner WebSocket-Client versteht es nicht.

Ein Gateway nimmt Verbindungen und Events an wie ein Controller Anfragen

Baustein	Bedeutung	Im Controller
@WebSocketGateway()	Klasse nimmt Verbindungen an; ohne Argument auf dem Port der API	@Controller()
@WebSocketServer() server	der socket.io-Server; NestJS setzt ihn nach dem Start	–
@SubscribeMessage('name')	Methode für ein Event vom Client; der Rückgabewert ist die Antwort	@Get(), @Post()
server.emit('name', daten)	sendet ein Event an alle verbundenen Clients	–
client.emit('name', daten)	sendet ein Event an genau einen Client	–
handleConnection(client)	läuft bei jeder neuen Verbindung	–

Der Service sendet das Event nach dem Speichern an alle verbundenen Clients



Das Event entsteht im Service nach dem Speichern. Lehnt ein Guard oder die Pipe die Anfrage ab, läuft create nicht, und es gibt kein Event.

Weg eines Events an den Brauereien: Postman sendet POST /breweries mit dem Token von admin an den BreweriesController, der nach Guards und Pipe create(dto) aufruft. Der hervorgehobene BreweriesService führt 1. save(dto) und 2. gateway.server.emit aus und gibt die Brauerei zurück; Postman bekommt 201. Zwischen BreweriesService und dem hervorgehobenen BreweriesGateway mit @WebSocketGateway() und server: Server stehen Pfeile in beide Richtungen: forwardRef auf beiden Seiten, Service und Gateway injizieren sich gegenseitig. Das Gateway sendet emit('breweryCreated', brewery)

Push lohnt sich, wenn viele Clients eine Änderung sofort sehen sollen

Fall	Ändert sich	Muss sofort sichtbar sein	Weg
Ranking auf einer Veranstaltung	bei jeder Bewertung	viele Bildschirme	Push
Chat, Spielstand, Benachrichtigung	laufend	alle Beteiligten	Push
Bierliste einer Brauerei	selten	nein	Abruf bei Bedarf
Bericht für den Betreiber	täglich	nein, eine Person liest ihn	Abruf bei Bedarf

Schritt 1: Der Server liefert die Seite aus dem Ordner public

TYPESCRIPT · SRC/MAIN.TS

```
const app = await NestFactory.create<NestExpressApplication>(AppModule);
// Dateien aus public/: /dashboard liefert public/dashboard.html
app.useStaticAssets(join(__dirname, '..', 'public'), {
  extensions: ['html'],
});
```

Schritt 2 und 3: Zwei Pakete installieren, die Nest CLI erzeugt das Gateway

BASH

```
npm install @nestjs/websockets@11.2.7 \
  @nestjs/platform-socket.io@11.2.7

npx nest generate gateway breweries
CREATE src/breweries/breweries.gateway.ts (243 bytes)
UPDATE src/breweries/breweries.module.ts (640 bytes)
```

Schritt 4: Nach dem Speichern sendet der Service über das Gateway ein Event

TYPESCRIPT · SRC/BREWERIES/BREWERIES.GATEWAY.TS

```
@WebSocketGateway()
export class BreweriesGateway {
  // Der socket.io-Server; emit sendet an alle verbundenen Clients
  @WebSocketServer()
  server: Server;
```

Schritt 5: Die Console verbindet sich und hört auf das Event

JAVASCRIPT

```
const socket = io();
socket.on('breweryCreated', (b) => console.log(b));
```

Postman bekommt 201, die Console die neue Brauerei als Event

BEISPIEL

- Postman: Antwort mit allen Feldern
- Console: die Brauerei als Objekt
- Messages: ein Frame mit dem Namen des Events
- Postman selbst bekommt kein Event.

```
POST /breweries → 201
{"id":5,"name":"Brauerei Gold Ochsen",
 "city":"Ulm","country":"Deutschland"}

Messages:
42["breweryCreated",
 {"name":"Brauerei Gold Ochsen","city":"Ulm",
  "country":"Deutschland","id":5}]
```

Einheit 15 · WebSockets

FRAGE

Live-Frage: Welchen Statuscode zeigt der Network-Tab für die WebSocket-Verbindung?

Network-Tab, Filter Socket, der Eintrag aus Schritt 5

Auflösung: 101 Switching Protocols, weil die Verbindung das Protokoll wechselt

BEISPIEL

- 101: Der Server stimmt dem Wechsel auf WebSocket zu.
- 1xx sind Zwischenantworten; hier folgt das neue Protokoll.
- Davor stehen vier Anfragen über HTTP mit 200.
- Danach Frames statt Anfragen, bis eine Seite schließt.

Fetch/XHR: 4 × GET /socket.io/
?EIO=4&transport=polling… → 200

Socket: GET /socket.io/
?EIO=4&transport=websocket&sid=…
→ 101 Switching Protocols

Messages: 2probe, 3probe, 5

Schritt 6: Ein Event vom Client mit Antwort braucht den Service im Gateway

TYPESCRIPT · SRC/BREWERIES/BREWERIES.GATEWAY.TS

```
// Der Service injiziert das Gateway: forwardRef auf beiden Seiten
constructor(
  @Inject(forwardRef(() => BreweriesService))
  private readonly breweriesService: BreweriesService,
) {}

// Event vom Client; der Rückgabewert geht als Antwort zurück
@SubscribeMessage('breweries')
findAll() {
  return this.breweriesService.findAll({ order: 'asc' });
}
```

Schritt 7: Zwei Fenster empfangen dasselbe Event

1. Ein zweites Fenster mit /dashboard öffnen.
2. Dieselben zwei Zeilen aus Schritt 5 in dessen Console eingeben.
3. In Postman POST /breweries mit dem Token von admin senden.
4. Beide Consoles zeigen dieselbe neue Brauerei.

MERKSATZ

server.emit sendet an alle verbundenen Clients, unabhängig davon, wer die Anfrage gestellt hat.

Einheit 15 · WebSockets

FRAGE

Live-Frage: gast sendet POST /breweries. Zeigt die Console ein Event?

POST http://localhost:3000/breweries mit dem Token von gast

Auflösung: kein Event, weil der RolesGuard ablehnt, bevor create läuft

BEISPIEL

- Der RolesGuard antwortet mit 403.
- Controller und Service laufen nicht, also auch kein emit.
- Das Event entsteht nur nach save im Service.
- Dasselbe gilt für 400 aus der Pipe.

POST /breweries mit gast → 403
{"message":"Forbidden resource",
"error":"Forbidden","statusCode":403}

Console: keine neue Zeile
Messages: kein neuer Frame

Einheit 15 · WebSockets

FRAGE

Live-Frage: start:dev startet nach einer Änderung neu. Was passiert mit der Verbindung?

Kommentar in breweries.service.ts ändern und speichern, danach POST /breweries mit admin

Auflösung: Das Event kommt an, weil socket.io sich selbst neu verbindet

BEISPIEL

- Der Neustart schließt die Verbindung.
- Kurz darauf steht eine neue mit 101 und neuer sid.
- Events kommen wieder an, ohne Neuladen.
- Events während der Trennung bekommt niemand.

```
Socket: Verbindung geschlossen
  neue Verbindung, neue sid → 101

POST /breweries mit admin → 201
Messages:
42["breweryCreated",
 {"name":"Brauhaus Riegele",…,"id":6}]
```

Fehlersuche: Postman zeigt die Brauerei, die Console ein leeres Objekt

BEISPIEL

- Erwartet: die Brauerei als Event
- Postman: 201 mit allen Feldern
- Console: {}
- TypeScript und ESLint melden nichts.

```
POST /breweries mit admin → 201
{"id":5,"name":"Brauerei Gold Ochsen",
 "city":"Ulm","country":"Deutschland"}

Console auf /dashboard: {}
```

Fehlersuche: Ohne await trägt das Event ein Promise, und JSON macht daraus {}

BEISPIEL

- Spur: Der Frame ist schon im Server leer.
- Die Brauerei ist gespeichert, Postman zeigt sie.
- Ursache: emit läuft, bevor save fertig ist.
- Beheben: await vor save; das Event trägt alle Felder.

```
Messages: 42["breweryCreated",{}]
GET /breweries/5 → 200, gespeichert
Terminal: keine Meldung

JSON.stringify(Promise.resolve(1))
→ '{}'
```

Übung

Die Aufgaben stehen vollständig im Skript

Aufgaben dieser Einheit

ÜBUNG

1. Aufgabe 15.1: Ranking beim Verbinden
2. Aufgabe 15.2: Neues Ranking nach jeder Bewertung

SELBSTKONTROLLE

- Beispiel zum Übernehmen: Skript 15.8 bis 15.14
- Aufgaben ab Skript 15.15, Hinweise ab 15.21
- Lösungsstand nach der Übung

Review: zwei Lösungen für das Dashboard

- Wo entsteht das Ranking: im Service, im Gateway oder in der Seite?
- server.emit oder client.emit: Wer bekommt welches Event?
- Wie kommt das Ranking an die Biere, ohne das BeersModule zu importieren?
- Lohnt sich Push für das Dashboard der Veranstaltung?

MERKSATZ

Das Dashboard zeigt neue Bewertungen sofort. Einheit 16 liefert Biere als fertige HTML-Seiten.

Das solltest du jetzt können

- ✓ Polling und Push an einer Seite im Network-Tab unterscheiden
- ✓ Ein Gateway mit der Nest CLI erzeugen und Events vom Client annehmen
- ✓ Aus einem Service nach dem Speichern ein Event an alle Clients senden
- ✓ Einem neuen Client beim Verbinden den aktuellen Stand senden
- ✓ Begründen, wann sich Push für eine Funktion lohnt

PlanB. Trusted Agentic Operator

DHBW Duale Hochschule Baden-Württemberg Heidenheim

WEB-PROGRAMMIERUNG · EINHEIT 16

Dynamische serverseitige Erzeugung

Einheit 16 · Dynamische serverseitige Erzeugung

AUSSAGE

Eine Brauerei verlinkt /beers auf ihrer Website, und ihre Gäste sehen 488 Byte JSON

Die Brauereien wollen ihre Bierliste zeigen, ohne selbst zu programmieren. Warum wollen Marketing und Brauereien fertiges HTML vom Server?

Aufgabe: Die Beer API liefert Bierliste und Detailseite als fertige HTML-Seiten

- GET /beers mit Accept: text/html zeigt eine Tabelle mit Name, Stil, Brauerei und Durchschnitt.
- GET /beers mit Accept: application/json antwortet weiterhin mit JSON.
- GET /beers/1 mit Accept: text/html zeigt das Nattheimer Spezial mit allen Bewertungen.
- GET /beers/42 mit Accept: text/html antwortet mit 404.

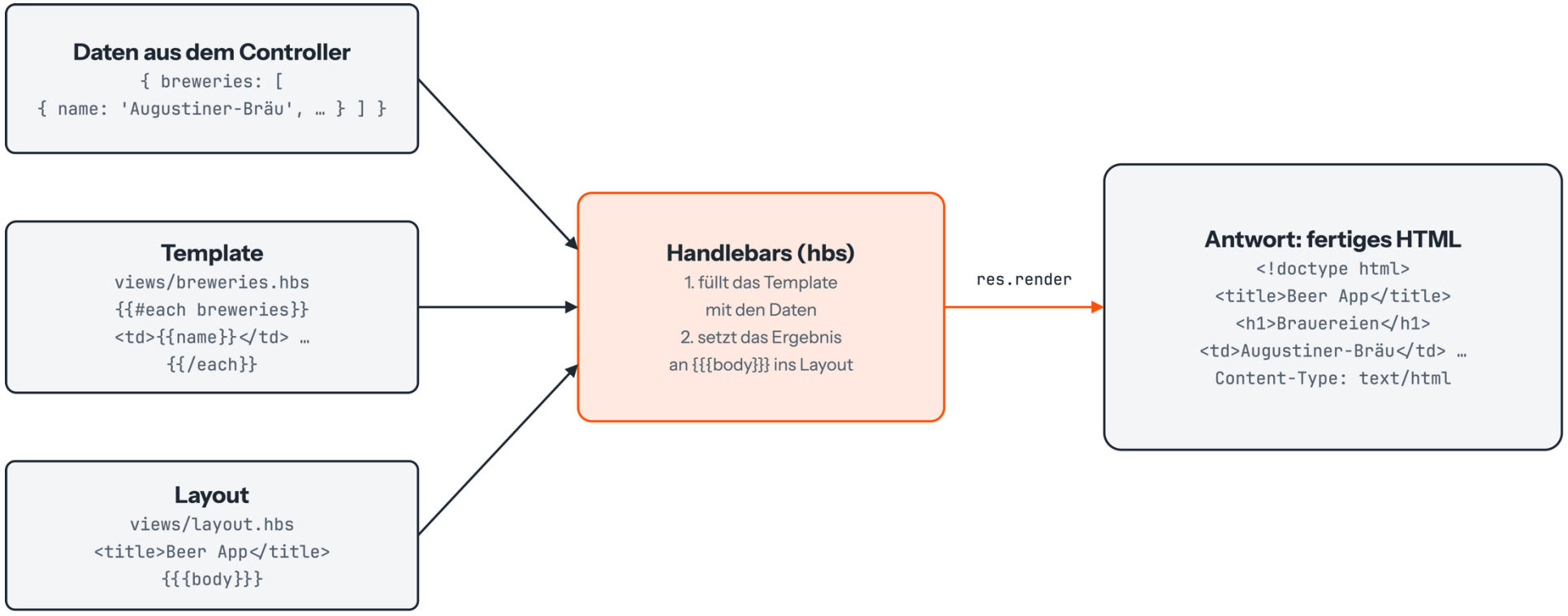
MERKSATZ

Als Brauerei möchte ich die Bierliste und die Detailseite eines Biers als fertige HTML-Seite abrufen, damit ich sie ohne eigene Programmierung verlinke.

Fertiges HTML vom Server erreicht Leser, die kein JavaScript ausführen

Leser	Führt JavaScript aus	Besser geeignet
Suchmaschine	nicht jede, Google erst in einer späteren Phase	HTML vom Server
E-Mail-Programm mit einem Bericht	nein, Skripte werden entfernt	HTML vom Server
Gast auf einem langsamen Gerät	ja, die Seite erscheint aber erst danach	HTML vom Server
Dashboard mit Live-Ranking aus Einheit 15	ja, die Seite ändert sich laufend	Frontend im Browser

Die Template-Engine setzt Daten in das Template und das Ergebnis in das Layout



{{name}} maskiert Zeichen mit Bedeutung in HTML: aus < wird <. {{{body}}} setzt HTML unverändert ein; das Layout bekommt so das fertige Template.

Fehlt ein Feld in den Daten, bleibt die Stelle leer. Die Engine meldet keinen Fehler.

Drei Eingaben links: Daten aus dem Controller, ein Objekt mit der Liste breweries; das Template views/breweries.hbs mit {{#each breweries}}, <td>{{name}}</td> und {{/each}}; das Layout views/layout.hbs mit <title>Beer App</title> und {{{body}}}. Alle drei gehen in die hervorgehobene Template-Engine Handlebars (hbs). Sie füllt 1. das Template mit den Daten und setzt 2. das Ergebnis an {{{body}}} ins Layout. Über res.render geht das Ergebnis als Antwort heraus: fertiges HTML mit <!doctype html>, <title>Beer App</title>, <h1>Brauereien</h1>, <td>Augustiner-Bräu</td> und Content-

Fünf Bausteine von Handlebars reichen für Tabelle, Layout und leere Werte

Baustein	Bedeutung
{{name}}	setzt einen Wert ein und maskiert HTML darin
{{#each breweries}} … {{else}} … {{/each}}	wiederholt den Teil je Element; else gilt für eine leere Liste
{{#if city}} … {{else}} … {{/if}}	zeigt einen von zwei Teilen; falsch sind null, undefined, false, 0, leerer Text und leere Liste
{{brewery.city}}	liest ein Feld eines übergebenen Objekts
{{! Kommentar }}	erscheint nicht in der Seite

Zwei Klammern maskieren HTML im Wert, drei Klammern setzen den Wert unverändert ein

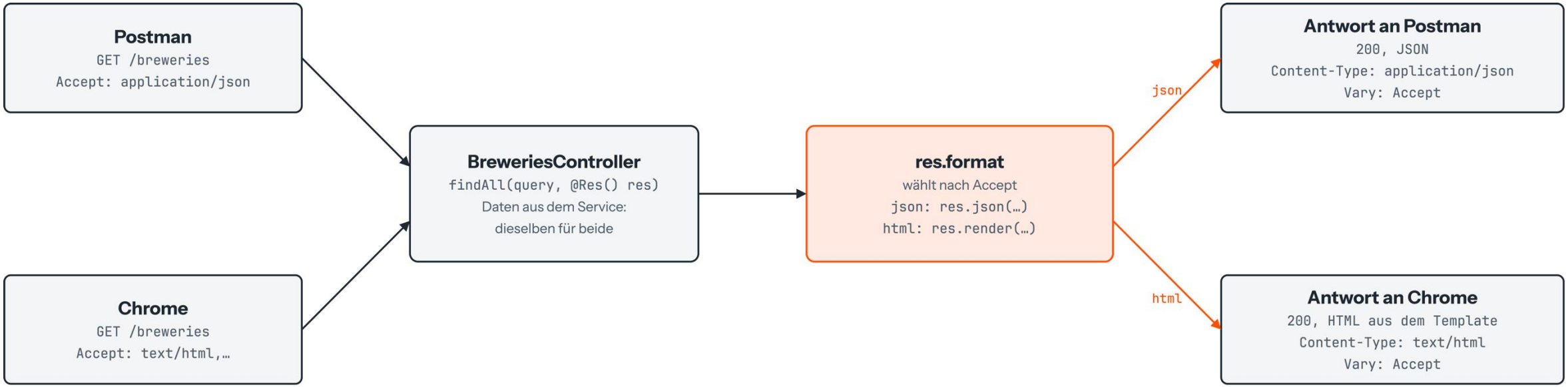
- {{name}} mit dem Wert Ulm
- Ausgabe: Ulm
- Chrome zeigt den Text Ulm

- {{{name}}} mit dem Wert Ulm
- Ausgabe: Ulm
- Chrome zeigt Ulm fett

Accept mit Gewichtung, Content-Type und Vary entscheiden über die Repräsentation

Angabe	Sendet	Bedeutung
Accept: text/html	Client	Liste der Formate, die der Client lesen kann
q=0.8 hinter einem Format in Accept	Client	Gewichtung von 0 bis 1; ohne q gilt 1
Content-Type: text/html; charset=utf-8	Server	Format, das der Server gewählt hat
Vary: Accept	Server	Die Antwort hängt von Accept ab; ein Cache speichert je Wert getrennt

res.format ruft je nach Accept den Zweig für JSON oder für HTML auf



Unter einem Pfad liefert dieselbe Ressource zwei Repräsentationen. Service und Regeln bleiben gleich; neu ist nur die Form der Antwort.

Vary: Accept sagt jedem Cache, dass die Antwort von Accept abhängt. Er speichert JSON und HTML getrennt.

Postman sendet GET /breweries mit Accept: application/json, Chrome dieselbe Anfrage mit Accept: text/html,…. Beide gehen an den BreweriesController mit findAll(query, @Res() res); die Daten aus dem Service sind für beide dieselben. Der Controller ruft das hervorgehobene res.format auf, das nach Accept wählt: json ruft res.json, html ruft res.render. Der Zweig json liefert Postman 200 mit JSON, Content-Type: application/json und Vary: Accept. Der Zweig html liefert Chrome 200 mit HTML aus dem Template, Content-Type: text/html und Vary: Accept. Darunter: Unter einem Pfad liefert dieselbe

Mit @Res() sendet die Methode die Antwort selbst

Bisher	Mit @Res()
NestJS sendet den Rückgabewert der Methode als JSON	die Methode ruft res.json oder res.render selbst auf
@ZodResponse prüft und filtert die Antwort	@ZodResponse entfällt, der Rückgabewert fehlt
ZodSerializerInterceptor entfernt Felder, die das Schema nicht kennt	niemand entfernt Felder; was der Service liefert, geht an den Client
Swagger UI liest das Schema aus @ZodResponse	Swagger UI liest es aus @ApiOkResponse

Schritt 1: Das Paket hbs bindet Handlebars an Express an

BASH

```
npm install hbs@4.4.0
```

Schritt 2: NestJS findet die Templates im Ordner views

TYPESCRIPT · SRC/MAIN.TS

```
// Templates aus views/, Engine Handlebars (hbs)
app.setBaseViewsDir(join(__dirname, '..', 'views'));
app.setViewEngine('hbs');
```

Schritt 3: Das Layout rahmt jede Seite

HTML · VIEWS/LAYOUT.HBS

```
<!doctype html>
<html lang="de">
 <head>
  <meta charset="utf-8">
  <title>Beer App</title>
 </head>
 <body>
  {{! Hier setzt hbs das Template der Seite ein }}
  {{{body}}}
 </body>
</html>
```

Schritt 4: Das Template zeigt eine Zeile je Brauerei

HTML · VIEWS/BREWERIES.HBS

```
<h1>Brauereien</h1>
<table>
 <tr><th>Name</th><th>Stadt</th><th>Land</th></tr>
 {{! Eine Zeile je Brauerei aus der Liste breweries }}
 {{#each breweries}}
  <tr><td>{{name}}</td><td>{{city}}</td><td>{{country}}</td></tr>
 {{/each}}
</table>
```

Schritt 5: res.format wählt nach Accept zwischen JSON und Template

TYPESCRIPT · SRC/BREWERIES/BREWERIES.CONTROLLER.TS

```
async findAll(
  @Query() query: BreweryQueryDto,
  // Response-Objekt von Express für res.format und res.render
  @Res() res: Response,
) {
  const breweries = await this.breweriesService.findAll(query);
  // Accept entscheidet: JSON wie bisher oder HTML aus dem Template
  res.format({
    json: () => res.json(breweries),
    html: () => res.render('breweries', { breweries }),
  });
}
```

Schritt 6: Zwei Anfragen auf denselben Pfad bekommen zwei Repräsentationen

BEISPIEL

- Accept: text/html ergibt 200 mit HTML-Tabelle.
- Accept: application/json ergibt 200 mit JSON wie bisher.
- Beide Antworten tragen Vary: Accept.
- Postman zeigt die Tabelle im Tab Preview.

```
GET /breweries, Accept: text/html → 200
Vary: Accept
Content-Type: text/html; charset=utf-8

<!doctype html>
…
<title>Beer App</title>
…
<tr><td>Privatbrauerei Schlumberger</td>…
```

Schritt 7: Chrome bekommt unter derselben URL die Tabelle

1. http://localhost:3000/breweries in Chrome öffnen.
2. Die Seite zeigt eine Tabelle mit vier Brauereien.
3. Im Network-Tab den Header Accept der Anfrage ansehen.
4. View Page Source zeigt das HTML, das der Server gesendet hat.

Einheit 16 · Dynamische serverseitige Erzeugung

FRAGE

Live-Frage: Postman sendet Accept: */*. Kommt JSON oder HTML?

GET http://localhost:3000/breweries ohne eigenen Header Accept

Auflösung: JSON, weil bei */* der erste Eintrag in res.format gewinnt

BEISPIEL

- */* passt auf json und auf html mit derselben Gewichtung.
- Bei gleicher Gewichtung nimmt res.format den ersten Eintrag.
- json steht zuerst: Postman und curl bekommen JSON wie bisher.
- Mit html zuerst bekäme jeder Client ohne Accept HTML.

```
GET /breweries, Accept: */* → 200
Vary: Accept
Content-Type: application/json; charset=utf-8

[{"id":1,"name":"Privatbrauerei Schlumberger",…}]
```

Einheit 16 · Dynamische serverseitige Erzeugung

FRAGE

Live-Frage: GET /breweries?sort=price mit Accept: text/html. Welche Antwort kommt?

sort kennt nur name und city

Auflösung: 400 als JSON, weil die Pipe ablehnt, bevor die Methode läuft

BEISPIEL

- Die ZodValidationPipe prüft die Query vor der Methode.
- res.format läuft nie, das Template auch nicht.
- Fehler beantwortet der Exception Filter von NestJS, immer als JSON.

```
GET /breweries?sort=price, Accept: text/html
→ 400 Bad Request
Content-Type: application/json; charset=utf-8

{"message":["sort: Invalid option: expected one of
 \"name\"|\"city\""],"error":"Bad Request",
 "statusCode":400}
```

Einheit 16 · Dynamische serverseitige Erzeugung

FRAGE

Live-Frage: GET /breweries mit Accept: application/xml. Welcher Statuscode?

Die Beer API kennt kein XML

Auflösung: 406 Not Acceptable, weil kein Eintrag in res.format zu Accept passt

BEISPIEL

- res.format findet weder json noch html für application/xml.
- Express antwortet selbst mit 406.
- Der Body hat kein Feld error wie die Fehler aus Einheit 8.
- Am Stand von Einheit 15 kam 200 mit JSON.

```
GET /breweries, Accept: application/xml
→ 406 Not Acceptable
Vary: Accept
Content-Type: application/json; charset=utf-8

{"statusCode":406,"message":"Not Acceptable"}
```

Fehlersuche: Chrome öffnet beim Laden der Brauereien ein Fenster mit „Beer“

BEISPIEL

- admin legt eine Brauerei an, Antwort 201.
- Chrome zeigt ein Fenster „Beer“.
- Danach steht eine Zeile mit leerem Namen in der Tabelle.
- TypeScript, ESLint und die Collection melden nichts.

```
POST /breweries mit dem Token von admin → 201
{"name":"<script>alert('Beer')</script>",
 "city":"Ulm","country":"Deutschland"}

http://localhost:3000/breweries in Chrome:
Fenster „Beer“
Zeile 5:  | Ulm | Deutschland
```

Fehlersuche: Drei Klammern setzen den gespeicherten Namen als Skript in die Seite

BEISPIEL

- Spur: Das JSON zeigt den Namen als Text, korrekt gespeichert.
- View Page Source zeigt script unverändert in der Zelle.
- Ursache: {{{name}}} maskiert nicht; Cross-Site Scripting.
- Beheben: {{name}}; Chrome zeigt den Namen als Text.

GET /breweries/5, Accept: application/json
{"id":5,"name":"<script>alert('Beer')</script>",…}

View Page Source:
<td><script>alert('Beer')</script></td>

Nach dem Beheben, in der Zelle:
<script>alert('Beer')</script>

Übung

Die Aufgaben stehen vollständig im Skript

Aufgaben dieser Einheit

ÜBUNG

1. Aufgabe 16.1: Die Bierliste als HTML-Seite
2. Aufgabe 16.2: Die Detailseite eines Biers

SELBSTKONTROLLE

- Beispiel zum Übernehmen: Skript 16.7 bis 16.12
- Aufgaben ab Skript 16.13, Hinweise ab 16.20
- Lösungsstand nach der Übung

Review: zwei Lösungen für die Seiten der Biere

- Wie kommt der Name der Brauerei in die Tabelle?
- Was verhindert, dass das JSON neue Felder bekommt?
- Wo steht der Strich für ein Bier ohne Bewertung: im Template oder im Code?
- Welche Stelle wirft die 404 für Bier 42, und warum als JSON?

MERKSATZ

Die Beer API liefert jetzt JSON und HTML. Einheit 17 blickt auf die Architektur der ganzen Beer API zurück.

Das solltest du jetzt können

- ✓ Eine Template-Engine in NestJS einrichten
- ✓ Ein Template mit Platzhaltern, Schleife und Layout schreiben
- ✓ Einen Endpunkt je nach Accept JSON oder HTML liefern lassen
- ✓ Erklären, warum zwei Klammern maskieren und wann drei Klammern gefährlich sind
- ✓ HTML-Antworten in Postman mit eigenen Tests prüfen

PlanB. Trusted Agentic Operator

DHBW Duale Hochschule Baden-Württemberg Heidenheim

WEB-PROGRAMMIERUNG · EINHEIT 17

Architektur-Rückblick und Ausblick

Einheit 17 · Architektur-Rückblick und Ausblick

FRAGE

Node 24 bekommt bis April 2028 Sicherheitsupdates. Was kostet die Beer API bis dahin?

Die Beer API ist fertig gebaut. Ab jetzt zahlen die regionalen Brauereien für Betrieb, Wartung und Weiterentwicklung. Was baut der Betreiber selbst, was kauft er?

Aufgabe: eine Leitfrage begründen und ein neues Projekt entwerfen

- Zu viert beantwortet ihr eine Leitfrage aus dem Semester mit Begründung.
- Zu zweit entwerft ihr den Fall Lagerinventur: Stories, JSON und Endpunkte.
- Ihr schreibt in dieser Einheit keinen Code; es gibt keine Story für die Beer API.

MERKSATZ

Es gibt keine richtige Antwort. Bewertet wird die Begründung mit Kosten, Nutzen und Risiko.

Betrieb, Wartung und Weiterentwicklung kosten über die Nutzungsdauer meist mehr als der Bau

Kostenart	Bedeutung	Beispiel Beer API
Bau	Stories umsetzen	Einheiten 5 bis 16
Betrieb	Server, Datenbank, Backup, Überwachung	heute ein Prozess und eine Datei beer.db
Wartung	Updates und Sicherheitslücken	Node 24 bis April 2028, NestJS 11
Support	Fragen und Fehler im Betrieb	eine Brauerei sieht einen falschen Durchschnitt
Weiterentwicklung	neue Stories über Jahre	Webseite, Mobile App, Bericht je Woche
Technische Schulden	Abkürzungen beim Bau verteuern jede spätere Änderung	Array als Speicher bis Einheit 12

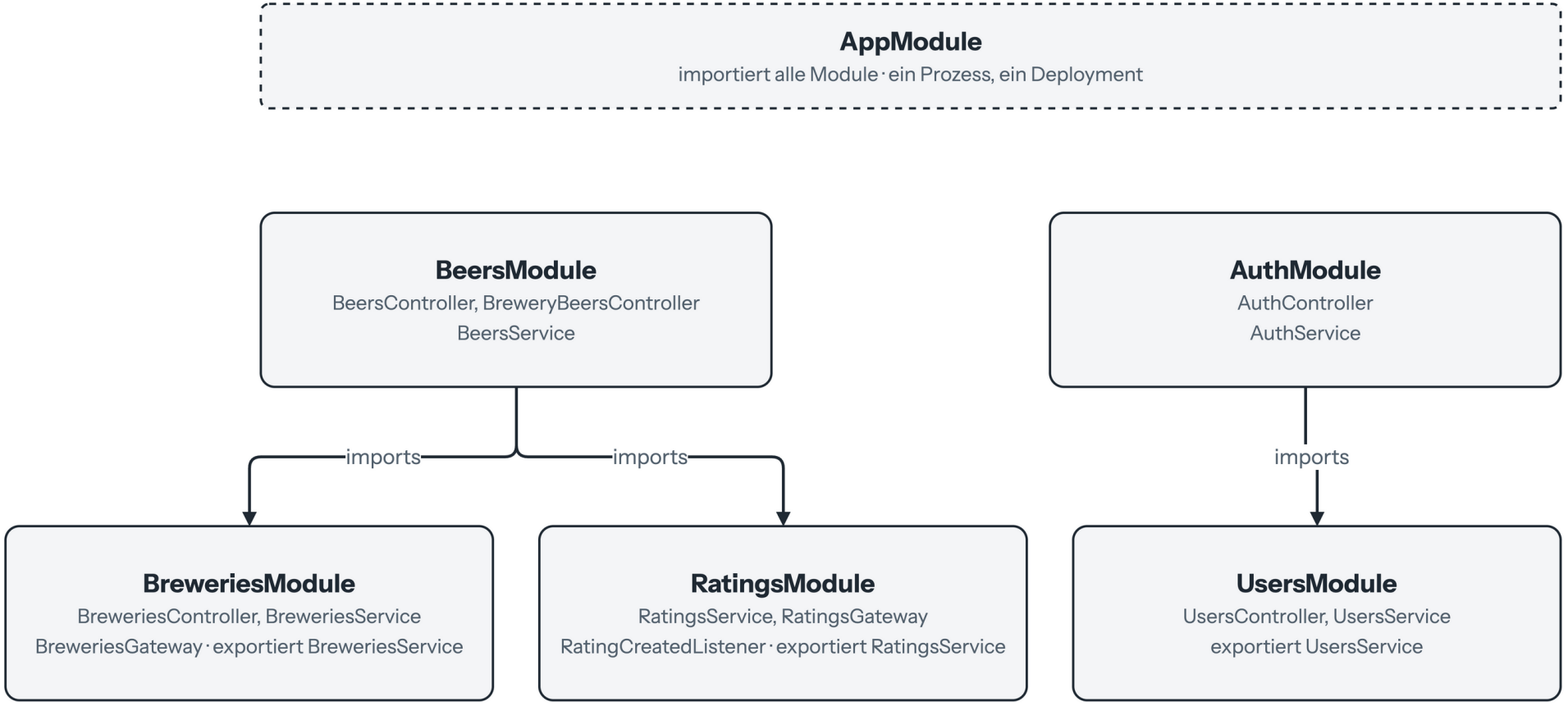
NestJS hat viel Arbeit abgenommen und bindet die Beer API an seine Versionen

- Abgenommen
  - Routing und Parameter
  - Dependency Injection
  - Pipes mit Zod, Exceptions mit Statuscode
  - Guards, Gateways, Templates
- Gekostet
  - NestJS 11, weil nestjs-zod NestJS 12 nicht unterstützt
  - Pakete in Version 12 nur als ES-Module
  - Jest braucht dafür eine eigene Option
  - jedes Update folgt dem Framework

Make or Buy: Vier Wege für die Beer App verteilen Arbeit und Risiko anders

Weg	Der Betreiber gibt ab	Bleibt beim Betreiber
Alles selbst bauen und betreiben	nichts	alle Kosten, volle Kontrolle
Anmeldung bei einem Identity Provider	Passwörter, Hashes, Login	Rollen, Regeln, Haftung für die Daten
Datenbank als Dienst mieten	Backup und Verfügbarkeit der Daten	monatliche Kosten, Bindung an den Anbieter
Fertiges Produkt anpassen	Datenpflege und API	Regeln für Bewertungen dort nachbauen

Die Beer API ist ein modularer Monolith: ein Prozess, fünf Module

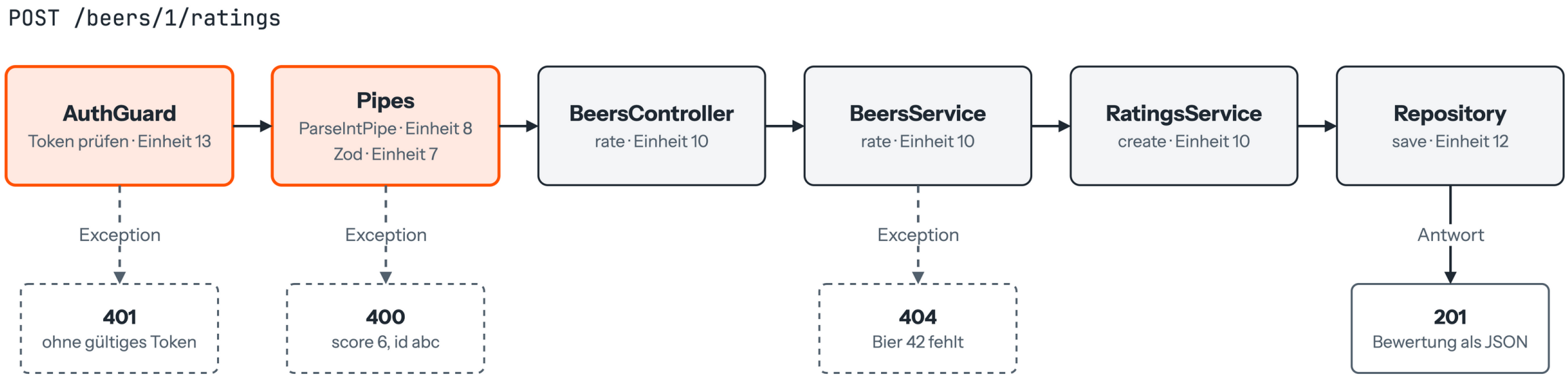


Fünf Module der Beer API: BeersModule importiert BreweriesModule und RatingsModule, AuthModule importiert UsersModule; AppMod ule importiert alle; ein Prozess, ein Deployment

Jede Anfrage an eine Ressource durchläuft dieselben drei Schichten

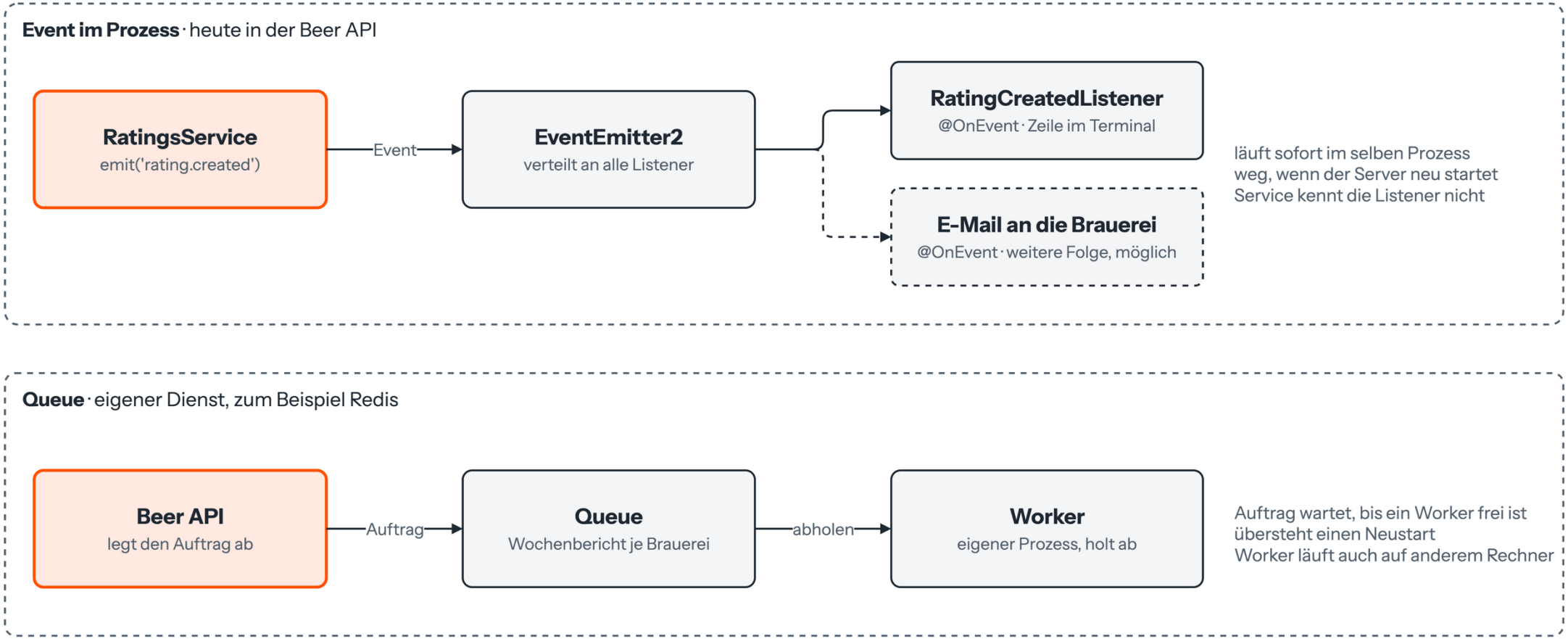
Schicht	Aufgabe	Beispiel in der Beer API	seit
Controller	HTTP lesen und beantworten	BeersController.rate liest Pfad, Body und Token	Einheit 5
Service	Regeln prüfen und anwenden	BeersService prüft, ob das Bier existiert	Einheit 6 und 10
Repository	Daten lesen und speichern	Repository von TypeORM speichert die Bewertung	Einheit 12

Guard und Pipes prüfen an der Grenze, bevor die Logik läuft



Weg von POST /beers/1/ratings: AuthGuard, Pipes, BeersController, BeersService, RatingsService, Repository; der AuthGuard bricht ohne gültiges Token mit 401 ab, die Pipes mit 400 bei score 6 oder id abc, der BeersService mit 404, wenn das Bier fehlt; sonst 201 mit der Bewertung als JSON

Ein Event entkoppelt die Folgen, eine Queue bewahrt Aufträge über einen Neustart



Oben Event im Prozess: RatingsService sendet rating.created an EventEmitter2, der an alle Listener verteilt: RatingCreatedListener schreibt eine Zeile ins Terminal, eine E-Mail an die Brauerei wäre eine weitere Folge. Läuft sofort im selben Prozess, ist nach einem Neustart weg, der Service kennt die Listener nicht. Unten Queue als eigener Dienst, zum Beispiel Redis: Die Beer API legt einen Auftrag wie den Wochenbericht je Brauerei ab, ein Worker als eigener Prozess holt ihn ab. Der Auftrag wartet, übersteht einen Neustart, der Worker läuft auch auf einem anderen Rechner.

Microservices machen aus Methodenaufrufen Netzwerkaufrufe

	Modularer Monolith	Microservices
Auslieferung	ein Prozess, ein Deployment	ein Prozess und ein Deployment je Dienst
Aufruf zwischen Teilen	Methode im selben Prozess	Anfrage über HTTP oder eine Queue
Daten	eine Datenbank	eigene Daten je Dienst
Ausfall	alles oder nichts	einzelne Dienste fallen aus, die anderen laufen
Passt zu	einem Team mit kleiner Last, wie beim Betreiber	vielen Teams, die unabhängig ausliefern

Ein Unit-Test prüft eine Regel ohne Server, ein e2e-Test das Zusammenspiel

- Unit-Test
  - ruft den Service direkt auf
  - ersetzt Datenbank und Gateway
  - läuft in Millisekunden
- End-to-End-Test (e2e)
  - sendet HTTP-Anfragen an den Server
  - nutzt die echte Datenbank
  - wie die Postman-Collections des Semesters

Deployment bringt die Beer API auf einen Server und hält sie dort am Laufen

- npm run build übersetzt nach dist/, npm run start:prod startet ohne Neuübersetzung.
- Secrets wie JWT_SECRET setzt die Umgebung des Servers, nie der Code.
- Ein Prozessmanager wie systemd startet den Prozess nach einem Absturz neu.
- Vor jedem Update: Datenbank sichern, dann npm run migration:run.
- Mit NODE_ENV=production legt die Migration keine Beispiel-User an.

Nach dem Server braucht die Beer App ihren ersten Client

- Die Beer App hat drei Teile: Beer API, Webseite und Mobile App.
- Webseite und App senden dieselben Anfragen wie Postman.
- Für die Webseite fehlt CORS mit der Origin der Webseite.
- Einheit 18 baut als Bonus diesen Client in Angular.

Fünf Schritte führen von einer Anforderung zu einem eigenen Projekt

1. Anforderungen lesen: Stories mit Akzeptanzkriterien und Regeln
2. Datenmodell als JSON entwerfen
3. Endpunkte als Tabelle: Methode, Pfad, Statuscodes
4. Gerüst anlegen: Basisprojekt und nest generate
5. In kleinen Schritten umsetzen: eine Story, dann prüfen

Demos: ein Event und ein Unit-Test an den Bewertungen

1. Paket für Events installieren und im AppModule eintragen
2. RatingsService meldet jede Bewertung als Event
3. Eine neue Klasse reagiert auf das Event
4. Jest einrichten und einen Test für den Durchschnitt schreiben
5. Rundung entfernen: Der Test wird rot

Demo 1, Schritt 1: Paket installieren, Module registrieren

TYPESCRIPT · SRC/APP.MODULE.TS

```
// Events im Prozess: emit im Service, @OnEvent in jeder Klasse
EventEmitterModule.forRoot(),
```

Demo 1, Schritt 2: Der Service meldet die Bewertung

TYPESCRIPT · SRC/RATINGS/RATINGS.SERVICE.TS

```
const rating = await this.ratingsRepository.save({
  beerId,
  userId,
  ...dto,
  createdAt: new Date().toISOString(),
});
// Meldet die Bewertung; wer darauf reagiert, weiß der Service nicht
const event = { beerId, score: dto.score };
this.eventEmitter.emit('rating.created', event);
```

Demo 1, Schritt 3: Eine Klasse reagiert auf das Event

TYPESCRIPT · SRC/RATINGS/RATING-CREATED.LISTENER.TS

```
// Läuft nach jeder Bewertung; RatingsService kennt diese Klasse nicht
@OnEvent('rating.created')
handle(event: RatingCreatedEvent) {
  this.logger.log(`Bier ${event.beerId} bewertet mit ${event.score}`);
}
```

Demo 1, Ergebnis: Postman bekommt 201, das Terminal meldet die Bewertung

BEISPIEL

- POST /beers/1/ratings mit dem Token von gast
- Body: score 5
- Postman: 201 wie bisher
- Terminal: eine neue Zeile

```
LOG [RatingCreatedListener] Bier 1 bewertet mit 5
```

Demo 2, Schritt 1: Jest einrichten

JSON · PACKAGE.JSON

```
"rootDir": "src",
"testRegex": ".*\\.spec\\.ts$",
"transform": {
  "^.+\\.(t|j)s$": "ts-jest"
},
"testEnvironment": "node"
```

Demo 2, Schritt 2: Der Test ersetzt die Abhängigkeiten

TYPESCRIPT · SRC/RATINGS/RATINGS.SERVICE.SPEC.TS

```
providers: [
  RatingsService,
  {
    provide: getRepositoryToken(Rating),
    useValue: {
      findBy: () =>
        Promise.resolve([{ score: 4 }, { score: 4 }, { score: 5 }]),
    },
  },
  { provide: RatingsGateway, useValue: {} },
  { provide: EventEmitter2, useValue: {} },
],
```

Demo 2, Schritt 3: Ein Test prüft den gerundeten Durchschnitt

TYPESCRIPT · SRC/RATINGS/RATINGS.SERVICE.SPEC.TS

```
// R5: Normal: 4, 4 und 5 → 4.3
it('rundet den Durchschnitt auf eine Nachkommastelle', async () => {
  expect(await service.averageFor(1)).toBe(4.3);
});
```

Demo 2, Schritt 4: Ohne Rundung wird der Test rot

- RatingsService › rundet den Durchschnitt auf eine Nachkommastelle

```
  Expected: 4.3
  Received: 4.333333333333333
  …
Tests:       1 failed, 1 total
…
```

Übung

Die Aufgaben stehen vollständig im Skript

Aufgaben dieser Einheit

ÜBUNG

1. Aufgabe 17.1: Eine Leitfrage zu viert begründen
2. Aufgabe 17.2: Die Lagerinventur entwerfen
3. Zusatz 17.3: Die Lagerinventur bauen oder kaufen

SELBSTKONTROLLE

- Vorgehen und Beispiel: Skript 17.13 und 17.14
- Aufgaben ab Skript 17.15, Hinweise ab 17.22

Review: zwei Entwürfe für die Lagerinventur

- Zwei Paare zeigen Stories, JSON und Endpunkt-Tabelle.
- Ist eine Buchung eine eigene Ressource oder eine Änderung am Artikel?
- Wer stößt die Meldung zum Mindestbestand an: Event, Queue oder Anfrage?
- Welche Entscheidung würdet ihr mit Total Cost of Ownership begründen?

MERKSATZ

Einheit 18 ist ein Bonus: Die Beer API bekommt ihren ersten Client in Angular.

Das solltest du jetzt können

- ✓ Die Architektur der Beer API mit Modulen, Schichten und Grenzen benennen
- ✓ Den Weg einer Anfrage von Guard bis Repository beschreiben
- ✓ Event, Queue und Microservice abgrenzen, Unit- und e2e-Test unterscheiden
- ✓ Für eine neue Aufgabe Stories, JSON und Endpunkt-Tabelle entwerfen
- ✓ Eine Entscheidung Make or Buy mit Total Cost of Ownership begründen